KLayout 用户手册

中文版

原著：Matthias Köfferlein

非官方中文译本 · 对应 KLayout 0.30.12

2026-10-24

关于本译本

版权与免责

KLayout 及其文档由 Matthias Köfferlein 开发，以 **GNU General Public License v3** 发布，完整许可见文末所附 `LICENSE`。本文件是其手册的非官方中文翻译，同样按 GPLv3 提供。译文仅供学习参考，遇有歧义或与程序行为不符之处，**以英文原版为准**。

本译本遵循以下约定：

- **术语保留英文原词**。例如 Layer View、Layer Set、Cell、Instance、PCell、Flatten、Partial Mode、DRC、LVS 等一律不译，首次出现时以“Cell（单元）”的形式括注中文，之后只用英文。
- **界面文字保留英文**。菜单项、按钮、对话框标题、输入框标签、命令、快捷键、文件格式、图层名与参数名均保持原样，以等宽字体或浅底框标出，例如 `Change Layer`、`View` > `Navigator`、`F3`。
- **代码与脚本不译**，注释亦保留原文，以免与程序实际行为脱节。
- 原文中的提示、注意事项放在浅底样式框中；界面元素说明单独成框并注明“原文保留”。

原文的章次结构保持不变，分为 KLayout Basics、Editing Functions、Advanced Topics、Design Rule Check (DRC) 与 Layout vs. Schematic (LVS) 五章。

目录

关于本译本 i

第一章 KLayout Basics 1

1.1 主窗口 . . . 1
1.1.1 左栏：层次浏览器、Library View 与导航器 . . . 1
1.1.2 中栏：画布 . . . 3
1.1.3 右栏：图层列表与图层样式控件 . . . 3
1.2 加载文件 . . . 4
1.3 管理面板与已加载版图 . . . 5
1.4 选择 Cell . . . 5
1.5 选择层次深度 . . . 5
1.6 配置 Cell 列表 . . . 6
1.7 library view . . . 6
1.8 隐藏 Cell . . . 6
1.9 缩放版图 . . . 7
1.10 全局旋转与翻转 . . . 7
1.11 回到之前的视图 . . . 7
1.12 为视图添加 bookmark . . . 7
1.12.1 可停靠的 bookmark 列表 . . . 8
1.13 带上下文下行进入 Cell . . . 8
1.14 图层列表（layer view） . . . 8
1.15 选择图层颜色 . . . 9
1.16 区分“有内容”与“空”图层 . . . 10
1.17 选择线型 . . . 10
1.18 图层动画 . . . 10
1.19 修改显示样式 . . . 10
1.20 图层的可见性 . . . 11
1.21 valid 与 invalid 图层 . . . 11
1.22 按层次组织图层 . . . 11
1.23 用标签页管理多套图层配置 . . . 11
1.24 从图层集中增删图层 . . . 12
1.25 视图变换与 property selector . . . 12

1.26 为单个图层或图层组指定层次层级 13
1.27 载入与保存图层集 15
1.28 截图 15
1.29 进行测量 15
1.30 标尺属性 16
1.31 添加图像 18
1.32 用 landmark 对齐图像 19
1.33 浏览 Shape 20
1.34 浏览 Instance 20
1.35 marker 浏览器 20
1.36 technology 管理 21
1.37 选择标尺、Shape 与 Instance 22
1.38 更多配置选项 23
1.39 撤销与重做 23
1.40 保存版图或其局部 23
1.41 保存与恢复会话 24
1.42 其它视图选项 24

第二章 Editing Functions **25**
2.1 编辑模式 25
2.1.1 取放（pick and drop）原理 25
2.1.2 编辑器基本选项 25
2.1.3 背景组合模式 26
2.1.4 选区 27
2.1.5 局部模式 27
2.2 编辑模式的基本原理 27
2.2.1 从零创建版图 27
2.2.2 新建图层 28
2.2.3 新建 Cell 28
2.2.4 画 Polygon 28
2.2.5 画 Box 29
2.2.6 画 Path 29
2.2.7 创建 Text 对象 30
2.2.8 放置 Cell Instance 30
2.2.9 移动选区 31
2.2.10 选区的其它变换 31
2.2.11 局部编辑 31

2.2.12 把 Shape 移到其它图层 . . . 32
2.2.13 其它图层操作 . . . 32
2.2.14 复制与粘贴选区 . . . 33
2.2.15 删除 Cell . . . 33
2.2.16 重命名 Cell . . . 33
2.2.17 复制与粘贴 Cell . . . 33
2.3 基本编辑操作 . . . 34
2.3.1 整版图变换 . . . 34
2.3.2 搜索与替换 . . . 34
2.3.3 Find（查找） . . . 35
2.3.4 Delete（删除） . . . 36
2.3.5 Replace（替换） . . . 36
2.3.6 Custom queries（自定义查询） . . . 36
2.3.7 层次操作：打平 Instance、由选区建 Cell、在层次中上移 . . . 36
2.3.8 生成裁剪片 . . . 37
2.3.9 打平 Cell . . . 37
2.3.10 拆解阵列 . . . 37
2.3.11 PCell 操作 . . . 38
2.3.12 图层布尔运算 . . . 38
2.3.13 图层 sizing . . . 39
2.3.14 按 Shape 的布尔运算 . . . 39
2.3.15 按 Shape 的 sizing . . . 40
2.3.16 对象对齐 . . . 40
2.3.17 圆角处理 . . . 41
2.3.18 调整 Cell 原点 . . . 42
2.3.19 创建 Cell variant . . . 42
2.4 高级编辑操作 . . . 42

第三章 Advanced Topics **45**
3.1 XOR 工具 . . . 45
3.2 Diff 工具 . . . 45
3.3 填充（tiling）工具 . . . 46
3.4 导入 Gerber PCB 文件 . . . 47
3.4.1 导入对话框 . . . 48
3.4.2 叠层流程 . . . 49
3.4.3 自由图层映射流程 . . . 50
3.4.4 通用选项 . . . 53

3.5 导入其它版图文件 55
3.6 net 追踪功能 56
3.6.1 单网络追踪（Single net tracing） 56
3.6.2 追踪全部网络（Tracing all nets） 57

第四章 Design Rule Check (DRC) 58
4.1 DRC 基础 58
4.1.1 基本脚本 58
4.1.2 安装与运行脚本 60
4.1.3 DRC 豁免流程（DRC waiving） 60
4.1.4 把 KLayout 当作独立的 DRC 引擎 61
4.2 DRC runset 编写 62
4.2.1 runset 基础 62
4.2.2 包含其它文件 62
4.2.3 输入与输出 63
4.2.4 尺寸规格 64
4.2.5 对象与方法 65
4.2.6 Polygon 层与 edge 层 66
4.2.7 文字集合 67
4.2.8 edge pair 与 edge pair 集合 67
4.2.9 raw 与 clean 图层模式 68
4.2.10 屏蔽 69
4.2.11 通用 DRC 69
4.2.12 日志与输出详细程度 70
4.2.13 tiling 选项 70
4.2.14 层次模式 71
4.2.15 DRC 与用户属性 72

第五章 Layout vs. Schematic (LVS) 75
5.1 LVS 概览 75
5.1.1 LVS 脚本的基本用法 75
5.1.2 KLayout 的 LVS 实现 75
5.1.3 术语 76
5.2 LVS 入门 77
5.2.1 LVS 入门 77
5.2.2 LVS 脚本示例 79
5.2.3 LVS 脚本的结构 80

5.2.4 带下压二极管的反相器 . . . 83
5.3 LVS 器件 . . . 85
5.3.1 器件提取器与器件类 . . . 86
5.4 LVS 器件类 . . . 86
5.4.1 Resistor（电阻） . . . 86
5.4.2 带 bulk terminal 的电阻 . . . 87
5.4.3 Capacitor（电容） . . . 87
5.4.4 带 bulk terminal 的电容 . . . 88
5.4.5 Diode（二极管） . . . 88
5.4.6 MOS 晶体管 . . . 88
5.4.7 带 bulk 的 MOS 晶体管 . . . 89
5.4.8 双极晶体管 . . . 90
5.4.9 带衬底的双极晶体管 . . . 91
5.5 LVS 器件提取器 . . . 91
5.5.1 电阻提取器（resistor 与 resistor_with_bulk） . . . 92
5.5.2 电容提取器（capacitor 与 capacitor_with_bulk） . . . 93
5.5.3 二极管提取器（diode） . . . 94
5.5.4 MOS 晶体管提取器（mos3 与 mos4） . . . 95
5.5.5 扩散 MOS 晶体管提取器（dmos3 与 dmos4） . . . 97
5.5.6 双极晶体管提取器（bjt3 与 bjt4） . . . 97
5.5.7 器件提取器与器件类 . . . 99
5.6 LVS 输入输出 . . . 101
5.6.1 写出 netlist . . . 101
5.6.2 读取 netlist . . . 102
5.6.3 Layout-to-Netlist 数据库 / 报告 . . . 103
5.6.4 Layout-vs-Schematic 数据库 / 报告 . . . 104
5.7 LVS 连接关系 . . . 104
5.7.1 层内连接与层间连接 . . . 104
5.7.2 全局连接 . . . 105
5.7.3 隐式连接 . . . 105
5.7.4 显式连接 . . . 106
5.7.5 软连接 . . . 108
5.8 LVS 比对 . . . 110
5.8.1 net 等同提示 . . . 111
5.8.2 circuit 等同提示 . . . 111
5.8.3 器件类等同提示 . . . 111
5.8.4 容差 . . . 112

5.8.5 忽略参数 . . . 113
5.8.6 启用与禁用参数 . . . 113
5.8.7 pin 互换 . . . 113
5.8.8 消去电容与电阻 . . . 113
5.8.9 检查 pin 标注 . . . 114
5.8.10 比对与网表层次 . . . 114
5.8.11 比对算法的工作方式 . . . 115
5.9 LVS 网表调整 . . . 116
5.9.1 生成顶层 pin . . . 116
5.9.2 器件合并 . . . 116
5.9.3 打平 circuit（消去层次） . . . 117
5.9.4 自动打平 circuit（网表对齐） . . . 117
5.9.5 黑盒化（circuit 抽象） . . . 117
5.9.6 对称节点合并 . . . 118
5.9.7 分裂栅 . . . 118
5.9.8 清除冗余 . . . 119
5.9.9 归一化封装（simplify） . . . 119

附录 A 许可证（GNU General Public License v3） 120

Chapter 1

KLayout Basics

1.1 主窗口

主窗口分为三个部分：左侧是层次浏览器与导航器，中间是画布，右侧是图层列表及图层样式控件。各组件的位置可以重新排列，因此上述布局只是默认布局。拖动某组件的标题栏即可把它移到别处，也可以把它从主窗口中拖出，成为一个独立的浮动窗口。

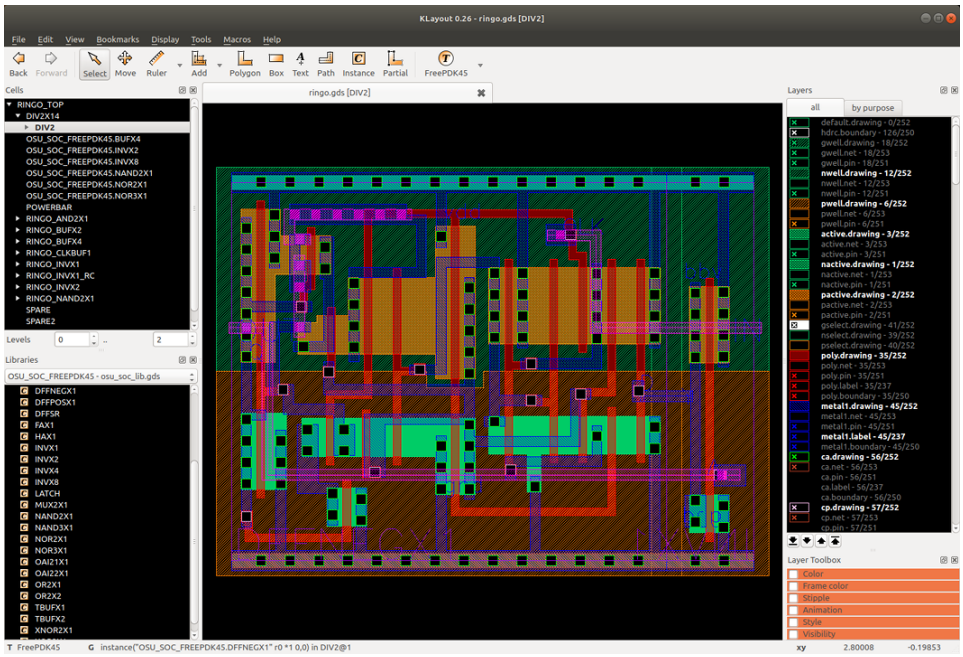


图 1.1: KLayout 主窗口的默认布局

1.1.1 左栏：层次浏览器、Library View 与导航器

左侧面板是层次浏览器，用来显示 Cell 的层次结构；还可以在其中显示导航器，后者给出整个版图的缩略总览。

在层次浏览器中，Cell 节点可以展开以显示其子节点。current cell（当前单元）是指当前显示在中间画布里的那个 Cell，以粗体绘制。可以选中一个或多个 Cell，各种操作就作用于被选中的这些 Cell。context cell（上下文单元）则是绘制操作真正落在其上的 active cell（活动单元）；上下文通常与当前单元相同，但向层次内部下行之后，当前 Cell 的子 Cell 也可以成为上下文单元。它以下划线字体显示。

在下图的例子中，"TOPTOP_BIG" 处于选中状态，"TOPTOP" 是当前 Cell，"TOP" 则是上下文 Cell：

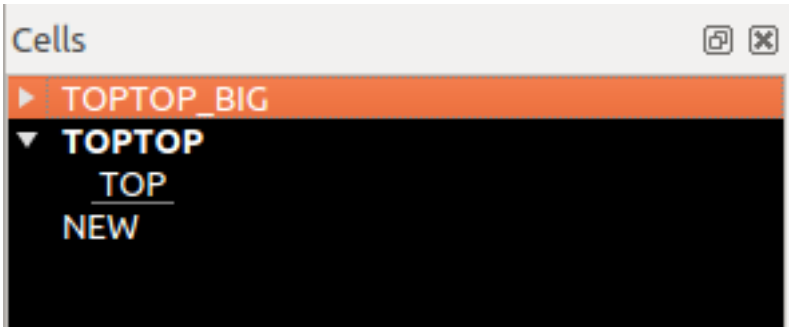


图 1.2: 层次面板中选中、当前与上下文三种状态

层次浏览器下方的子面板是 library view。该视图显示系统中已注册的库及其内容：

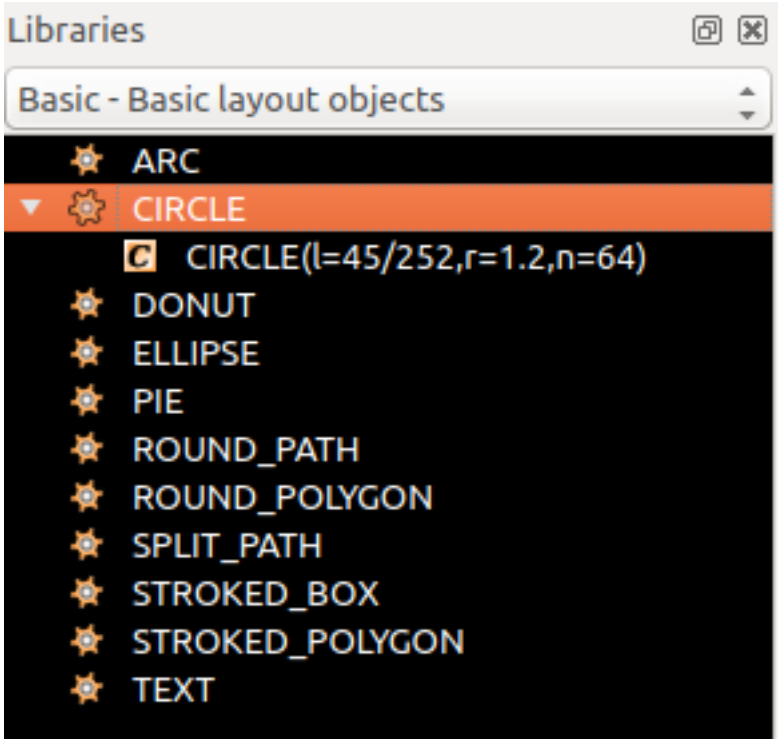


图 1.3: library view

library view 用于浏览并放置库中的 Cell，这些 Cell 可以是普通 Cell，也可以是 PCell。详见 1.7。

导航器默认不可见，勾选 `View` 菜单里的 `Navigator` 菜单项即可启用。导航器给出整个 Cell 的缩略图，并用一个方框标出中间画布当前显示的区域：

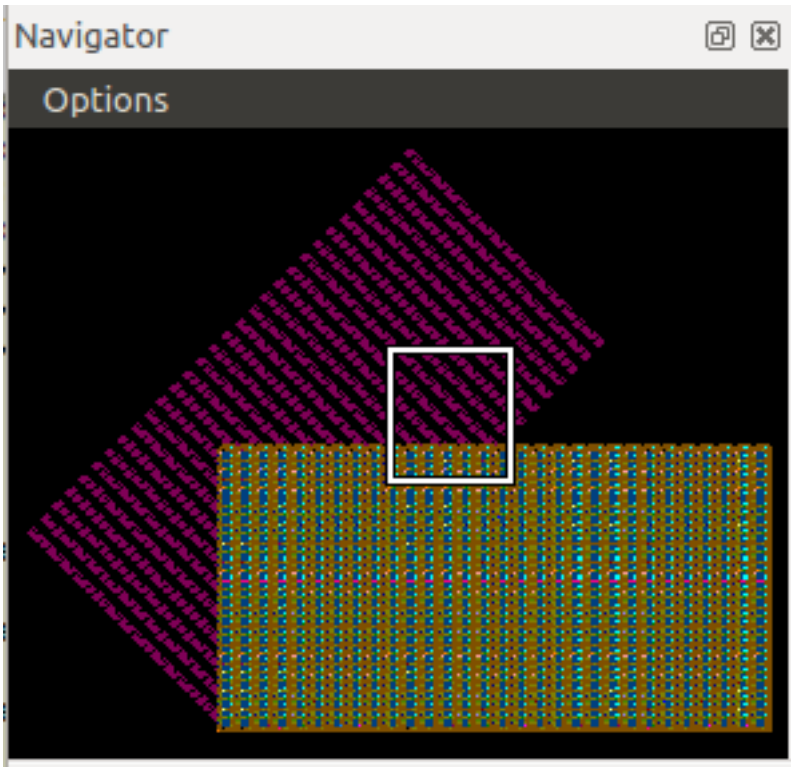


图 1.4: 导航器：缩略图与当前视区方框

1.1.2 中栏：画布

中间面板才是真正的画布，版图就绘制在这里。要放大，按住鼠标右键拖出一个矩形，该矩形即为新的显示区域。要选择对象，用鼠标左键单击，或按住左键拖出选择矩形。这里还提供大量编辑与显示相关的功能，例如添加标尺、叠加背景图片等。

1.1.3 右栏：图层列表与图层样式控件

右侧面板显示图层列表（或称图层树）。默认是普通图层列表，但配置方式很多：分组、设置样式、加标签页以便在不同配置之间快速切换等。

图层树决定了版图画什么、怎么画。表中的条目可以随意删除或配置，这不会影响版图数据本身。本质上，图层列表可以完全独立于版图：既可以把版图中的某些图层省略不显示，也可以加入版图里根本不存在的（空）图层。这种用法下，图层列表通常反映的是某套工艺所定义的一批绘制图层。对于本身没有样式的版图文件，图层列表正好提供了带样式的显示视图。图层可以重新排序，从而改变绘制优先级：排在最上方的图层会先画，其余图层随后覆盖其上，所以靠前的图层反而容易被后面的图层挡住。

图层上还可以挂接修改算子（modification operator），例如对绘制的版图做几何变换（平移、旋转、缩放、镜像），或仅在用户属性满足特定条件时才绘制该版图。

图层列表下方是一组控制面板，默认处于折叠状态，点击标题栏即可展开。这些

控件构成 Layer Toolbox，用来修改图层样式：工具箱中选定的样式会作用于图层列表中选中的那些图层。

多个版图可以同时显示：既可以叠加显示，也可以放在各自独立的视图中；后者会在主窗口顶部多出一个标签栏，切换标签即可切换版图。不同版图的图层会以带标注的形式出现在图层列表中，例如"1/0@1" 表示第一个版图的第 1 层、datatype 0，"1/0@2" 表示第二个版图的同一层。

1.2 加载文件

在 File 菜单中选择：

Open：关闭当前视图，并加载一个新版图来取代当前已加载的版图

Open in Same Panel：在当前已加载的版图之外，再于同一面板中加载一个新版图

Open in New Panel：在新视图中加载一个新版图

无论走哪条路径，都会弹出文件选择对话框，在其中挑好文件即可。选好文件并点击 Ok 后，文件即被加载。

程序会自动判断文件类型。目前支持 OASIS、GDS2、DXF、CIF、LEF/DEF 以及 GDS2 的文本版本。Gerber PCB 数据经过一些准备后也能读取（见下文）。若文件是 gzip/zlib 压缩的，会自动解压。

文件读取器（loader）可以通过 reader option 页设置若干选项。要打开读取器选项对话框，从 File 菜单选择 Reader Options。该对话框对所有 Open 操作生效，可指定的选项例如：

- 把读取范围限制在若干图层上，其余图层不读取。
- 禁止读取 Text 对象。Text 对象对掩模而言不携带几何信息，可以借此丢弃。
- 禁止读取 user properties（用户属性）。若不需要属性，禁用后可以减少内存占用。
- 若干 GDS 专有选项，主要用于控制与其它工具的兼容程度。
- 其它格式也可能提供各自的选项。尤其是 DXF 或 LEF/DEF 这类信息丰富的格式，配置方式很多。对话框用不同标签页分别呈现不同格式或格式组的选项。

使用 File > Reload 可以把当前已加载的文件重新从磁盘读入。通常并不需要这么做，因为 KLayout 会自动检查文件是否发生变化，并提示是否加载。

从 File 菜单的 Open Recent 列表中挑选文件，即可重新加载之前用过的文件。

也可以在命令行中把文件交给 KLayout，程序会自动加载。可以指定多个文件，默认分别显示在不同页面中；若要把多个文件加载到同一页面，在命令行加一个 -s 选项。命令行中同样可以给出 http: 或 https: 地址，此时 KLayout 会从该 URL 下载文件。

文件和 URL 可以直接拖放到 KLayout 主窗口上，KLayout 会加载并显示它们。

版图文件可以与 technology 关联。technology 允许把版图与附加数据（库、宏、net tracer 设置、layer properties 等）绑定在一起。详见相关章节。

1.3 管理面板与已加载版图

从 File 菜单选择 Close，可以移除某个面板中的版图；若面板中已不再加载任何版图，面板也随之关闭。当前面板中加载了多个版图时会弹出对话框，可从中选择一个或多个版图关闭。Close All 会关闭所有面板。

从 File 菜单选择 Clone 可以复制面板。新面板是当前面板的精确副本，当前面板与新面板都是同一个版图的视图，因此版图在内存中只保留一份。

选择 Pull In Other Layout 可以把已加载到其它面板中的版图合并到当前面板里。KLayout 允许同一版图显示在多个面板中，既可以单独以不同配置显示，也可以与其它版图一起显示。Pull In Other Layout 让某个面板改为显示另一个面板中的版图，因此它正好是“从含多个版图的面板中关闭一个版图”的逆操作。

1.4 选择 Cell

要显示某个 Cell，在左侧层次浏览器的 Cell 层次中选中它，然后在 Cell 树上单击右键调出上下文菜单，选择 Show as top；或者直接用鼠标中键单击该 Cell。

要按名字选择 Cell，从 Display 菜单选择 Select Cell。弹出的对话框既可以输入名字查找，也可以从按字母排序的列表中挑选。此外，该对话框还允许在 Cell 树中导航，跳到某个子 Cell 或父 Cell。

1.5 选择层次深度

默认只显示所选 Cell 的边界框，这相当于显示的层次层级数为零。若多选一级（Levels 0 到 1），当前 Cell 的各子 Cell 内容会被画出，而再下一层（子 Cell 的子 Cell）则以方框表示。层级数增大后，子 Cell 的细节会画得越来越多，直到当前 Cell 以下的所有 Cell 都以细节形式画出。

要选择更多层次层级：

从 Display 菜单选择 Full Hierarchy，或按 * 键，显示所有层次层级

从 Display 菜单选择 Box Only，或按 0 键，只显示边界框（默认）

从 Display 菜单选择 Top Level Only，或按 1 键，只显示顶层元素

从 Display 菜单选择 Increment Hierarchy，或按 + 键，多显示一级层次

从 Display 菜单选择 Decrement Hierarchy，或按 - 键，少显示一级层次

使用 Cell 列表下方的层次层级输入框，调整当前的最小或最大层级

1.6 配置 Cell 列表

Cell 列表提供两种模式：树状视图（默认）与平坦 Cell 列表。要切换到平坦模式，在 Cell 面板的上下文菜单中勾选 `Flat Cell List` 选项。

若加载了多个版图，各版图的 Cell 列表会分开显示。Cell 列表上方的下拉框用于选择当前要显示的 Cell 树。另一种可选模式是把各 Cell 列表并排显示在 Cell 树面板中；该模式在 Cell 面板的上下文菜单中选 `Split Mode` 开启。在 Split Mode 下点击标题即可选中对应的 Cell 树。当前 Cell 树在某些场合会起作用，例如 `Edit` 菜单中 `Layout` 子菜单里的版图操作。

此外还提供三种排序方式：按名字字母序、按 Cell 尺寸（边界框面积）升序或降序。Cell 尺寸大体上反映设计的规模层次：库与叶 Cell 通常很小，宏模块通常较大。按 Cell 尺寸升序排列时，叶 Cell 会显示在最前面。要改变排序，在 Cell 面板的上下文菜单的 `Sorting` 子菜单里勾选对应选项。

1.7 library view

除 Cell 列表之外，还提供一个 library view 作为版图面板的辅助视图。这个子面板显示可用的库，并允许浏览库中的 Cell 与 PCell。

默认情况下 library view 位于 Cell 树下方。拖动各视图的标题栏即可重新排列，把它们停靠到主窗口的其它位置。要把窗口布局恢复为默认配置，使用 `View` 菜单中的 `Restore Window`。

library view 一次显示一个库中的 Cell 与 PCell。要选择库，在 library view 顶部的选择框里挑选。

PCell 在 library view 中以一个小齿轮图标表示。若某个 PCell 已被实例化，正在使用中的各个 variant 会作为条目列在该 PCell 条目之下。

在编辑模式下，把 Cell 从 library view 拖到版图画布上即可方便地放置。拖动的是某个 PCell variant 时，会新建该 variant 的又一份 Instance；拖动的是 PCell 主条目时，放下时 KLayout 会弹出 PCell 参数定义对话框。

1.8 隐藏 Cell

与显示多少层次层级无关，Cell 还可以被隐藏。被隐藏的 Cell 不再显示内容，只显示其边界框。要隐藏 Cell，在 Cell 列表中选中它，然后在上下文菜单中选 `Hide`；要重新显示，选 `Show`；要让所有 Cell 都重新可见，选 `Show All`。

1.9 缩放版图

在版图画布中按住鼠标右键框出要缩放的区域。

按住键，把框拖到合适的位置，再松开。

要放大，把框往右下拖；要缩小，把框往左上拖。要换一个新的中心点，用鼠标右键单击新的中心位置即可。

此外，以下功能也可以通过菜单或热键使用：

用方向键或 Display 菜单中的对应功能向左、右、上、下平移

按 F2，或从 Display 菜单选择 Zoom Fit，把选中的整个 Cell 充满窗口

按 Enter 或 Shift+Enter，或从 Display 菜单选择 Zoom In/ Zoom Out，按固定步长放大或缩小

若鼠标带滚轮，也可以用滚轮缩放。缩放时鼠标当前位置保持不动，其周围的版图被放大或缩小

按住 Shift 的同时用鼠标右键拖动，可以像近年地图服务网页那样拖拽版图

1.10 全局旋转与翻转

KLayout 提供翻转或旋转整个窗口的选项。例如查看芯片的镜像版掩模数据时，这个选项很有用。

要使用该选项，从 Display 菜单的 Global Transformation 子菜单中挑一个所需的变换。

1.11 回到之前的视图

从 Display 菜单选择 Last State 可回到上一个显示的窗口状态；选择 Next State 则再切回较新的状态。

1.12 为视图添加 bookmark

视图（窗口与 Cell 的组合）可以存成 bookmark 以便日后调回。从 Bookmarks 菜单选择 Bookmark This View，会要求给 bookmark 起个名字。之后该 bookmark 就会出现在 Goto Bookmarks 列表中。

已定义的 bookmark 列表可用 Bookmarks 菜单中的相应功能载入或保存：载入用 Load Bookmarks，保存用 Save Bookmarks。

bookmark 列表也可以做成可停靠的工具窗口：勾选 View/Bookmark List 选项即可启用。bookmark 列表默认显示在版图视图的右下角。

1.12.1 可停靠的 bookmark 列表

要从可停靠的 bookmark 列表跳转到某个 bookmark，双击该条目即可。在上下文菜单（右键单击）中可以选择以下功能：

- `Follow Selection`：勾选后，选中的 bookmark 会立即改变视图。配合该选项可以用方向键在 bookmark 列表中浏览，视图自动随之更新。
- `Manage bookmarks`：打开 bookmark 管理对话框（与`Bookmarks`菜单中的同名功能一致）。
- `Load bookmarks`与`Save bookmarks`：从文件载入或把 bookmark 保存到文件（与`Bookmarks`菜单中的同名功能一致）。

1.13 带上下文下行进入 Cell

一个 Cell 可以孤立显示（当它是当前 Cell 时的默认方式）、嵌入式显示（作为当前 Cell 的子 Cell），或者在某个直接或间接父 Cell 的上下文中作为当前 Cell 显示。最后这种模式下，该 Cell 被高亮，而上下文 Cell 以变暗或另一种用户自定义的颜色显示。

要在上下文中高亮某个 Cell，先选好上下文 Cell，然后在要显示的那个 Cell 中选中一个 Shape 或 Cell Instance，再从`Display`菜单选择`Descend`或按`Ctrl+D`。此时，通向所选 Shape 的第一个子 Cell 被高亮，而父 Cell（即原先的当前 Cell）周围的 Shape 以变暗的颜色显示。反复选`Descend`可继续向层次内部下行，直到所选 Shape 或 Instance 落在当前 Cell 这一层。在 Cell 树中，当前 Cell 以下划线显示，上下文 Cell 照旧以粗体显示。

该操作的逆操作是`Display`菜单中的`Ascend`（或`Ctrl+A`）。

上下文版图的显示方式可以在 setup 对话框的`Background`标签页中调整。

1.14 图层列表（layer view）

layer view 是 KLayout 中的一个重要概念。默认情况下 KLayout 把版图的图层显示在主窗口右侧的列表里，但这个列表并不直接等同于版图数据库中的图层，它是一组 view 的集合。所谓 view，就是对“显示什么、怎么显示”的描述。

本质上，layer 列表中的每个条目都是指向数据库中某个图层的指针，外加一份“如何绘制该图层上的 Shape”的描述，即 layer properties。

这个指针就是 layer view 的 source（来源）。通常是 GDS 的层号与 datatype，也可以是图层名（例如 DXF 中的情况）。还存在抽象来源（例如 Cell 边界），并且 source 可以带 selector 或 modifier。selector 用来只显示具有特定 user properties 的 Shape，或只显示来自特定层次层级的 Shape；modifier 则在绘制之前对 Shape 做变换之类的事。source 由 source specification 定义，即一个描述数据库图层、selector 与 modifier

的字符串。最简单的 source 字符串是 `1/0`，表示 GDS 第 1 层、datatype 0，不带任何 selector 与 modifier。要修改 source，使用 layer 列表上下文菜单中的 `Change Source`。

source specification 字符串的更多用法与细节见相关章节。

除 source 之外，每个 layer 条目还有一个显示名（display name），即一段供用户阅读的任意文字。默认情况下若没有设置显示名，就显示该 layer 的 source。要修改显示名，使用 layer 列表上下文菜单中的 `Rename`。

当然，layer view 还有许多选项用于指定绘制样式、动画与装饰。

把 view 与数据库图层分离这一设计带来了若干有意思的可能性：

- layer view 可以通过 source 中的 `@1`、`@2`等标注引用多版图视图中的各个版图，因此一个 layer 列表里可以混合多个版图。
- 列表中可以存在数据库中并不需要的图层，这样的图层显示为空。这一点很重要：在 GDS 中空图层与不存在的图层是等价的，而你可能仍希望它在 layer 列表中出现，view 正好提供了这个选项。
- 反过来，若没有对应的 layer view，数据库中的图层也可以不出现在 layer 列表中。这样就能把辅助或调试用的图层从列表中省掉。若需要看到全部图层，可以使用 wildcard specification。
- 可以有多个标签页，为同一批版图提供多套视图，也就是另一组 layer view。
- 图层的分组、排序等操作都只是对 view 的操作，不涉及数据库改动。

另一方面，这套设计初看有些反直觉。以下是几点提醒：

- 重命名图层并不改变 source。若你把某图层改名为 `1/0`之类，很可能只是在骗自己，以为它就是第 1 层、datatype 0。
- 改变某个 layer view 的 source 同样不改变数据库，只是改了指针。要修改数据库中图层的层级信息，使用 `Edit` > `Layer` > `Edit Layer Specification`。
- 从 layer 列表中删除图层不会把该图层从数据库中删除。要真正删除，使用 `Edit` > `Layer` > `Delete Layer`。
- 新增图层时并不会立刻在数据库中创建该层，只有当你在这个层上画了东西之后，它才会在数据库中生成。

1.15 选择图层颜色

先选中要改颜色的图层，再打开右侧 layer 面板中的取色面板。若取色面板不可见，勾选 `Color`标题栏上的小复选框把它展开。

要改颜色，直接点击所需的颜色即可。若要选的颜色不在列表中，点击 `More Colors` 按钮，会弹出取色对话框。

若只想改 Shape 外框的颜色而保持填充色不变，使用 `Frame Color`取色面板。

图层还可以“变暗”或“变亮”，即把颜色调深或调浅，使其与背景的对比减弱。做法是在 color 面板中选 `Dark` 或 `Bright`；每按一次，颜色就更深或更浅一档。用 `Reset` 按钮可以恢复原来的明暗。

1.16 区分“有内容”与“空”图层

在某些应用中，layer 列表会变得非常长，盯住其中重要的图层颇为费劲。KLayout 在这方面提供两种支持：它会检查某个图层上是否载有信息，若为空，就在 layer 列表中以不同方式显示该图层。

判断图层是否含内容有两种方式：一是若当前 Cell 在该图层上没有任何 Shape，就认为该层为空；二是若当前视图中没有出现任何 Shape（更准确地说，没有任何 Shape 的边界框与当前视图矩形相交），就认为该层为空。后一种方式可在 layer 列表的上下文菜单中用 `Test For Shapes in View` 选项启用。

判定为空的图层要么变灰显示，要么干脆不显示。后者能让 layer 列表保持简短，可在 layer 列表的上下文菜单中选择 `Hide Empty Layers`。

1.17 选择线型

line style 控制 Shape 的外框如何绘制，默认是实线。可用的线型有虚线、点划线以及自定义样式。要选定某种线型，先选中要改的图层，再在 `Style` 面板中挑线型。

用 `Custom Style` 按钮可以新建样式，会弹出线型编辑器，可新建与编辑样式；预定义图案不可修改。要应用新样式，在样式选择面板中选 `More`，再从列表中挑新样式。自定义样式会随图层的显示属性一起保存。

1.18 图层动画

图层可以做成动画，即闪烁，或让填充图案滚动。先选中要改的图层，再在 `Animation` 面板中选择动画方式。闪烁模式下可选两个相位：`Blink` 与 `/Blink`；给两个图层选不同的相位，它们就会交替出现。

1.19 修改显示样式

先选中要施加改动的图层，再用 `Style` 面板上的宽度按钮改变元素外框的线宽：`0px` 不画线，`1px` 画一个像素宽的线（默认），`2px` 依次加粗，依此类推。

`Simple` 是常规绘制方式；`Marked` 则在元素的每个顶点处画一个十字。十字尺寸恒定，因此在元素本身会缩成一个像素的宏大尺度下，Shape 依然可见。

图层还可以配置成在矩形上画对角十字。要启用这种样式，在 `Styles` 工具箱中选 `Cross`；要关闭则选 `No Cross`。十字绘制叠加在填充图案之上，且只对矩形生效。

1.20 图层的可见性

在 Visibility 面板中选择 Hide 选项即可把选中的图层隐藏；选 Show 则重新可见。

另外，在 layer 列表中双击一个可见图层，也能切换该图层的可见性。

要让图层“透明”（即让其它图层透出来），在 Visibility 面板中选 Transp.；要恢复不透明，选 Opaque（默认）。

1.21 valid 与 invalid 图层

从 0.23 版起，KLayout 提供“invalid”图层：invalid 图层可以显示，但不参与选中（selection）与吸附（snapping），只起到背景绘制的作用。

在图层上下文菜单中选择 Make invalid 即可把图层设为 invalid；选 Make valid 可恢复为 valid。invalid 图层在 layer 列表中以一个小写“x”标记。

1.22 按层次组织图层

图层可以按层次组织，例如把某些图层归成一组。在 layer 列表的上下文菜单（右键单击 layer 列表）中选择 Group，选中的图层就会被一个代表这些图层的树节点取代。点击该树节点即可展开或折叠这一组。

归组之后，只要在组代表节点上双击一次就能隐藏或显示整组图层。组代表节点还控制组内图层的外观：若给组代表指定了颜色或样式，它会覆盖组内所有图层的相应样式，例如借此一次性改变组内图层的颜色。要取消组代表的颜色覆盖，把颜色设为 None。

要拆散一组，选中组代表节点，然后在上下文菜单中选 Ungroup。

KLayout 还提供多种自动归组方式。例如图层上下文菜单中的 Regroup views by layout index 会把所有图层收拢起来，按面板中显示的各个版图分别成组。

1.23 用标签页管理多套图层配置

0.21 版引入了新功能：在 layer 面板中使用标签页，切换不同配置（setup）变得非常简单。

在 layer 面板的上下文菜单（右键单击）中，从 Tabs 子菜单选 New Tab 即可新建标签页。新标签页会出现在 layer properties 面板顶部，起初是当前配置的副本。此后对 layer properties 的任何修改只作用于该标签页；切换到另一个标签页时，版图视图会反映该标签页的设定。这样就能预先准备好几套配置并方便地切换。

保存 layer properties 时，所存文件会包含全部标签页，因此多页配置可以方便地存盘与取回。

标签页的初始标题是标签序号。标题可用 layer 面板上下文菜单 `Tabs` 子菜单中的 `Rename Tab` 修改；要删除标签页，选 `Remove Tab`。

1.24 从图层集中增删图层

layer 列表中显示的图层更像是指向真实版图图层的“指针”，而不是真实图层本身。正因如此，这些条目更准确的叫法是 view。图层可以从列表中删掉再建回来，而不影响真正的版图数据。

要新建图层，在图层上下文菜单（右键单击 layer 列表）中选 `Insert View`，随后弹出输入框要求给出 source specification。source specification 说明显示数据取自哪个真实数据层。最简单的形式是“层号/datatype”，例如 `5/0`；若 OASIS 中有图层名，也可以直接写图层名。该说明还可以附加版图序号：面板中加载的第一个版图写作 `@1`，也可以不写；因此 source specification `10/5@2` 指的是面板中第二个版图的第 10 层、datatype 5。

source specification 支持通配符：层号、datatype 或版图序号都可以写成 `*`。此时该图层必须位于某个组内，由组的父节点补全缺失的部分。例如图层写作 `10/*`、父节点写作 `*/5`，实际查找的图层就是 `10/5`。与显示样式的继承规则相反，在 source specification 上是子节点覆盖（细化）父节点。

可以用上下文菜单中的 `Clean up views` 清理 layer 列表，去掉那些与真实版图图层不对应的 layer view。反过来，版图中存在但列表中没有 view 的图层，可以用 `Add other views` 补上。

1.25 视图变换与 property selector

前面介绍的 source specification 远不止能描述数据来源：图层还可以做几何变换，显示范围也可以限定为具有某种 property selector 所描述属性的 Shape。

几何变换以圆括号追加在层号/datatype 的 source specification 之后。变换的写法如下（各部分顺序任意）：

```
( [<dx>,<dy>] [r<angle> | m<angle>] [*<mag>] )
```

例如 `(r90)` 表示逆时针旋转 90 度；`(0,100.0 m45 *0.5)` 则把版图缩到一半、沿 45 度轴翻转（即交换 x 与 y 轴），最后整体向上平移 100 微米。

变换语法的详细说明见第原文对应章节节。

变换会沿图层层次累加：若某图层自身带变换，同时它又位于某个也指定了变换的组代表节点之内，则最终变换是“先施加该图层的变换，再施加组代表的变换”。

也可以同时给出多个变换，此时同一版图会以多个实例的形式显示。

一个典型用法是按版图序号重新归组，并给属于某个版图的组代表指定变换，从而把多个版图对齐。

property selector 写在方括号内。selector 用运算符 `&&`、`||`、`!` 把若干形如 `<prop>==<value>` 或 `<prop>!=<value>` 的表达式组合起来，并允许用圆括号规定运算优先级：

```
[ <expr> ]
```

在 GDS2 文件中，属性名总是整数，写作 \# 后接数字（例如 `#43`）；GDS 属性的值总是字符串，可以直接写成不加引号的文本原子，也可以用单引号或双引号括起。下面是 GDS 文件下一个合法的 property selector 示例：

```
10/5 [#43==X]
```

用这个 source specification，图层只会显示第 10 层、datatype 5 上那些带编号为 43 且值为字符串“X”的 user property 的 Shape。再复杂一点的例子：

```
10/5 [!(#43==X&&(#2==Y||#2==U))]
```

在 OASIS 文件中，属性名可以是字符串，此时 property selector 可以写作 `[prop==X]`。此外属性值可以是整数或浮点数，这由写法本身区分：`[prop==#200]` 检查名为 prop 的属性是否具有整数值 200；类似地，`[prop==##0.5]` 检查 prop 是否为浮点值 0.5。

property selector 会沿图层层次叠加：若组代表指定了一个 selector，组内某图层也指定了 selector，则只显示同时满足两者条件的 Shape。

source 记法的总体说明见第原文对应章节节。

1.26 为单个图层或图层组指定层次层级

默认情况下只显示在层次层级选择框中选定的那些层级；也就是说，若选中 Levels 0 到 1，就只显示顶层 Shape 与 Instance。这个选择可以对某些图层或图层组单独修改。要给某个图层指定不同的层次选择，可使用 source specification 的可选组成部分——hierarchy level selector：

```
#[<lower-level>][ .. <upper-level>]
```

上下层级都可以省略，省略时就沿用全局设置、不予覆盖。上层层级可以写 `*`，表示一切可用的层级。若只写一个层级而没有 `..`，则把它当作上层层级，下层设为 0。

下面几个例子可以说明：

```
#*
```

显示所有层次层级

```
#0..1
```

只显示顶层

```
#..5
```

把上层层级覆盖为 5

```
#2..
```

把下层层级覆盖为 2

```
#..*
```

把上层层级设为“所有层级”

为了支持更多用法，该记法还提供了一些变体：层级不必只写一个数字，也支持下列写法：

```
(1)
```

相对写法：层级 1 相对于当前 Cell 所在层级。在 `Descend` 模式下结果会不同，此时当前 Cell 处于比上下文 Cell（被绘制的最顶层 Cell）更低的层级。

```
<1
```

受限写法：层级 1，且若界面中设定的默认上/下层级更小，则取更小者。

```
>1
```

受限写法：层级 1，且若界面中设定的默认上/下层更大，则取更大者。

```
(>1)
```

组合写法：相对于当前 Cell 所在层级的层级 1，或取更小者。

```
>*
```

等于当前设定的最大层次层级。

例如：

```
#(0)..(1)
```

当前 Cell 的顶层（在 `Descend` 模式下同样有效）。

```
#>0..<1
```

恰好位于顶层的一切，除非顶层在控件中未被选中。

```
#>1..<*
```

上下文 Cell 顶层以下的一切，除非界面控件未选中这些层级。

```
#(>1)..<*
```

同上，但相对于当前 Cell 而非上下文 Cell。

1.27 载入与保存图层集

图层的显示属性可以用 File 菜单中的 Save Layer Properties 存到文件，之后用 Load Layer Properties 重新载入。

1.28 截图

要把画布存成 PNG 文件，从 File 菜单选择 Screenshot，或按 Print 键。随后弹出文件对话框，指定截图要保存到的文件。

1.29 进行测量

在工具栏点击标尺图标，再从下拉选项中选择 Ruler 即可测量。在版图中左键单击一个点，再左键单击一次或按 Enter 确定第二个点，就会画出标尺并显示所测距离。

拖动端点时可以按住 Shift 或 Ctrl，或两者同时按。只按 Shift 时，标尺方向被限制为水平或竖直；只按 Ctrl 时，限制为水平、竖直或 45 度斜向；同时按 Ctrl 与 Shift 时不作方向限制。

拖动标尺时，当前拖动距离会显示在版图视图顶部的三个输入框中。按 Tab 键可把输入焦点切到这些输入框，在其中输入距离的数值，用 Tab 与 Shift+Tab 在框间跳转，按 Esc 退出输入框。光标位于 dx 或 dy 框时按 Enter 会接受输入值并应用到标尺上。若在标为 d 的最后一个框中输入数值并回车，标尺会进入“定长”模式。

测量距离有一种便捷做法：单击测量标尺。在标尺图标的下拉选项中选 Measure，此时单击一次即可在指定位置放置标尺。该功能会寻找标尺附近的边，把标尺接到相邻的边上，并使标尺垂直于起点旁的那条边。

Measure Edge 标尺类型同样是单击测量，但测的是单击位置处一条边的长度。

选择 Cross 标尺类型，单击一次即可标记一个位置，标尺会显示该处的 x、y 坐标。

Multi-Ruler 可以把多条标尺串成一个对象。单击第一点开始，再单击更多点继续添加分段；每一段都以独立的标尺显示，带刻度和长度。双击或按 Enter 结束。注意标尺标签给出的是各分段的长度；若要显示累计长度（所有分段长度之和），把标尺的标签格式从 $D 改为 $DD（DD 是表示累计长度的变量）。详见第 1.30 节。

Angle标尺用于测角：需要单击三次，定义两条呈“V”形排布的标尺，标尺上显示这两条线所夹的角度。

另一种特殊标尺是 Radius，同样需要三次单击：三个点定义一个圆，标尺显示该圆的半径与圆周。

Box与 Ellipse标尺与普通标尺一样是两点式，但显示由两点张成的矩形或椭圆。对这两种标尺，移动第二点时 Shift 与 Ctrl 的含义不同：按 Shift 时第一点成为矩形或椭圆的中心；按 Ctrl 时矩形被约束为正方形、椭圆被约束为圆；两键可同时按。凡使用 box 或 ellipse 轮廓样式的标尺都遵循这一规则。

标尺的配置方式很多。从 Edit菜单打开 Rulers And Annotations Setup 即可配置。勾选 Snap to edge/vertex 可让标尺吸附到对象的边或顶点；用 Angle Constraint 选项可约束标尺方向；用 Limit number of annotations 可限制标尺数量。

在画标尺或移动某个端点时，可以用 Shift 与 Ctrl 临时改变方向约束：按住 Shift 强制正交，按住 Ctrl 强制斜向，两者同按则解除一切方向约束。在画 Box 或 Ellipse 标尺时，这两个键改为让标尺居中或约束为正方形/圆（见上）。

Edit菜单中的 Clear all rulers可清除所有标尺。

拖动标尺的过程可以用 Esc 键或 Edit菜单中的 Cancel取消。

用工具栏上的 speedbar 按钮切到 Move模式，或从 Edit菜单的 Mode子菜单选 Move，即可移动标尺：左键单击并拖动标尺本身或其某个端点。

在 Select模式下选中某个标尺后按 Delete，可以单独删除它。

标尺可以做出各种变化，例如显示成箭头。要编辑标尺属性，双击该标尺，或选中它后用 Edit菜单中的 Properties。属性说明见第 1.30 节。

可以为标尺配置多个模板，每个已定义的模板都会出现在 Ruler模式工具栏按钮的下拉菜单中。选中某个模板后，由它产生的新标尺会继承该模板的属性。模板在标尺设置页（File菜单的 Setup）或 Edit菜单的 Ruler And Annotation Setup中管理。

1.30 标尺属性

以下是标尺可配置的属性：

- Labels（标签）：随标尺轮廓不同，最多可有三处标签，每处都可单独指定显示一段文字还是测量值。主标签总是存在；只有当标尺具有明确的竖直或水平分量时，X 与 Y 标签才出现（diagonal之外的所有轮廓样式都满足）。主标签可指定位置（P设置）：可放在第一点、第二点或标尺中点。标签的对齐方式也可指定：左对齐、右对齐或居中。
- Style（样式）：决定标尺及其各部件如何绘制，可以是“带刻度的尺状”、箭头样式、普通直线，或在两端带十字标记。

- Outline（轮廓）：决定构成标尺的两点之间如何连成标尺形状，可以只是一条直线（`diagonal`），也可以是水平线与竖直线（部分轮廓样式会再叠加斜线），或由两点张成的矩形。还有一种特殊轮廓：`ellipse`，即在两点张成的矩形内画椭圆。
- Angle constraint（角度约束）：可以用多种方式限制标尺朝向，例如只允许水平。默认情况下标尺沿用全局设置，也可以让它自带独立的约束。
- Mode（模式）：普通模式下需要两下单击，分别确定第一点与第二点；`Single click` 模式下单击一次即把两点设在同处；`Auto measure` 模式下则会在单击点附近寻找边，据此自动调整两点。
- Never snap to objects：勾选后标尺绝不吸附到绘制对象上，即使全局选项已打开。

Label format（标签格式）是一段任意文字，其中可嵌入表示测量值的表达式。表达式以美元符号开头，后接表达式字符串。表达式语法支持基本运算（`*`、`/`、`+`、`-`等）、位运算（`|`、`&`等）、条件运算符（`x:y?z`），以及 `abs`、`sqrt`、`exp`等函数，还包含`sprintf`函数。示例如下：

- `$X`：变量 X 的值（水平距离，变量全表见下）。
- `$(sprintf('%.2f',X))`：把变量 X 格式化为两位小数，`sprintf`的用法与同名标准 C 函数一致。
- `$(abs(X)+abs(Y))`：标尺的曼哈顿距离。
- `$min(X,Y)`：X 与 Y 中的较小者。
- `$(X) ($Y)`：先 X 的值，再括号中的 Y 值，结果形如 “2.5 (-0.5)”。注意不能直接写 `$X ($Y)`，因为表达式求值会把它读成调用名为 X 的函数。

表达式语法与可用函数的说明见第原文对应章节节。
可用变量如下：

- `D`：标尺长度，单位微米。
- `L`：标尺的曼哈顿长度，单位微米。
- `U`：标尺第一点的 x 坐标，单位微米。
- `V`：标尺第一点的 y 坐标，单位微米。
- `P`：标尺第二点的 x 坐标，单位微米。
- `Q`：标尺第二点的 y 坐标，单位微米。
- `X`：标尺的水平延伸量，单位微米。
- `Y`：标尺的竖直延伸量，单位微米。
- `A`：标尺所围面积（若视为矩形），单位平方毫米。
- `G`：标尺首段与末段所夹角度（用于量角标尺）。

对 multi-ruler 还提供表示 “累计” 值的额外变量，即相应数值到所指分段为止的和：

- DD：到被标注分段为止的所有长度之和。
- LL：到被标注分段为止的所有曼哈顿长度之和。
- XX：第一点到当前点的水平距离。
- YY：第一点到当前点的竖直距离。

1.31 添加图像

某些应用需要把平面的像素数据与版图一起显示，例如拍摄的 SEM 图像，或某个仿真工具的输出。KLayout 提供了把图像加入显示、并置于所绘版图之下显示的方法。

目前可以读取 Qt 支持的各类常见图像格式（如 PNG、JPG、TIF 等），彩色与单色图像均支持。图像在内部以浮点矩阵存储，也可以用 RBA 编写自定义导入器。

要添加图像，使用 Edit 菜单中的 Add Image，会弹出图像属性对话框，在其中指定图像；点击文件名框旁的 Browse 按钮选择文件。

图像有若干属性，主要影响其显示方式：

- Pixel size：单个像素的尺寸，单位微米。这决定了图像的整体大小。
- Center：图像中心放置的位置（单位微米）。
- Rotation：图像旋转的任意角度。
- Shear/Perspective Tilt：斜切与透视倾斜角。虽然可以直接指定这些角度，但要使图像与版图对齐，用 landmark 调整功能要容易得多。
- Mirror flag：勾选后，图像在旋转之前先沿底边镜像。
- Pixel value range：对应最小与最大的像素值。常见的 8 位图像格式中这两值为 0 与 255；调整它们可以让图像变亮或变暗。对浮点图像（例如仿真数据），该范围应取输出值的边界，例如归一化数据取 0.0 与 1.0。
- Color mapping：对单色图像，像素值通过映射函数转成颜色。图像属性页中有一个标签专门用于定义数据值到颜色的任意映射：在数据范围轴上放置颜色取样点，并给它们指定颜色。在轴上双击可新建取样点，单击选中后可在取色框中调整其颜色；选中后按 Del 可删除取样点。
- Brightness、Contrast 与 Gamma：对应标签页上各有三个滑块用于调整。
- RGB channel gains：此外，各颜色通道还可以在对应标签页上乘以给定的权重系数。

图像放置好之后，可以用 Move 功能移动与缩放。要调整图像属性，使用 Edit 菜单中的 Properties，或双击该图像。

自 0.22 版起还可以定义 landmark：在图像上任意位置设点，把这些点拖到版图中对应的目标位置即可完成对齐。详见第 1.32 节。

版图视图中可以放置任意数量的图像。要把这套配置存下来，使用 File >

Save Session保存会话。

1.32 用 landmark 对齐图像

landmark 是图像上的任意位置。可以在图像上以图形方式定义这些位置，然后在版图视图中把它们当作把柄（handle），把图像与版图对齐。视所定义的 landmark 数量，该功能可以补偿图像的平移，乃至旋转、斜切甚至透视畸变。

landmark 在 landmark 编辑器中定义。在图像属性对话框中按 Define按钮即可打开 landark 编辑器。

landmark 编辑器左侧显示图像，右侧列出已定义的 landmark。可以用与 KLayout 版图视图相同的方式在图像视图上缩放（滚轮、鼠标中键、右键框选）。

要添加 landmark，选择 Add模式，然后在想要的位置单击，图像上会出现一个小“靶心”十字作为标记。要删除 landmark，选择 Delete模式并单击该 landmark；也可以先在列表中选中，再进入 Delete 模式，此时当前选中的 landmark 会被删除。

要移动 landmark，进入 Move模式后移动它。注意移动遵循 KLayout 的“取放”（pick and place）规则：需用鼠标单击两次，一次拾取、一次放下。这样做的好处是拾起 landmark 的过程中仍可缩放。

定义超过 4 个 landmark 意义不大，因为再多变换就过定了。为了精度，应挑选覆盖较大范围的 landmark 位置：彼此不能太近，且三个 landmark 不应共线。

视 landmark 数量，KLayout 可调整以下图像参数：

1 个 landmark：平移

2 个 landmark：平移、旋转与缩放

3 个 landmark：平移、旋转、缩放与斜切

4 个 landmark：平移、旋转、缩放、斜切与透视畸变

定义好 landmark 之后，就可以用它们调整图像的显示变换：在 KLayout 中进入 Move模式，鼠标移到图像上时 landmark 符号会显现，可用鼠标拾取。拾起某个 landmark 并移动鼠标时，KLayout 会相应调整图像变换，并按新变换显示图像外框；放下 landmark 后，图像即按此变换呈现。

视 landmark 数量，KLayout 可以只调整部分变换参数或全部，使其它 landmark 保持在原位。某些场合这未必是想要的：例如定义了 4 个 landmark 却只移动其中一个，结果可能出现极端的透视畸变。通常的做法是先粗调位置与放大倍率，最后才用透视变换细调。

因此 KLayout 允许在拖动 landmark 时降低变换的自由度，可用的修饰键如下：

Shift：只调整平移

Ctrl：调整平移、旋转与缩放

Shift+Ctrl：调整平移、旋转、缩放与斜切

1.33 浏览 Shape

有一个简单的 Shape 浏览器，可以浏览某图层上的所有 Shape。在 layer 列表中选中要浏览的图层，再从 `Tools` 菜单选择 `Browse shapes`。

弹出的浏览器对话框列出 Cell、Shape 与 Cell Instance。选中某个 Cell 后，中间列表显示该 Cell 中的所有 Shape，右侧列表显示该 Cell 相对于顶层 Cell 的各个 Instance。

选中某个 Shape 时，版图画布会在它周围画一个标记框并缩放到该 Shape。Shape 的显示方式可以在该对话框的 `Configure` 标签页中，或在 `Setup` 对话框的相应页中配置。

另一个可用来检查 Shape 的功能是 `Search and replace`（见第 2.3.2 节）。

1.34 浏览 Instance

要浏览某个 Cell 的全部 Instance，在 Cell 列表选中该 Cell（不必把它设为新的顶层），再从 `Tools` 菜单选择 `Browse instances`。弹出的简单 Instance 浏览器会列出该 Cell 被实例化在哪些 Cell 中，以及实例化的方式。

另一个可用来检查 Instance 的功能是 `Search and replace`（见第 2.3.2 节）。

1.35 marker 浏览器

KLayout 提供了一套通用机制来保存错误标记（error marker）或相关信息，称为 report database（RDB）。一个版图视图可以关联任意多个 report database；通常每个数据库对应一个版图，但这并非硬性要求。

report database 首先是一个通用“值”（value）的集合，值可以是字符串或其它条目。通常一个 value 是一组几何对象的集合，用来标出某个位置或某段绘制几何。若干这样的 value 构成一个 marker item。report database 再把这些 marker item 与附加信息关联起来：

- Tags：表示特定状况的标志。marker 浏览器使用几个预定义标签，如 `important`、`waived` 与 `visited`，用户可设置或清除它们，以表示某个 marker item 被认为重要、或该错误已被豁免。
- Image：marker 可以附上一张截图，供文档记录之用。

marker item 按 category 组织，每个 marker item 必须属于某个 category。category 本身也可以分层，即拆成子 category，从而更好地组织归类。

marker item 通常关联到某个 Cell，即发现错误的位置所在。默认情况下 marker item 直接关联到顶层 Cell。

report database 使用基于 XML 的专有格式，能够保存数据库所提供的各项标注。不过也可以导入 Calibre DRC 的 ASCII 格式文件。

marker 浏览器用来浏览与某个视图关联的 report database，可从 `Tools` 菜单中的 `Marker Browser` 启动。它会跟踪某个 marker 是否已被查看过，类似邮件客户端的“已读”标记，从而便于跟踪一次复核进度；`visited` 状态会写入数据库文件。

在 marker 浏览器中，用 `Open` 按钮载入 XML 数据库文件或导入其它格式的文件；`Reload` 重新载入文件；`Save As` 把数据库写成 XML 格式。

marker 浏览器提供三个面板：

- Directory：列出数据库中的 category 与 Cell。尚有未查看 marker 的 category 或 Cell 以粗体显示，完全没有 marker 的则以绿色显示。在目录的上下文菜单中取消勾选 `Show All` 可以把后者隐去。点击计数列的标题可按 marker 数量排序。可以选中多个 category 或 Cell，此时 markers 面板会显示所有选中项的 marker。
- Markers：列出所选 category 和/或 Cell 中的 marker。列表长度有限制，可在配置页（marker 浏览器上的 `Configure` 按钮，或 setup 对话框）中修改。各种 tag 也在此面板中显示。点击相应标题可对列表按多种方式排序。选中某个 marker 时，若关联了合适的版图，该 marker 会在版图中高亮；高亮方式以及视图如何调整可在配置页中指定。
- Info：汇总所选 marker 的信息。若该 marker 附有截图，会在此显示；点击缩略图可在单独窗口中按原始尺寸查看。

与 Shape、Instance 浏览器类似，marker 浏览器也提供导航按钮，用于跳到下一个 marker、category 或 Cell。

marker 浏览器支持 tagged value：每个 marker 都可关联一个带名字的值。这类值可从 RVE 文件导入，在那里它们被称为 properties。tagged value 由特定生成器产生，可以表示测量值之类。存在 tagged value 时，它们会显示在 markers 列表中；点击相应的列标题即可按这些值排序。

1.36 technology 管理

KLayout 0.22 的新功能之一是 technology management。它涵盖那些需要对版图做出特定解释的功能，尤其是给版图图层赋予物理含义，例如 CMOS 工艺中的 via 层、有源区层。这种解释往往取决于产品将用哪种工艺制造，因此“能够提供多套配置”这一能力被统称为 technology management。

`Tools` 菜单中的 `Technology Manager` 是用于新建、删除与编辑 technology 配置的用户界面。详见第 1.36 节。

1.37 选择标尺、Shape 与 Instance

在 Select 模式下单击标尺、图片、Shape 或 Instance 即可选中它；也可以按住鼠标左键拖出选择矩形，此时矩形内的所有对象都会入选。

选择 Shape、Instance 或其他对象时，按 Shift 会在当前选择基础上扩展，按 Ctrl 会把指定对象从选择中移除。

对于版图对象（Shape 与 Instance），既可以穿透层次选择，也可以只在当前 Cell 这一层选择（仅顶层对象）。默认是穿透层次选择，也就是说子 Cell 中的对象也能被选中；当同一个 Cell 被多次实例化时，同一个对象可能被重复选中。虽然能够编辑子 Cell 内部的对象有时很方便，但最终可能带来令人困惑的结果，因此只处理顶层（即当前 Cell）往往更直观。要启用仅顶层模式，勾选 View 菜单里的 Select Top Level Objects Only 菜单项，或在编辑模式选项对话框（F3）中做同样的设置。

在仅顶层模式下，只有当 Cell 具有可见外观时，其上的 Instance 才能被选中：例如该 Cell 在某个可见图层上至少含有一个 Shape。若相对于当前可见图层而言它是空的，就无法被选中。如果因为所显示的层次层级被限制在足够小的一段区间内而显示出 Cell 的边框，那么这些 Cell Instance 总是可以被选中。

图片和标尺比较简单，它们不嵌入层次结构，单击它们或把它们框进选择框即可选中。

打开属性对话框可以查看对象属性（在允许的情况下也可以修改）。要查看或修改所选对象的属性，从 Edit 菜单选择 Properties，随后弹出的对话框会显示第一个选中对象的属性。按 Next 前进到下一个对象，按 Previous 回到上一个对象。

根据当前应用模式不同，所选对象的属性也可能允许修改。要把改动应用到当前对象，选 Apply；要应用改动并关闭对话框，选 Ok；要把改动应用到当前对象以及所有同类选中对象，选 Apply To All（若该按钮可用）；不应用任何改动直接关闭对话框，选 Cancel。

Apply To All 会以某种智能方式把改动套到其它对象上。例如对 Box 而言，尺寸的改动会以与当前对象相同的方式（四个方向上的同等偏移）施加；若改的是 Path 的宽度，则同样的宽度会套到其余所有 Path 上，依此类推。

对于 Shape 和 Instance，也可以编辑 user properties（用户属性）。user properties 是任意挂在 Shape 或 Instance 上的属性，由一个 key（键，为兼容 GDS2 最好用数字）和一个 value（值，在 GDS2 中最好是字符串）组成；一个对象上可以挂多条这样的属性。点击 User Properties 按钮可打开专门编辑这类属性的对话框。若该对话框以 Ok 关闭，user properties 会被保留下来，但要等到按下 Apply / Ok 或 Apply To All 之后才真正写入版图对象；其中 Apply To All 会把新属性套用到其余同类 Shape 上。

最后，对于 Shape 和 Instance，点击 Instantiation 按钮可以显示该 Shape 或 Instance 的实例化路径。

通过属性对话框不能改变 Shape 所在的图层。要把 Shape 移到别的图层，请使用

Edit菜单下 Selection子菜单中的 Change Layer。

1.38 更多配置选项

从 File菜单的 Setup可打开选项对话框，其中提供大量配置选项，从背景设置到标尺配置都有。

例如在该对话框中可以编辑调色板，以便使用不同的颜色，也可以配置填充图案（stipple）调色板。此外还可以定义这些颜色或填充图案初始分配给图层时的顺序，以及哪些颜色不用于给图层上色。

其中特别有用的一项是 oversampling（过采样）。过采样是为了提升画面质量：先以更高分辨率渲染，再降采样到屏幕分辨率，效果是线条显得更细、细节分辨得更清楚。目前的副作用是填充图案会变得更细，marker 模式下的十字也更小，不过清晰度提升相当可观。

过采样可在 setup 对话框的 Display/General页启用，提供 2× 与 3× 两档。下面几张截图展示过采样的效果：

常规（1×）

2× 过采样

3× 过采样

1.39 撤销与重做

大多数操作（例如修改图层颜色）都可以用 Edit菜单中的 Undo撤销；相应地，也可以用 Edit菜单中的 Redo重做。

1.40 保存版图或其局部

版图或其中的某个子 Cell 都可以存成 GDS2 或 OASIS。要保存整个版图，从 File菜单选择 Save As；要只保存某个 Cell，在 Cell 树中选中它（不必是当前显示的那个），然后在 Cell 树的上下文菜单（右键单击）中选择 Save Current Cell As。

随后弹出文件对话框，指定要把该 Cell 或版图写入的文件名。选好文件后会出现选项对话框，用来指定更多选项，例如可以把保存范围限制在部分图层上（如只保存可见图层），也可以修改数据库单位（database unit），或按给定系数缩放版图。

对 OASIS 可以指定压缩级别。级别 0 时基本不做压缩尝试；级别更高时会先对 Shape 分类，再尝试阵列压缩。级别越高，把 Shape 压成阵列的尝试越多。尤其对平坦的版图，压缩 Shape 需要一定内存，并会显著拖慢 OASIS 写入。

1.41 保存与恢复会话

会话(session)可以保存,之后再恢复。一个 session 包含已加载的文件、bookmark、标注、layer 设置、层次设置、图像以及应用配置。session 以 XML 文件形式保存，后缀为 `.lys`。

要保存 session,从 `File` 菜单选择 `Save Session`;要恢复,选择 `Restore Session`。也可以在命令行用 `-u` 选项后接 session 文件来启动 KLayout。在 Windows 上, session 文件已注册为由 KLayout 自动打开。

1.42 其它视图选项

`View` 菜单还提供一些调整显示方式的选项:

- `Show Grid`: 对应配置项的快捷入口。取消勾选后不显示网格线与比例尺。
- `Grids`: 该子菜单可快速切换当前网格设置，可从 setup 对话框 `Application` 部分的 `Default Grids` 页中配置的若干默认网格中挑选。网格主要用于编辑几何图形，若标尺配置了吸附，也用于标尺吸附。
- `Show Texts`: 取消勾选时不绘制文字，这对 Text 对象与 user properties 都适用。不绘制文字时绘制速度会快一些。
- `Show Cell Frames`: 取消勾选时, 被隐藏的 Cell 以及超出所选层次层级的 Cell, 其边框不再绘制。
- `Show Layers Without Fill`: 勾选后 Shape 不带填充绘制，版图呈线框状。该选项本意是临时关闭填充图案；若一直勾选着会令人困惑，因为此时设置填充图案可能毫无效果。
- `Synchronized Views`: 勾选后主窗口中所有视图（标签页）同步，即显示版图的同一区域。当为了清晰而把两个版图放在不同视图中、又需要比对相同区域时，这个选项很有用。
- `Select Top Level Objects`: 对应编辑器选项的快捷入口。勾选后只选中顶层对象。这对编辑尤为重要，因为编辑子 Cell 会在该 Cell 被实例化的其它位置产生连带影响，而那里本来并不打算改动。
- `Toolbar`、`Navigator` 等: 分别启用或禁用对应的工具窗口。
- `Highlight Object Under Mouse`: 勾选后，鼠标悬停的对象会被临时高亮。由于这在少数情况下会造成困惑，该菜单项只是提供这个配置项的快捷入口。

Chapter 2

Editing Functions

2.1 编辑模式

只有在 KLayout 处于 edit mode 时才能使用编辑器功能。只要在命令行中加上 `-e` 选项，KLayout 就会进入编辑模式：

```
klayout -e [<input file>] [-l <layer properties file>]
```

相应地，用命令行选项 `-ne` 可以强制进入不可编辑模式。

在 Windows 上，开始菜单里分别有编辑器模式与查看器模式的入口。也可以把 KLayout 配置为启动时默认使用编辑模式：在 setup 对话框（`File` > `Setup`）的 `Application` 标签页中勾选 `Edit mode` 复选框。

在编辑模式下，某些优化被关闭，因此加载时间略长、内存占用略高。实际增幅强烈取决于输入文件的特性：例如 OASIS 中的 Shape 阵列在编辑模式下不再原样保留，而是展开成一个个单独的 Shape。

2.1.1 取放（pick and drop）原理

多数绘图程序采用“单击并拖动”的范式：按住左键点中一个元素，一直按着把它拖到目标位置。这种范式虽然相当直观，却有一个缺点：在按住鼠标键的同时很难做别的事——特别是无法缩放（难道要同时按下右键、拖出缩放框，再只松开右键?）。为了既能缩放又能拖动，KLayout 采用了“取放”（pick-and-drop）方式。

在 pick-and-drop 中，用左键单击元素把它“取起”，然后移动鼠标（不按任何键）移动它，最后在目标位置左键单击“放下”。由于鼠标键并未按住，鼠标可以自由执行其它操作，只有被拖动的对象“粘”在光标上。

此外，拖动对象的过程中可以用 Shift 与 Ctrl 强制某些方向约束，或覆盖选项（例如 move 或 edit 选项）中设定的约束：按 Shift 时 KLayout 进入正交模式，移动被限制为水平或竖直（若不适用则不作限制）；按 Ctrl 时进入斜向模式，移动被限制为水平、竖直或 45 度斜向；同时按 Ctrl 与 Shift 则解除一切方向约束。

2.1.2 编辑器基本选项

编辑模式下用到的大多数工具都有各自的选项，例如画 Path 时要指定宽度与延伸方式。有一个通用的编辑选项设置对话框，可从 `Edit` 菜单选 `Editor Options` 打开，或

用快捷键 F3（若未被改键）。

对话框中总有一个通用的设置标签页，并按所选工具另有一个工具专属标签页。通用标签页上可修改以下设置：

- Editor grid：每个编辑操作都被限定在此网格上。网格可以关闭、与全局网格对齐（用于标尺与显示的那套），也可以显式指定；它甚至可以是各向异性的，即 y 方向与 x 方向的网格不同。
- Connection angle constraint：在画连线（例如 Path 的某一段、Polygon 的某条边）时，该模式决定线段或边是否受限于特定方向。Any Angle 模式不作限制；Diagonal 模式允许竖直、水平或两个 45 度斜向；Manhattan 模式只允许水平与竖直。
- Movement direction constraint：拖动（即移动）对象时，该模式决定移动是否受限于特定方向。Any Direction 不作限制；Diagonal 允许竖直、水平或两个 45 度斜向；Manhattan 只允许水平与竖直。
- Top level selection mode：勾选复选框时进入顶层选择模式，只选中当前显示 Cell 所在层级上的元素（这里的“顶层”指当前显示 Cell 的顶层）。也就是说，若选中的是子 Cell 中的 Shape，实际选中的是整个子 Cell 的 Instance。不勾选时进入层次选择模式，子 Cell 中的元素也会被选中。该模式允许就地编辑子 Cell。

在设置对话框中做了改动之后，用 Apply 或 Ok 使其生效。

2.1.3 背景组合模式

KLayout 提供多种方式，把新画的 Shape 与已有 Shape 组合。可通过工具栏上的 Select background combination mode 工具按钮选择模式：

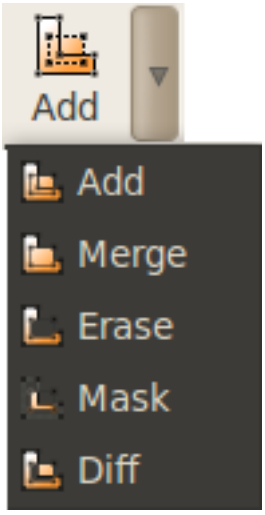


图 2.1: 背景组合模式

可用的模式如下：

- Add：默认模式，新画的 Shape 直接加入已有 Shape 之中。
- Merge：新画的 Shape 会与已有 Shape 合并。该操作总是生成一个 Polygon，即新画 Shape 与所有与之接触或重叠的 Polygon 的并集；Path 一律转换为 Polygon。此模式等价于布尔“OR”运算。
- Erase：新画的 Shape 会从已有 Shape 中减去，可用来在与新画 Shape 接触或重叠的 Shape 上开出缺口或细缝；新画的 Shape 本身消失。此模式等价于布尔“NOT”运算。
- Mask：新画的 Shape 充当已有 Shape 的掩模，只有与之重叠的部分保留。此模式等价于布尔“AND”运算。
- Diff：新画的 Shape 会反转已有 Shape。此模式等价于布尔“XOR”运算。

2.1.4 选区

某些操作所作用的基本对象集合是 selection（选区），即一组要对之施加操作的 Shape 与 Instance。可以在 `Select` 模式下单击元素建立选区，也可以拖出选择矩形；松开鼠标时，矩形内的所有元素都被选中。

选区可以修改：追加元素（选择元素时同时按 Shift）、移除元素（同时按 Ctrl），或切换选中状态（同时按 Shift 与 Ctrl，即把已选中的去掉、未选中的加入）。

2.1.5 局部模式

partial editing（局部编辑）是一项强大的功能，可以对 Shape 做局部修改：可以移动 Polygon、Path 的边或段，删除 Polygon 或 Path 的顶点、边或段，也可以往 Polygon 与 Path 中插入新点。partial editing 支持复杂的局部选区：可以同时选中多条边或多个顶点，一并删除或移动。

普通的选中是“整元素”方式的，此时整个 Shape 被移动或删除。只有在整元素模式下，Shape 或 Instance 才能送入剪贴板。

2.2 编辑模式的基本原理

本节介绍编辑模式的基本工作原理。

2.2.1 从零创建版图

要从头开始创建一个空版图，从 `File` 菜单选择 `New`。弹出的表单要求填写几个基本参数：

- Top cell：版图中第一个（也是唯一一个）Cell 的名字。
- Database unit：数据库单位，即整数坐标与微米之间的换算系数，本质上就是版图的分辨率。

- Initial window size：首次打开顶层 Cell 时所显示窗口的尺寸。由于初始视图是空的，没有任何几何参照，指定初始尺寸至少能让“画布”的大小已知。

若在 setup 对话框（Application标签页）中指定了默认的 layer properties 文件，它会自动载入 layer 视图列表。若没有这样的文件，layer 列表初始为空，必须先通过 Edit > Layer > New建好图层，才能画 Shape。

2.2.2 新建图层

可以用 Edit菜单中的 Layer > New Layer新建图层。系统会要求输入 GDS 的层号与 datatype 号，以及可选的 OASIS 图层名。按 Ok后图层即被创建并插入 layer 面板。

2.2.3 新建 Cell

可以在层次面板的上下文菜单（右键单击 Cell 层次面板）中用 New Cell新建 Cell。系统会要求输入新 Cell 的名字（该名字的 Cell 尚不能存在）以及窗口尺寸。窗口尺寸是显示该 Cell 时视图的初始尺寸，除此之外没有其它含义。

2.2.4 画 Polygon

在工具栏选择 Polygon模式，在 layer 面板中选择新 Polygon 要画在哪个图层，然后在 Polygon 的第一个顶点处左键单击。移动鼠标到下一个顶点，左键单击放下该点，再移到下一个顶点。画出的边会依 connection mode 被限制在某些方向上，各模式的详细说明见第 2.1.2 节；可用 editor options 对话框（F3）更改模式，编辑过程中也可以改。

要临时约束线段方向，拖动线段时按住 Shift 或 Ctrl，或两者同时按。Shift 施加曼哈顿约束（只允许竖直与水平），Ctrl 在此外还允许斜向，同时按 Shift+Ctrl 则允许任意方向。

在最后一点双击或按 Enter 键结束 Polygon；按 Esc 键取消操作；按 Backspace 键删除最后一段。

拖动 Polygon 的某一段时，版图视图顶部会出现两个输入框。按 Tab 键进入输入框，用 Tab 与 Shift+Tab 在框间跳转，可在其中输入该段向量的数值；按 Enter 会应用这些相对坐标并开始新的一段，按 Esc 退出输入框。

Polygon 永远不会处于“开放”状态：当前顶点与起始顶点之间总会有边相连。视模式不同，这条收尾连接是一条直线，或是若干边的组合。在 diagonal mode下，收尾连接的实现方式有很多种，程序用一些启发式规则挑出一种可行的组合；这套启发式虽不一定足够聪明，但通常能满足需要。

2.2.5 画 Box

在工具栏选择 Box模式，在 layer 面板中选择新 Box 要画在哪个图层，在第一点处左键单击，移动鼠标到第二点，左键单击即完成。按 Esc 键取消操作。

拖动 Box 时按住 Ctrl 键可把 Box 约束为正方形；按住 Shift 键则让第一点成为 Box 的中心。两键可同时按。

拖动 Box 时，版图视图顶部会出现两个输入框。按 Tab 键进入输入框，用 Tab 与 Shift+Tab 在框间跳转，可在其中输入 Box 的宽与高；按 Enter 应用这些尺寸，按 Esc 退出输入框。

注意：Box 一旦创建就始终保持为 Box，例如不能删掉其中一个顶点来把它变成三角形，那只有 Polygon 才能做到。

2.2.6 画 Path

在工具栏选择 Path模式，会弹出 editor options 对话框，其中还要求填写 Path 的基本参数，例如宽度与延伸方式（extension scheme）。画 Path 时它会采用该对话框中设定的参数；Path 正在绘制过程中也可以修改这些属性。若选项页被误关，用 F3 快捷键可重新打开。

着手画 Path 时，先在 layer 面板中选择要画在哪个图层，在第一个顶点处左键单击，移动鼠标到第二个顶点单击放下，如此继续到最后一个顶点。在最后一点双击或按 Enter 键结束 Path；按 Esc 键取消操作；按 Backspace 键删除当前段并退回上一段。

要临时约束线段方向，拖动线段时按住 Shift 或 Ctrl，或两者同时按。Shift 施加曼哈顿约束（只允许竖直与水平），Ctrl 在此外还允许斜向，同时按 Shift+Ctrl 则允许任意方向。

拖动 Path 的某一段时，版图视图顶部会出现两个输入框。按 Tab 键进入输入框，用 Tab 与 Shift+Tab 在框间跳转，可在其中输入该段向量的数值；按 Enter 会应用这些相对坐标并开始新的一段，按 Esc 退出输入框。

和 Polygon 一样，Path 的各段也受 connection angle constraint 所施加的方向限制，各模式的详细说明见第 2.1.2 节；可用 editor options 对话框（F3）更改模式，编辑过程中也可以改。

Path 常被用来当作导线。KLayout 可以配置 via，使得能在图层之间切换，并在两条 Path 重叠处于层间放置一个 via。使用 via 功能之前，KLayout 必须先具备 via 定义。via 本质上就是 PCell，只是需要支持一组特定参数：via PCell 需要暴露若干描述符来定义该 PCell 所支持的选项（例如它所连接的两层）。

2.2.7 创建 Text 对象

在工具栏选择 Text 模式，会弹出 editor options 对话框，其中还要填写文本内容。别忘了点 Apply 使当前字符串生效。若对话框被误关，可用 F3 快捷键重新打开。

要真正放置文字，把鼠标移到目标位置左键单击即可。

Text 可以带一个尺寸，该尺寸会存入 GDS2 文件（OASIS 不支持此特性）。只有选择了可缩放字体（Default 字体不可缩放）且启用了文字缩放时，文字尺寸才会在版图中体现出来：在 setup 对话框的 Display 标签页的 Text font 选择框里挑一个可缩放字体，并在同一标签页勾选 Apply text scaling and rotation。

Text 还可以旋转，但同样只有在启用了文字缩放与旋转后才会体现。放置文字时按右键可把它旋转 90 度（逆时针）。

2.2.8 放置 Cell Instance

在工具栏选择 Instance 模式，会弹出 editor options 对话框，其中还要求填写一些 Instance 参数。最重要的自然是所要放置的 Cell。几何方面可以指定旋转角、设置镜像选项，也可以把 Instance 指定为规则阵列；作为阵列时，该 Instance 表示该 Cell 在规则网格上的多次放置。

要放下 Instance，把鼠标移到目标位置左键单击。移动过程中按右键可把 Instance 旋转 90 度（逆时针）；按 Esc 键取消操作。

自 0.22 版起 KLayout 支持库（library）。库提供版图之外的 Cell，这些 Cell 会被导入版图并在版图内保存一份副本。同时 KLayout 仍保留对原始版图的引用，因此库发生变化后，重新加载版图时该 Cell 会被替换。从库导入的 Cell 在 Cell 树与版图中显示为 Library.Cell 形式，其中 Library 是库名。

库在 Cell 名字右侧的下拉框中选择；也可以用 ... 按钮浏览所选库中可用的 Cell。

库是提供常用版图构件的便捷方式。此外 KLayout 现在还支持 PCell（参数化 Cell）：这类 Cell 没有静态内容，而是由一段代码按该 PCell 类型专属的一组参数动态生成。例如圆有两个参数：圆出现在哪个图层，以及圆的半径。

PCell 由库提供。从库中选中某个 PCell 时，Instance 属性页还会多出一个面板用于编辑 PCell 参数；可用的参数取决于 PCell 的类型。

KLayout 自带一个标准库 Basic，其中有若干基本的曲线形状类型以及一个文字生成器。

PCell 的实现有一个独特之处：PCell 可以使用“引导图形”（guiding shape）。引导图形本身不会作为版图出现，而是被 PCell 用来推导自己的几何。例如 Basic 库中的 rounded path PCell 就是用一条 Path 作为引导图形，对这条 Path 做处理以得到最终的形状。

引导图形画在 Cell 边界层上，可以用常规的图形操作手段处理（特别是移动与局部编辑）；其图形属性也可以用 Edit > Properties 编辑。

库的更多说明见第原文对应章节节，PCell 的细节见第原文对应章节节。

2.2.9 移动选区

整个选区可以在 Move 模式下移动。若已有元素被选中，进入 Move 模式后在某个位置左键单击指定参考点，该参考点即作为“拖拽把柄”，每个元素都相对它移动。若进入移动模式时没有选中任何元素，则直接单击要移动的元素，再左键单击把它放到别处。

移动过程中按右键可把整个选区旋转 90 度（逆时针）；按 Esc 键取消操作。

移动受 movement direction constraint 约束，各模式的细节见第 2.1.2 节。例如在曼哈顿模式下只允许水平与竖直移动。移动鼠标时按 Shift（正交）、Ctrl（斜向）或同时按 Shift 与 Ctrl（任意角度）可临时覆盖全局约束。

移动选区时，版图视图顶部会出现两个输入框。按 Tab 键进入输入框，用 Tab 与 Shift+Tab 在框间跳转，可在其中输入位移量；按 Enter 应用该位移，按 Esc 退出输入框。

要施加确定的位移，也可以用 Edit > Selection 菜单中的 Move By：弹出的对话框可以按微米指定水平与竖直移动距离，正值向右或向上、负值向左或向下。该对话框同样适用于局部模式，因此可以把版图中的边或局部精确地按给定方向移动给定距离。

类似地，Move To 可以把选区重新定位到某个点。所选的点可以相对于选区的边界框指定：例如以选区左下角为参考点，并在对话框中给出某个位置，则选区会被移动，使其边界框左下角与给定坐标重合。

2.2.10 选区的其它变换

利用 Edit 菜单中 Selection 子菜单里的功能，可以对选区做 x 轴或 y 轴翻转、整体旋转，或按给定距离移动。例如 Flip Vertically 就是沿 x 轴翻转选区；用 Selection 子菜单中的 Rotation By Angle 则可以把选区旋转任意角度。

2.2.11 局部编辑

对象创建之后需要修改时，就要用到局部编辑（partial editing）。“局部”指的是只编辑 Polygon 或 Path 的一部分，例如只移动 Polygon 的一个顶点或一条边。局部编辑模式还允许删除单个顶点或边，也允许插入新的顶点或边。在局部编辑模式下可以同时选中多条边或多个顶点，甚至可以先选中整个 Shape 再做局部处理。

移动所选局部时受 movement direction constraint 约束，各模式的细节见第 2.1.2 节。例如在曼哈顿模式下只允许水平与竖直移动。同样，移动鼠标时按 Shift（正交）、Ctrl（斜向）或同时按 Shift 与 Ctrl（任意角度）可临时覆盖全局约束。

点击工具栏上的 Partial 按钮进入局部模式，此后单击即可选中某个局部（边或顶点），也可以拖出选择矩形。与常规选择一样，Shift 与 Ctrl 修饰键可以把新内容追加到已有选区，或从选区中移除。局部选择同样受“仅顶层”约束（该模式的说明见第 2.1.2 节）。

直接单击某个条目会立即进入 move 模式：在目标位置摆好该元素，左键单击放下即可；按 Esc 取消操作。存在复杂的选区时，要单击选区中某个条目（是边或顶点本身，而不是它们所属的 Shape）才进入移动模式。移动过程中版图视图顶部会显示两个输入框，可在其中输入位移量。

移动局部时受若干约束，例如单独的边只能沿垂直于其当前位置的方向移动；此外移动被限制在编辑网格上。

选中的条目可以用 Edit 菜单的 Delete 功能或按 Delete 键删除。若剩余顶点不足以构成合法对象（例如少于 3 个点的 Polygon），该对象会被整体删除。

在边或 Path 的段上双击，会在光标所在位置于该边上新建一个点。要在 Path 上做出一个“折弯”，可以在该段上放两个新顶点，再把两点之间的那一段从原来的中心线移开：这基本上需要对 Path 中心线双击两次，对由此形成的新段单击一次，再单击一次把它放到新位置。

2.2.12 把 Shape 移到其它图层

选中的 Shape 可以整体移到别的图层：从 Edit 菜单的 Selection 子菜单选 Change Layer，所有选中的 Shape 都会被移到 layer 列表中当前图层（带矩形标记的那个）上。Shape 不会跨层次移动，只在它们所在的 Cell 内换层。

所有图层（源与目标）必须位于同一个版图中。要把 Shape 移到别的版图，请用复制粘贴。

也可以在 Shape 属性中修改图层，效果同样是把这些 Shape 移到别的图层。

2.2.13 其它图层操作

可以用 Edit 菜单 Layer 子菜单中的 Edit Layer Specification 修改图层定义：弹出的对话框中可编辑 layer 面板里当前选中图层的层号、datatype 与（OASIS）名字，保存后这些 Shape 就被映射到新的图层上。

可以用 Edit 菜单 Layer 子菜单中的 Clear Layer 清空图层，范围可选按 Cell、按某个 Cell 的层次，或所有 Cell。

图层可以用 Edit 菜单 Layer 子菜单中的 Copy Layer 复制，用 Delete Layer 彻底删除。

2.2.14 复制与粘贴选区

复制粘贴当然也照常支持，而且可以在版图之间复制 Shape：打开两个版图，即可把 Shape 从一个版图移到另一个。Shape 会被映射到与源版图中相同的图层；若目标版图中还没有该图层，会自动创建。

选区中的 Shape 按其在当前 Cell 中的呈现方式直接复制到剪贴板。这意味着，若把它们粘贴到另一个版图，位置保持不变，但会打平放进当前 Cell 中。这提供了一条打平（flatten）层次的办法：在 editor options 对话框中选择层次选择模式（取消 `top level only`），选中要打平的 Shape，再把它们复制到另一个 Cell。

在非层次选择模式（即 `top level only` 选择模式）下，或在层次层级受限时单击 Cell 边框，也可以选中 Instance。把 Instance 复制到剪贴板有两种方式：

- Shallow copy：只复制 Instance 本身。粘贴到任何目标版图时，会查找该 Instance 所指的 Cell 并把它实例化。
- Deep copy：不仅复制 Instance，连被实例化的 Cell 一并复制。粘贴到别的版图时，该 Cell 也会一并建立；若已存在同名 Cell，则新建一个 variant 并实例化它。

2.2.15 删除 Cell

要删除整个 Cell，在层次浏览器中选中它，然后在上下文菜单（右键）中选择 `Delete Cell`。此时提供三种模式：

- Shallow delete：只删除该 Cell 本身（其 Shape 与 Instance），不删除它引用的任何 Cell。这些被引用的 Cell 此后可能不再被引用，因而可能作为新的顶层 Cell 出现在版图中。
- Deep delete：删除该 Cell 及其所有子 Cell，除非子 Cell 还被别的（未被删除的）Cell 引用。用这种模式可以移除一整套层次而不产生副作用。
- Complete delete：删除该 Cell 及其所有子 Cell，即使其它 Cell 仍在引用这些子 Cell。

2.2.16 重命名 Cell

要重命名 Cell，在层次浏览器中选中它，然后在上下文菜单（右键）中选择 `Rename Cell`，输入新名字即可。新名字不能与已有 Cell 重名。

2.2.17 复制与粘贴 Cell

整个 Cell 也可以复制到剪贴板：在层次浏览器中选中该 Cell（确保焦点在该窗口），然后从 `Edit` 菜单选择 `Copy` 或 `Cut`。要把这样的 Cell 粘贴到目标版图，从 `Edit` 菜单选择 `Paste`。

提供两种复制模式。当选择 `copy` 或 `cut` 且该 Cell 实例化了其它 Cell 时，会弹出对话框供选择模式：

- Shallow copy：只复制该 Cell 本身，不复制子 Cell。复制出的 Cell 调用与原来的 Cell 相同的那些 Cell。若把 Cell 复制到另一个版图（另一个标签页），子 Cell 不会随行，此时会创建“幽灵”Cell，或以同名已有 Cell 充当子 Cell。
- Deep copy：把该 Cell 连同其子 Cell 一起复制。所有 Cell 都随行，粘贴时会把全部子 Cell 的副本一并建立。

当 Cell 被粘贴到另一个版图、而那里已有同名“幽灵 Cell”时，粘贴的 Cell 会取代该幽灵 Cell。若那里已有同名普通 Cell，则会新建一个 Cell variant，并给粘贴的 Cell 加后缀改名以保证名字唯一。

用深复制模式把 Cell 从一个版图复制到另一个版图，是把两个版图合并成一个的简便办法：例如把第一个版图的顶层 Cell 复制到第二个版图，再在新顶层 Cell 中同时把两者实例化。

2.3 基本编辑操作

本节介绍编辑版图时的一些基本操作。

2.3.1 整版图变换

有一些函数可以对整个版图做变换。整版图变换会以相同方式作用于所有 Cell，每个 Cell 都按同样方式修改。该功能对版图缩放尤其有用。值得注意的是，以这种方式缩放版图不会产生带放大倍率的 Instance。

版图变换函数位于 `Edit` > `Layout` 中（翻转、旋转、缩放、移动）。

2.3.2 搜索与替换

KLayout 提供“搜索与替换”功能，既有基本的查找、查找并替换，还有一个非常通用而强大的扩展功能——custom query（见下文）。对话框位于 `Edit` 菜单的 `Search and replace`。

对话框左侧面板有四个标签页：`Find`、`Delete`、`Replace` 与 `Custom`，其功能分别说明如下。

对话框左侧显示会产生结果的操作（例如 Find）的结果。结果以列表呈现，条目内容随操作性质与参数而不同。若条目代表版图对象（例如某个 Shape 或 Cell Instance），列表中选中的条目会在版图中高亮以指示其位置。为避免性能问题，列表长度有限制。

点左下角的 `Configure` 按钮可打开配置页，用于设定该对话框的行为；在 setup 对话框的 `Browsers > Search Result Browser` 中也能找到它。

2.3.3 Find（查找）

Find标签页的功能很简单：可以指定各种条件，按下 Find按钮后列出所有符合条件的对象。

该功能的参数包括：对象类型、Cell 范围以及对象特定的条件。对象类型可以是Instances或某种 Shape；对 Shape 还可把类型限定为 Box、Polygon、Path或 Text，从而启用各自的专属条件。

Cell 范围（cell context）可取以下之一：

- Current cell：只看当前 Cell，忽略子 Cell。
- Current cell and below：看当前 Cell 及其所有子 Cell，子 Cell 的各次实例化都会被计入。
- All cells：逐个 Cell 单独查看，但不考虑 Cell 如何被实例化。

视对象类型，可纳入条件的参数各不相同。每个条件针对一个具体参数，通常由一个运算符（小于、不大于、等于……）与一个值组成，用来检查该参数的值。长度与面积以微米或平方微米为单位给出。

字符串值可以用波浪号匹配运算符（\ui{~}）及不匹配运算符（\ui{!~}）按 glob 通配符比较。glob 通配符就是命令行里文件名所用的那一套：*表示任意长度字符序列，?表示单个任意字符。例如：

A*

字符串必须以大写“A”开头

A

字符串某处必须含大写“A”

ABC?

“ABC”后跟任意一个字符

N{AND,OR}

“NAND”或“NOR”

A[0-9]

“A”后跟一个数字

A[^0-9]

“A”后跟一个非数字字符

若值字段留空，则不对该参数作检查。所有被检查的条件都满足，对象才会出现在结果页中。

2.3.4 Delete（删除）

与查找页类似，同样要求对象类型、Cell 范围与对象特定的条件。

按 `Delete All` 按钮会直接删除所有符合条件的对象。

按 `Select+Delete` 按钮时，符合条件的对象先显示在结果页中（与 Find 类似），随后可以从中挑出部分或全部，按列表下方的 `Delete` 按钮删除。

2.3.5 Replace（替换）

与查找页类似，同样要求对象类型、Cell 范围与对象特定的条件。此外还可以为各参数指定替换值；某参数的替换值字段留空则不作替换。

对使用 glob 通配符匹配的字符串，匹配到的名称片段可以在替换串中复用。例如运算符为波浪号（~）、作用于某 Text 的字符串，匹配串写作 `A(*)`，替换串可写为 \ui{B\textbackslash{}1}，其中 `1` 表示匹配串中第一个括号的内容，于是该设置会把开头的“A”全都换成“B”。

按 `Replace All` 按钮会对所有符合条件的对象施加替换参数。

按 `Select+Replace` 按钮时，符合条件的对象先显示在结果页中（与 Find 类似），随后可以从中挑出部分或全部，按列表下方的 `Replace` 按钮替换。

2.3.6 Custom queries（自定义查询）

这个页面才把这套对话框的威力完全发挥出来。custom query 不仅能实现前三个页面（查找、删除、替换）的功能，还能做到远超这些简单场景的事情。

custom query 是类似 SQL 的语句，其中可嵌入表达式，并有一套描述 Shape 或 Cell 实例化细节的丰富语言。详细说明见第原文对应章节节。

custom query 页面上有一个输入框用于填写查询，下方有 `Execute` 按钮执行查询。最近用过的查询可以用输入框下方的下拉框调回编辑框。custom query 可以按给定名字保存并复用，已保存查询的列表可用其右侧的按钮管理；各按钮功能见其工具提示。

若切换标签页后再切回，custom query 会按另一标签页上当前所选功能对应的查询进行更新。

2.3.7 层次操作：打平 Instance、由选区建 Cell、在层次中上移

KLayout 提供若干操作，可以把 Shape 或 Instance 在层次中上移或下移，都可通过 `Edit` 菜单下的 `Selection` 子菜单进入。

- Flatten instances：用被实例化 Cell 的内容替换所选 Instance。KLayout 会询问是展开所有层级，还是只展开该 Cell 的第一层；若展开所有层级，该 Cell 就被解析成当前 Cell 层次中的一组 Shape。
- Move up in hierarchy：只适用于当前 Cell 的子 Cell 内部的选区（因此在 `top level only` 选择模式下没有意义）。选中的 Shape 与 Instance 被上提到

当前 Cell 这一层，并从原 Cell 中移除。若想不破坏性地把 Shape 上提，可以先复制再粘贴，这相当于对所选 Shape 做一次显式打平。

- Make cell from selection：把当前选中的对象移出，放进一个可以指定名字的新 Cell 中。

2.3.8 生成裁剪片

KLayout 提供一个工具，可以从给定 Cell 生成矩形裁剪片（clip）。可以指定一个或多个矩形，当前 Cell 沿这些矩形的边被切开，每个矩形生成一个新的 Cell 存放该矩形范围内的内容；若指定了多个矩形，所有裁剪片最后会归入一个主顶层 Cell，它在 Cell 层次中作为新的顶层 Cell 出现。

裁剪矩形可以由坐标给出，也可以取自另一个图层（该图层必须含 Box，这些 Box 会一并复制到输出中），或取自标尺：此时以标尺的起点与终点作为裁剪矩形的角点。因此不妨为此专门建一个 Box 外观的标尺类型。

裁剪是按层次进行的：子 Cell 也会被裁剪，可能因此产生 variant（若干裁剪片可以共用同一个 variant）。这样就能高效地从大型版图生成大块裁剪片。提示：若版图中含 45 度之类任意旋转角的 Cell Instance，裁剪结果不会完全精确。

2.3.9 打平 Cell

“平坦”Cell（flat cell）是没有层次的 Cell，即只含 Shape、不含任何子 Cell Instance。平坦 Cell 与其它 Cell 断开联系，因此打平是“冻结”某个 Cell 内容的一种手段：Cell 一旦打平，再改动其它 Cell 都不会影响它，也不会影响该 Cell 在别处出现的样子。反过来，平坦 Cell 逐个存储每个 Shape，因此无法共享几何。

层次化的 Cell 可以通过 `Edit` > `Cell` > `Flatten Cell`，或 Cell 列表上下文菜单中的 `Flatten Cell` 打平。该操作提供若干选项，例如要打平多少级层次，以及如何处理因该操作而变得孤立的子 Cell（orphan cell）：启用 prune 选项后，所有不再被需要的子 Cell 都会被删除，否则会生成新的 Cell。

`Flatten` 也可以作用于 Instance：此时该 Cell Instance 被移除，代之以该 Cell 内部的对象。因此 Instance 打平就是把某个 Cell 的内容拉进父 Cell，好处之一是可以直接编辑这些内容，而不必改动子 Cell 本身，从而避免因改动子 Cell 而在其所有实例化处产生连带影响。

按 Instance 打平可用 `Edit` > `Selection` > `Flatten Instances`；同样可以选项指定要打平的层次级数以及如何处理 orphan cell。

2.3.10 拆解阵列

Instance 阵列便于生成大量的规则 Cell 排布。有时需要把阵列拆开，例如要删除或修改阵列中的某一个 Instance。一种办法是创建 Cell variant（见第 2.3.19 节），但那

样也会复制被实例化的 Cell。

Edit > Selection 中的 Resolve Arrays 功能可以把阵列拆成一个个独立的 Instance，随后即可单独编辑。

2.3.11 PCell 操作

若某个 PCell 由“引导图形”派生而来，就可以从 Shape 创建 PCell。Basic 库中的 PCell 除少数例外都支持由引导图形派生。例如，选中若干普通 Polygon 后选 Edit > Selection > Convert To PCell，并把 PCell 选为 Basic.ROUND_POLYGON，就能得到圆角 Polygon。

PCell 可以通过 Edit > Cell > Convert Cell To Static 或 Edit > Layout > Convert All Cells To Static 转成普通 Cell。普通（静态）Cell 可以单独编辑，但不再提供控制该 Cell 外观的参数。

2.3.12 图层布尔运算

KLayout 自带一组布尔运算，位于 Edit 菜单的 Layers 子菜单中（Boolean Operations 与 Merge 两个功能）。点击后会弹出对话框，可指定模式、输入图层、输出图层以及其它一些选项。

- AND：交集。输出图层包含图层 A 与图层 B 的 Shape 相互重叠的所有区域。
- A NOT B：差集。输出图层包含图层 A 的 Shape 未与图层 B 的 Shape 重叠的所有区域。
- B NOT A：差集。输出图层包含图层 B 的 Shape 未与图层 A 的 Shape 重叠的所有区域。
- XOR：对称差。输出图层包含在两个图层中仅出现于其一的区域。

此外还提供 MERGE 运算，这是单图层操作，把该层上的所有 Shape 并起来。其特色是允许指定最小重叠数：0 表示只要有一个 Shape 就产生输出，1 表示需要两个 Shape 重叠才产生输出，依此类推。此规则不适用于单个 Polygon：Polygon 自身的自重叠不计入。

所有运算都可以在三种层次模式下进行：

- Flat：两个图层都打平，结果放入当前顶层 Cell。
- Top cell：只在顶层 Cell 的 Shape 上做运算。
- Cellwise：在当前顶层 Cell 之下的所有 Cell 上逐个做运算。该模式仅在两个输入与输出的版图相同时允许。

前两种模式下，源版图与目标版图可以不同，前提是所有版图都加载在同一视图中。这样就可以把不同版图的图层组合起来，例如用 XOR 比对它们。

KLayout 布尔运算还有一个特色：可以选择“尖角相接”（kissing corner）情形如何处理，提供两种模式：

- Maximum coherence：输出尽量少而连贯的 Polygon。这些 Polygon 可能多次包含同一个点，因为轮廓可以回到同一点而不闭合。
- Minimum coherence：输出尽量多的、可能彼此相触的 Polygon。

下面两张截图演示对两个矩形做 XOR 时 maximum coherence（左）与 minimum coherence（右）两种模式的结果。

布尔运算目前是以平坦方式实现的，基于全层级的边表示：若要求 `flat` 模式，整个图层会被打平并转成一组边，再由处理器在其上运算。对超大型版图这会带来巨大的资源需求，目前不建议用于此类场合。

布尔处理器采用迭代方式以覆盖网格吸附效应，因而精度很高，但比单遍扫描线实现略慢，性能损失约为 2 倍。

2.3.13 图层 sizing

sizing（尺寸调整）运算可以把某图层的 Shape 按给定偏置放大或缩小，偏置按边施加：正值放大，负值缩小。偏置可以分别给出水平与竖直方向的数值，但两者的符号必须相同（例如 `0.5,0` 或 `1.0,0.2`，而不能是 `0.2,-0.2`）。

sizing 功能位于 `Edit` 菜单的 `Layers` 子菜单中。弹出的对话框可指定输入与输出图层，以及以微米为单位的 sizing 值：一个数值表示 x、y 方向同等缩放，或用逗号分隔的两个数值（例如 `0.2,0.1`）分别指定。

与布尔运算一样，可以指定层次模式与尖角相接的处理方式（各模式说明见第 2.3.12 节）。此外，锐角的截断（cutoff）策略可从“严格”到“近乎不限”中选择。下面的截图演示 `stricT`（红色）到 `weak`（紫色）各截断模式的效果。

2.3.14 按 Shape 的布尔运算

布尔运算也可作用于所选的 Shape 集合，这类运算使用“主选区”（primary）与“次选区”（secondary）的概念：主选区包含第一步选中的所有 Shape，次选区包含随后按住 Shift 修饰键追加选中的所有 Shape。

以下操作位于 `Edit` 菜单的 `Selection` 子菜单中：

- Merge：把主选区与次选区中的所有 Shape 合并，结果写入主选区所在图层。
- Intersection：求主选区与次选区的交集（AND），结果写入主选区所在图层。
- Subtraction：求主选区（A）与次选区（B）的差集（A NOT B），结果写入主选区所在图层。

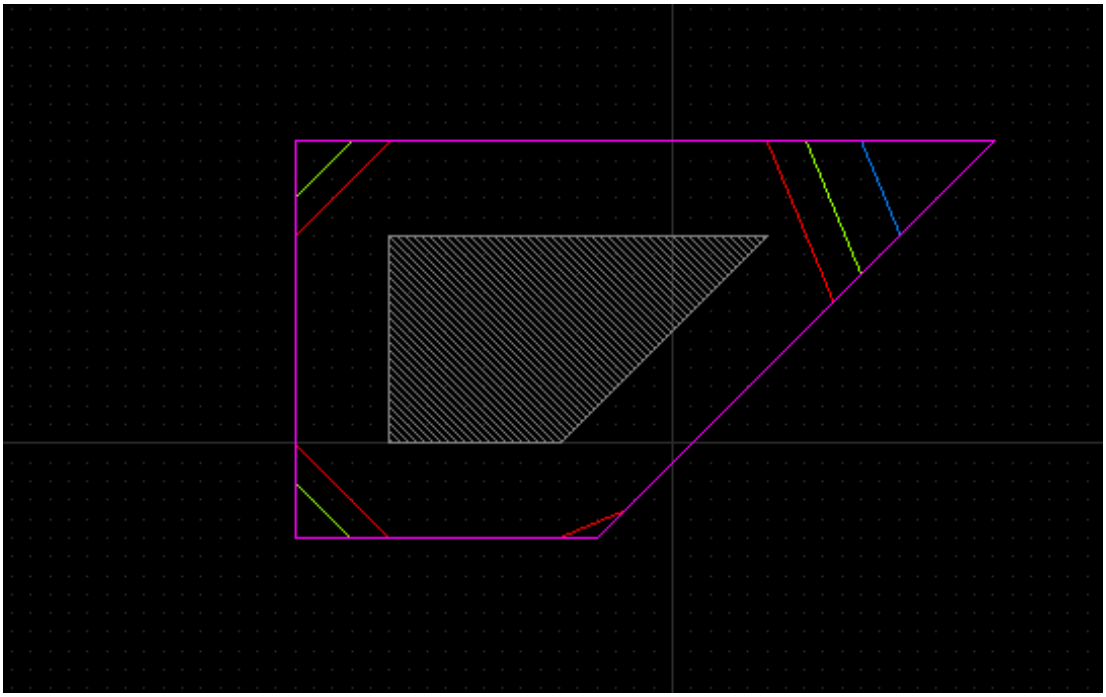


图 2.2: sizing 的截断模式

2.3.15 按 Shape 的 sizing

选中的 Shape 可以按给定的放大与缩小距离做 sizing，用法类似图层操作但选项更少。sizing 功能位于 `Edit` 菜单的 `Selection` 子菜单中，弹出的对话框要求输入 sizing 值（以微米为单位：一个数值表示 x、y 同等缩放，或逗号分隔的两个数值分别指定 x、y 方向）。

2.3.16 对象对齐

该操作使用主选区与次选区的概念：主选区包含第一步选中的所有 Shape，次选区包含随后按住 Shift 修饰键追加选中的所有 Shape。

对象对齐功能可以把次选区中的所有对象对齐到主选区的对象上（即主选区的对象只提供参考点，本身不被移动）。“对象”可以是 Shape，也可以是 Cell 的 Instance；Cell Instance 以其边界框参与对齐，边界框既可以只按可见图层计算，也可以按所有图层计算。

水平与竖直方向可以分别指定对齐方式。水平方向可取 `none`（不变）、`left`（左边对齐）、`center`（中心对齐）或 `right`（右边对齐）；竖直方向可取 `none`、`bottom`、`center` 或 `top`。

对齐功能位于 `Edit` 菜单的 `Selection` 子菜单中。弹出的对话框可指定对齐方式，以及 Cell Instance 边界框的计算方式。

2.3.17 圆角处理

某些应用（例如功率器件）希望用圆角代替尖角，以限制电场。KLayout 为此提供了一种便捷做法：先用 90 度尖角画出结构，再按给定半径把角“软化”成圆角，得到的 Polygon 可以直接写进 GDS 文件——尽管 GDS 本身并不支持圆弧。

有意思的是：圆角功能可以按 Polygon 逐个重新施加在这样得到的几何上。也就是说，即便这些被处理过的 Polygon 已存进 GDS 或被其它方式改动，仍能重建原始几何并修改圆角半径，无需特殊的几何对象或特殊的 GDS 标注。这一要求带来了一些（可能较弱的）限制：

- 每个角上的点数不能太少（目前整圆至少 32 点）。
- 原始几何不得含尖角，且原始各段长度至少为圆角半径的两倍。
- 角的各段必须能被识别为“角”，即相邻边之间的角度须“接近”180 度。这对这类段的最小长度以及它们在数据库单位下的表示精度提出了限制，归根结底是数据库单位下的某个长度下限。

下面的截图演示圆角功能。可以看到在这个例子里，“内角”与“外角”需要允许指定不同的半径。

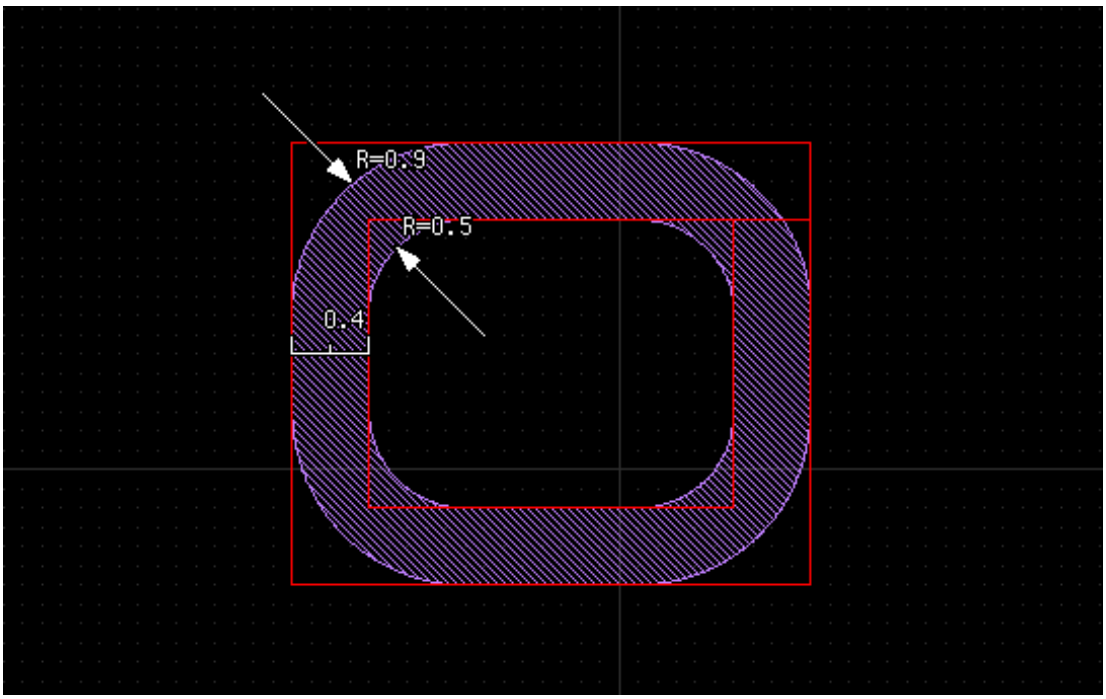


图 2.3: 内角与外角使用不同半径的圆角

圆角功能作用于所选的 Shape，位于 `Edit` 菜单的 `Selection` 子菜单中。弹出的对话框可指定半径值以及期望的分辨率。若所选 Polygon 此前已有圆角，会先去掉圆角、还原原始 Polygon，再施加新的圆角；把半径指定为 `0` 则可去除圆角。

2.3.18 调整 Cell 原点

Cell 的原点很重要，因为它是 Cell Instance 的实例化锚点。原点调整功能可以把原点移到相对于 Cell 边界框的某个位置：可以是边界框的中心、某个角，或某条边的中点。边界框既可以按所有图层计算，也可以只按可见图层计算。

若勾选 `Adjust instances in parents` 复选框，该 Cell 的各 Instance 会朝相反方向调整，于是 Cell 本身实际未变，只是在局部上版图的原点被挪了位置。

原点调整功能位于 `Edit` 菜单的 `Cell` 子菜单中。

2.3.19 创建 Cell variant

当你只想编辑某个 Cell 的某一个 Instance、而不想改整个 Cell 时，KLayout 提供了一个很有用的功能。编辑 Cell 意味着改动会出现在该 Cell 被放置的每一处；若只想在某一处改动版图、不希望波及其它地方，这就未必合适了。

常规做法是复制该 Cell，把某个 Instance 换成新 Cell，再编辑新 Cell。这相当繁琐，尤其当存在阵列 Instance 而只需修改其中一个时：必须先把阵列拆开，单独隔离出那一个 Instance，才能把它换成新 Cell。

KLayout 提供了自动完成此事的办法，位于 `Edit` > `Selection` > `Make Cell Variants`。其做法大致是：对每个选中对象，沿层次一直追溯到当前 Cell，在沿途遇到的每个 Cell 都创建一份新的 Cell 副本，并用这些新 Cell 取代原来的。

效果是：用过 `Make Cell Variants` 之后就可以随意修改（例如删除）选区，而不会产生任何不良的连带影响。

该功能还能转换阵列 Instance，并隔离出阵列中的某些 Instance。下面几张截图演示这一点。

初始情形：某个 Cell 以 3×3 阵列被实例化 9 次，并且选中了其中某个 Instance 内部的一个 Shape。随后两张截图分别显示全层级视图与仅顶层视图，以表明该 Cell 被放置了 9 次。

如果把这个 Shape 删掉，它在所有 9 个 Instance 中都会消失——假定这并不是想要的。此时可以用 `Make Cell Variants` 为选中该 Shape 的那一个 Instance 创建 Cell variant。做完之后，其中一个 Instance 会被替换为该 Cell 的副本，命名为 `INV2$1`，情形如下：

现在就可以删掉这一个 Shape，而对其它 Instance 毫无影响：

2.4 高级编辑操作

本节介绍编辑模式中较为高级的功能。

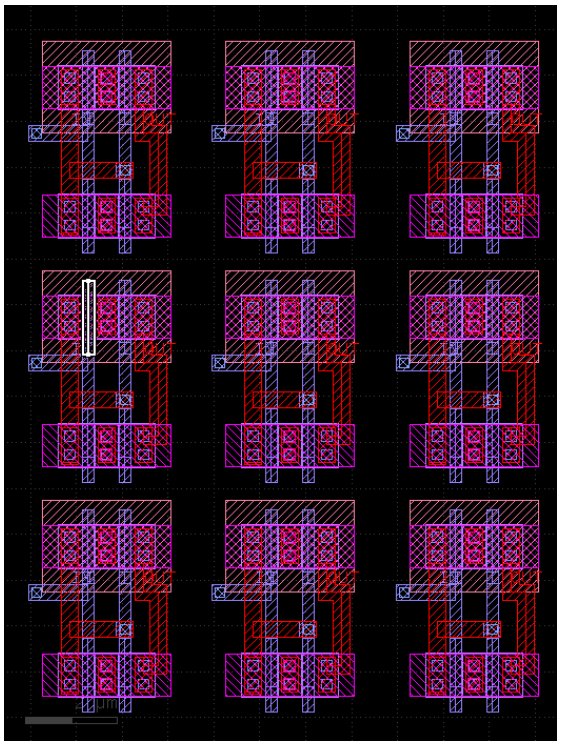


图 2.4: 全层级视图：Cell 以 3×3 阵列放置

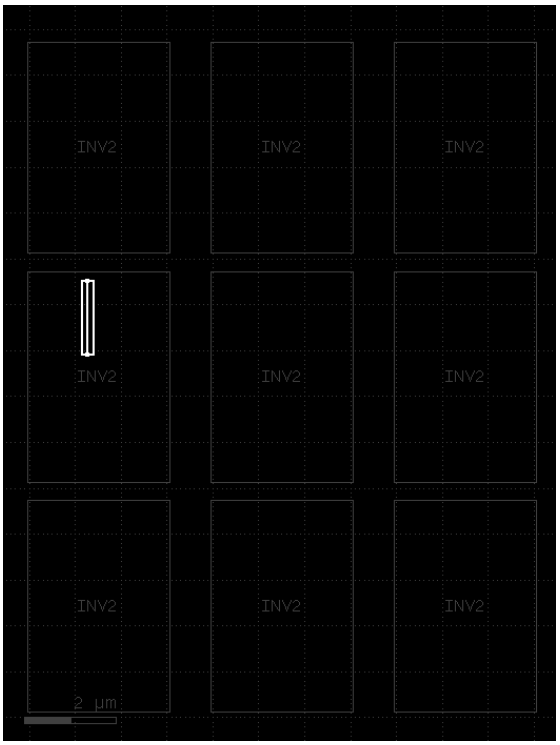


图 2.5: 仅顶层视图

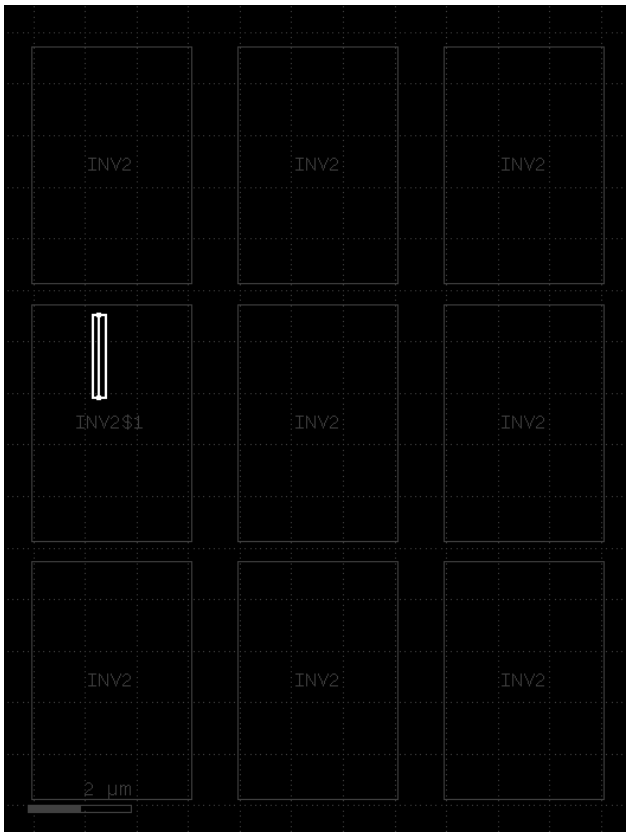


图 2.6: Make Cell Variants 之后：一个 Instance 被替换为副本 INV2$1

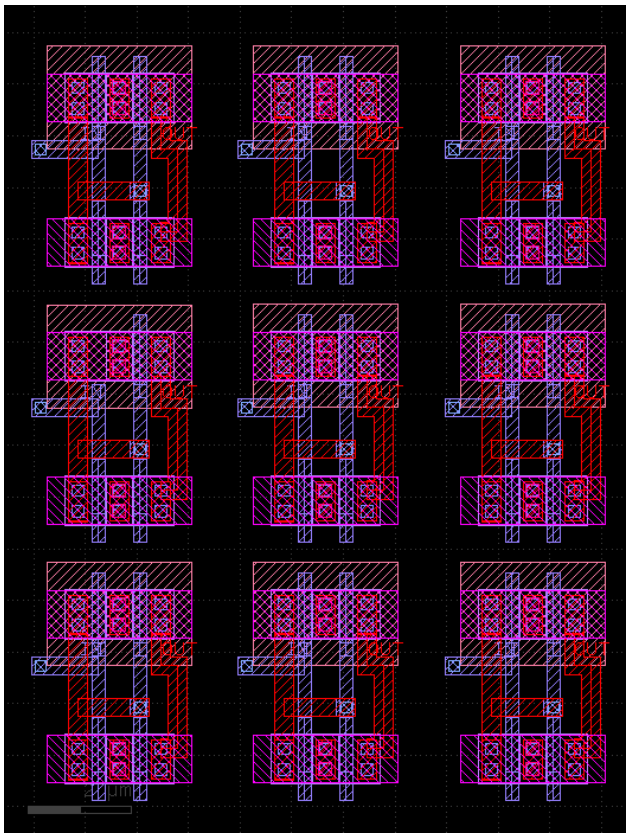


图 2.7: 删除单个 Shape 后其余 Instance 不受影响

Chapter 3

Advanced Topics

3.1 XOR 工具

XOR 工具通过对两个版图逐层执行相应的布尔运算，实现几何 XOR（非对称差还可用 ANOTB 与 BNOTA）。从 `Tools` 菜单的 `XOR Tool` 启动。目前可以比较所有图层或仅可见图层；比较的是某个版图与另一个版图中同名同号的对应图层。

目前的实现采用平坦的 XOR 处理器，因此大体上只适用于中小型版图，不利用层次结构；这基本上排除了对层次很深的应用（例如存储器阵列）。平坦方式带来的内存占用可以用 tiling 功能缓解——把运算限制在一个尺寸有限的 tile 上。这能降低内存峰值，但不会缩短运行时间。

XOR 工具可以指定容差。容差本质上是在布尔运算之后做一次 undersize，借此抑制小的标记，这对剔除因被比较版图间细小差异而产生的标记特别有用。可以指定多个容差，此时会做多次 undersize，生成若干容差各不相同的图层集合。

在输入框中填入 tile 尺寸即可启用 tiling。对标准单元块这类半平坦版图，1000 微米是个不错的起点；tile 尺寸的选择主要决定内存需求。

XOR 工具可以把结果送进 marker 数据库，或写入另一个版图（也可以是某个输入版图）。模式用 `Output` 下拉框选择。若输出写到某个原始输入版图中，必须指定图层偏移，把原图层映射到新图层：例如偏移 `1000/0` 表示 `16/0` 层上 Shape 的差异会被写到 `1016/0`（用于第一个容差类别）。

最后，XOR 还可以限制在某个区域内，若只关心版图的局部差异就能省时间。选择区域用 `From region` 旁的下拉框：

All：对整个区域做 XOR（默认）

Visible region：把 XOR 限制在版图的可见部分

Clipped to ruler：画一条或多条标尺来指定 XOR 的范围，每条标尺以其延伸范围给出一个区域。若用 Box 类型的标尺，区域的可视化效果更好

3.2 Diff 工具

与 XOR 工具一样，Diff 工具也对两个版图做比较。区别在于它逐个 Cell、逐个对象地比对，并报告有差异的 Cell、Instance 与几何对象。因此这种比较更严格，也不纯

粹是几何层面的：它并不在掩模层面判断两个版图是否等同，而是判断构成版图文件的对象是否完全一致。

通常这类比较对“装饰性”改动（例如 Cell 改名）非常敏感。KLayout 的 Diff 工具用以下特性来缓解这一问题：

- 在逐 Cell 比较之前，它会先通过比较实例化方式来识别被改名的 Cell，从而即便名字不同也能比对到正确的 Cell。该功能的基础是 Cell 匹配算法：算法综合边界框、各图层 Shape 数、Instance 数等参数来比较 Cell，并为每个 Cell 选出合适的配对。
- 它可以部分控制比较的严格程度。例如可以在比较各 Instance 之前先把 Cell 阵列展开；默认情况下，user properties 或某些文本属性之类的二阶信息不参与比较。
- Diff 工具还可以工作在 XOR 模式下：把找到的差异作为后续仅针对 Polygon 的 XOR 步骤的输入，结果相当接近真正的“按打平处理”的 XOR，且是真正 XOR 结果的超集——可能把某些其实相同的位置报为不同，但不会漏报真实差异。与 XOR 工具相比，这里少了一些选项。

Diff 工具位于 `Tools` 菜单中。在其对话框里：

- 在 `Input` 区选择版图 A 与 B。
- 取消勾选 `Don't use names to match cells` 可退回纯按名字匹配，此时被改名的 Cell 不会参与比较。
- 勾选 `Run XOR on differences` 即选择 `XOR mode`。
- 勾选 `Summarize missing layers`，把缺失图层汇总成一条差异报告，而不是每个 Shape 一条。
- 勾选 `Detailed information` 可获得每条差异的详细信息；不勾选时只报告有差异的 Shape 或 Instance 数目。
- 勾选 `Expand cell arrays` 可对阵列 Instance 的各个成员分别比较。
- 勾选 `Exact compare` 可把二阶信息（例如 user properties、文字朝向）也纳入比较。

Diff 工具会创建 marker 数据库，并在 marker 数据库浏览器中显示结果。

3.3 填充（tiling）工具

fill 工具在版图的特定区域内生成规则的填充单元（fill unit）Instance 阵列，通常称为 tiling 或 fill。它以矩形单元为基础，在 x、y 方向重复该单元来填满可用空间；多数情况下目的是填补版图的空白区域，提高版图的均匀性以改善工艺表现。

使用 fill 工具之前，必须先在要被填充的版图里准备好一个填充 Cell。该 Cell 的尺寸由画在任意图层上的一个 Box 定义，这个 Box 必须代表该 Cell 的“占位”（footprint），即一个 Instance 在待填区域内所覆盖的空间。

fill 工具位于 Edit 菜单的 Utilities 子菜单中，且仅在编辑模式下可用。使用该工具需打开对话框，然后：

- 选择填充区域的外边界（即“填什么”）。可选：整个 Cell、某图层上某 Polygon 的内部、所有选中 Polygon 的内部、单个 Box，或由某条标尺定义的区域。
- 指定填充区域是否要与边界保持某个最小距离。
- 指定填充区域内不得填充的区域。可选：所有图层（不在任何 Polygon 之上填充）、所有可见图层、所有选中图层，或不排除任何区域。
- 若填充单元必须与排除区域保持某个最小距离，在 `Spacing around exclude areas` 输入框中填写该距离。
- 在 Fill Cell 组中指定填充 Cell，以及定义该 Cell 占位的边界图层。

默认情况下 fill 工具在固定栅格上操作，某些场合会导致填充效率不佳。它提供 Enhanced fill 选项，尝试寻找不一定要落在同一公共栅格上的单元排布，以获得更好的填充效果。此外还支持二阶填充：再指定一个（通常更小的）填充 Cell，用来填补剩余区域。

下面的截图展示不同填充模式在一个刻意构造的填充问题上的效果。

默认：

增强：

增强 + 二阶填充：

3.4 导入 Gerber PCB 文件

Gerber PCB 导入功能可以把 Gerber PCB 文件转成 GDS 版图数据，或把 Gerber 文件作为新图层加入 GDS 文件。导入器支持 artwork 文件的大部分 RS274X 特性，也支持钻带文件的几种格式。它接收一组文件并转换为版图几何，同时提供一些功能对数据做相应调整，例如定义输出图层、施加几何变换。

由于选项繁多，导入配置可能变得相当复杂，因此可以以 XML 格式存成文件（建议后缀 .pcb），其中保存导入器的各项配置。一旦有了这样的文件，KLayout 就能像读取普通 stream 文件那样读取它（例如在命令行给出），并把它当作配方来导入与之关联的 Gerber 文件。

PCB 导入功能位于 File 菜单的 Import > Gerber PCB，提供多个入口，分别用于新建工程、打开已有工程，或接着上次的工程继续。

导入 PCB 数据的基本流程是：

- 指定 PCB 数据文件所在的目录（base 目录）。
- 指定导入模式（即版图数据的去向）。

- 决定图层映射方式：自由图层映射，或金属叠层映射。自由映射允许 PCB 图层与 GDS 图层之间任意对应，最为灵活但填写繁琐；金属叠层映射较易填写，但只限于把一组 PCB 文件映射到 metal/via/metal 叠层方案上。
- 指定文件、GDS 图层，以及 PCB 到 GDS 的图层映射。
- 按需指定变换，可以直接给出变换，也可以用对应点来指定。
- 决定其它选项（例如是否合并、数据库单位、顶层 Cell 名等）。

首先要决定的是如何指定图层映射。自由模式下，需按以下步骤填写：

- 在 `Files` 页的对话框中指定要加载的文件。
- 指定输出图层列表。
- 填写“输入到输出”的映射矩阵，为每个输入文件指定一个或多个输出图层。

在叠层模式下，填写流程为以下步骤：

- 指定 GDS 叠层，即可用于承载 PCB 数据的完整叠层。思路是在原有 GDS 叠层之上再叠一组 metal/via/metal 层：最靠近芯片表面的 PCB 层放进第一层金属（也就是所选叠层中符号最靠上的那层），依次往下排。
- GDS 图层定好之后，指定该 PCB 文件组中包含多少层金属、多少个钻孔文件，以及芯片是装在 PCB 的正面还是背面。后者决定 PCB 各层与 GDS 图层的对应顺序（第一层 GDS 层对应最靠近芯片表面的 PCB 层）。
- 为各金属层对应的 artwork 文件填写文件名。
- 为钻带文件填写文件名，并指明这些（镀通）钻孔连通哪些金属层。由于一个钻孔可以在叠层中连通多层，连接信息总是“从某金属层到某金属层”的形式，其间各金属层由该孔连通。

3.4.1 导入对话框

导入对话框按导入流程分成多页。在任一页上都可以用 `File` 菜单按钮把当前设置存成 PCB 导入工程、打开已有工程，或新建工程从头开始。

第一页提供一些基本选项：

- Base directory：所有 PCB 文件所在目录。文件不必都在该目录中，但会以它为基准查找。若整体搬迁了文件，只需改动这个基准目录。该目录不会被存进工程文件，而是每次由用户给出。
- Import mode：PCB 数据可以导入当前版图（进入当前 Cell），此时通常会给当前版图新增图层；也可以新建一个版图，放进新面板或加到当前面板中。
- Layer mapping mode：在这里选择自由模式还是叠层模式，勾选复选框即使用自由图层映射。

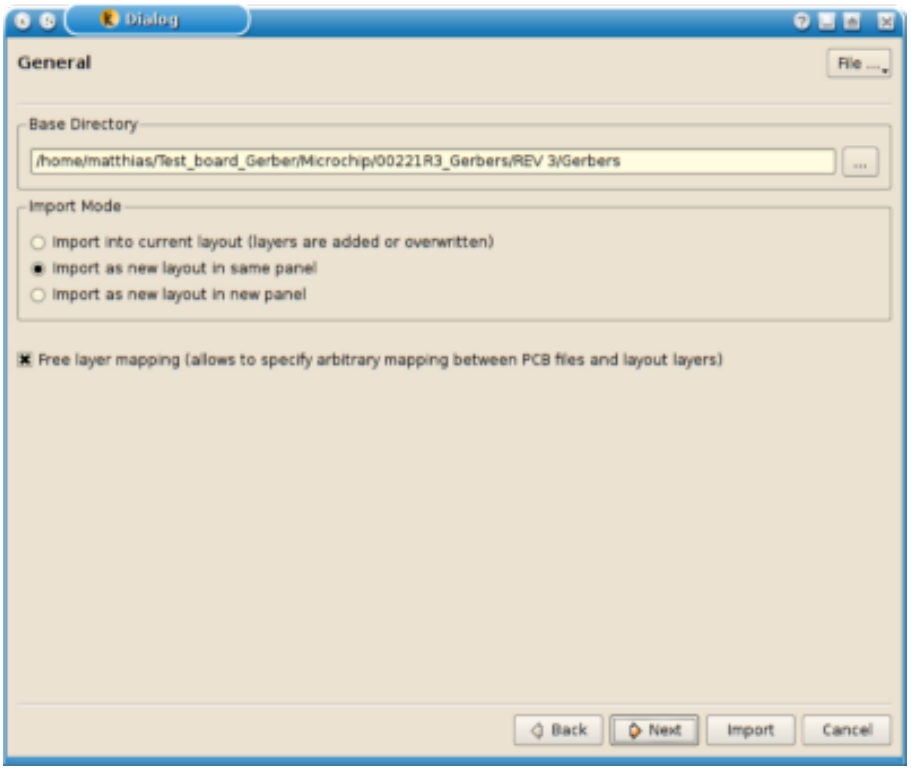


图 3.1: 自由映射流程第一步：基本参数

3.4.2 叠层流程

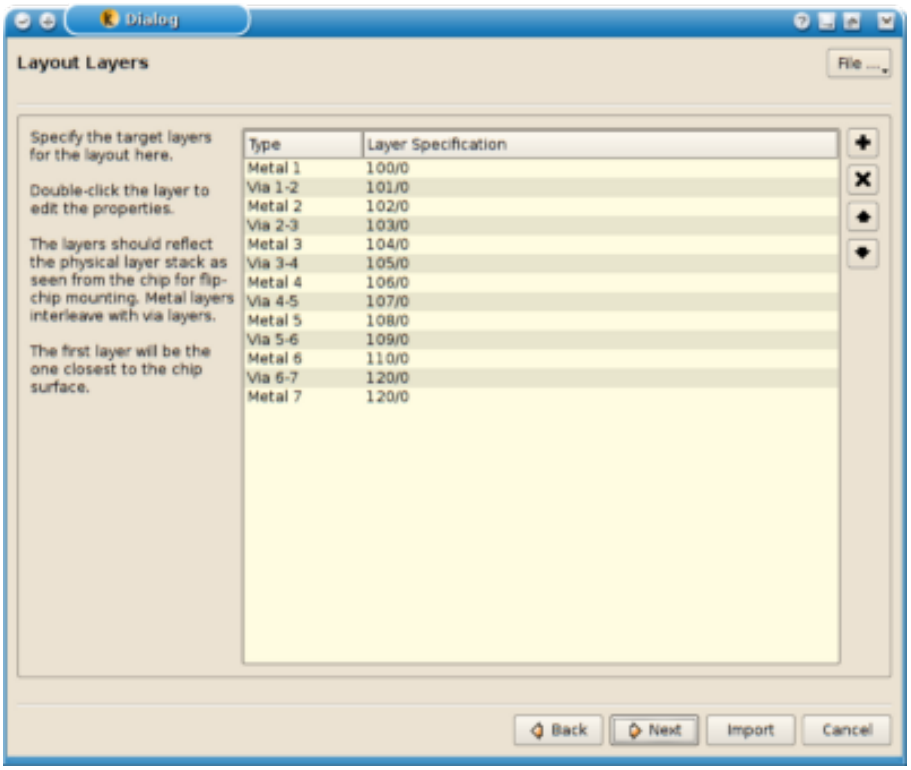


图 3.2: 叠层流程第二步：叠层定义

叠层模式下，第一页需要指定一串金属层与 via 层。金属层与 via 层是自动交替的：序列总是金属层后接 via 层，层数必须是奇数，以便最后一层仍是金属层。可以用 [+]按钮增加图层，选中图层后用上下箭头按钮移动，用 [X]按钮删除所有选中图层。

继续用 [+]按钮添加新图层；选中图层后可用箭头按钮上下移动；用 [X]按钮删除所有选中图层。

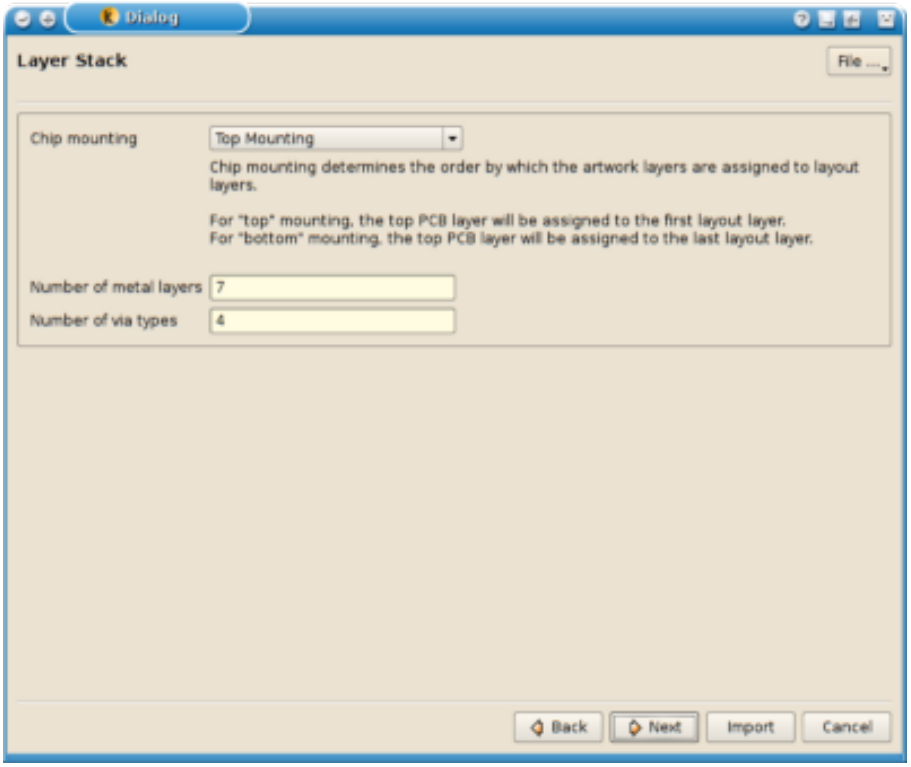


图 3.3: 叠层流程第三步：增删图层

下一页需要指定 artwork 文件与钻带文件的数量，之后再填入实际文件名并把它们分配给金属层或 via 层。此外还要指定芯片的安装位置：`top mounting`模式下假定芯片以正面朝下安放在 PCB 上。

在 `Artwork Files`页填写 artwork 文件名，它们会自动分配给相应的金属层，分配顺序取决于安装模式。

在 `Drill Files`页填写钻带文件名。每个钻带文件描述一道钻孔工序，可能连通多层金属；因此需要在本页指定该镀通孔所连通的首末金属层，中间的 via 层随之确定。

3.4.3 自由图层映射流程

在 `Files`页指定全部 PCB 数据文件，包括 artwork 与钻带文件。顺序无所谓，但建议按物理叠层的次序排列，便于后面分配给 GDS 图层。用箭头按钮上下移动选中条目。

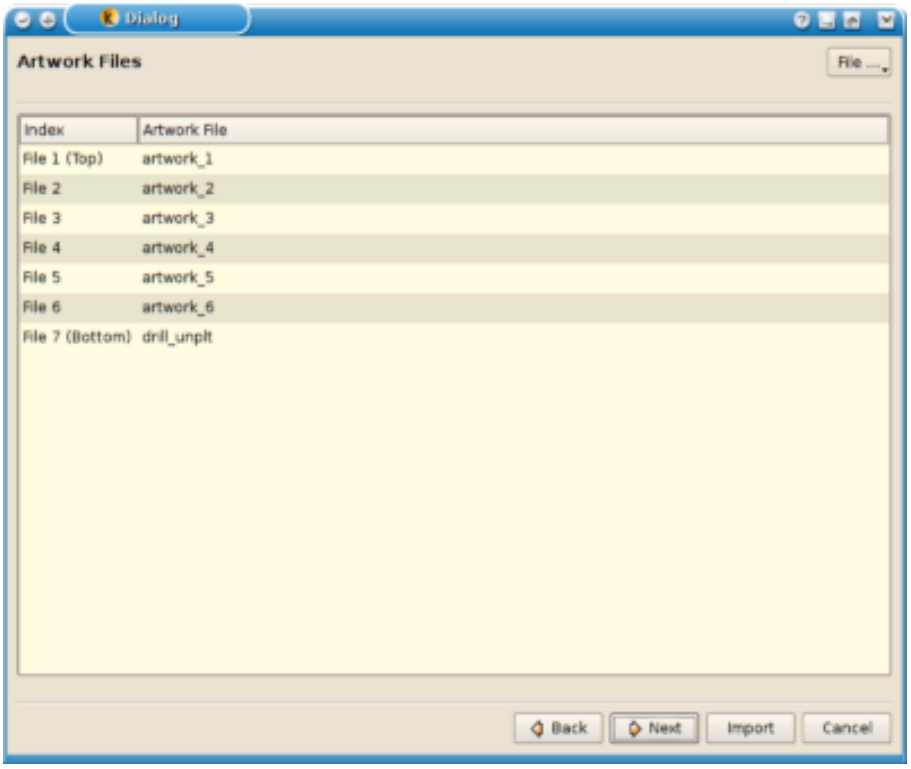


图 3.4: 叠层流程第四步：文件数量与芯片安装位置

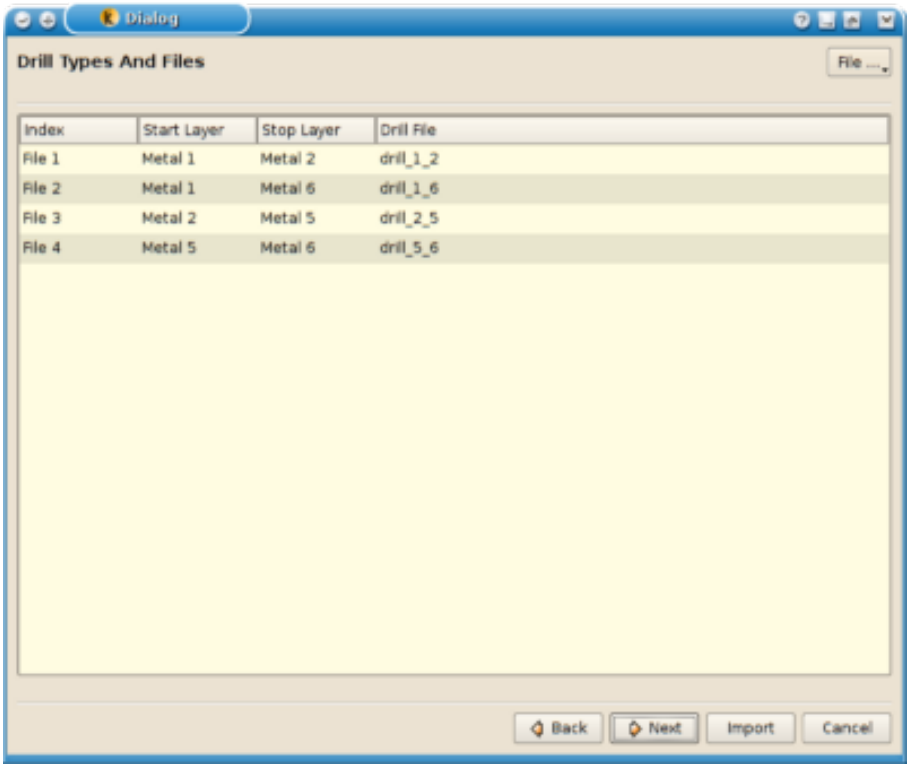


图 3.5: 叠层流程第五步：钻带文件与连通层

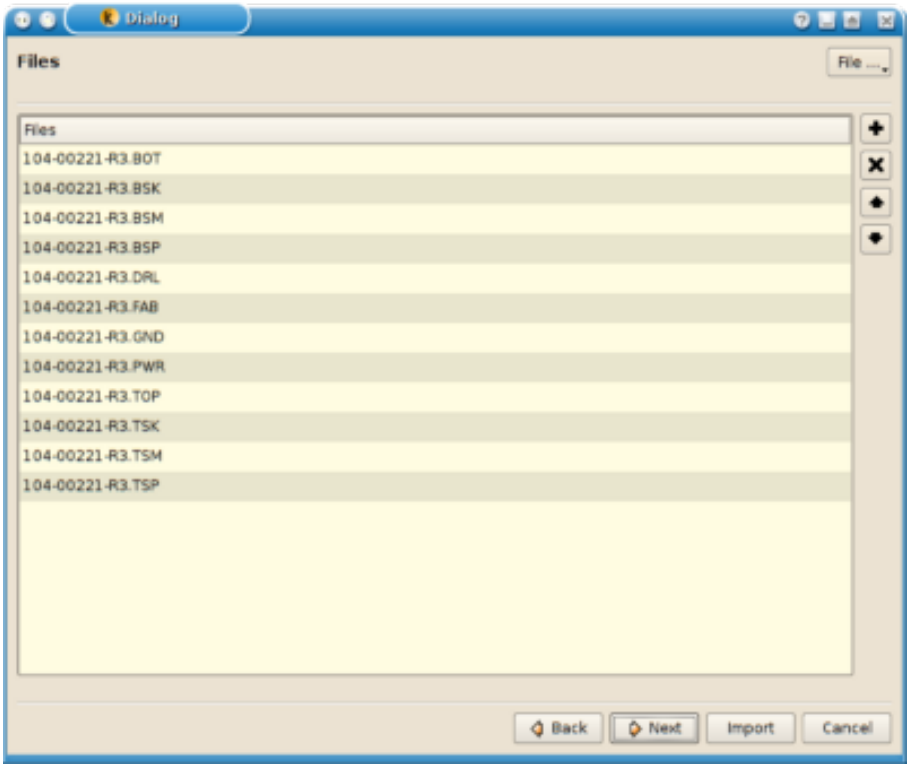


图 3.6: 自由映射流程第二步：文件列表

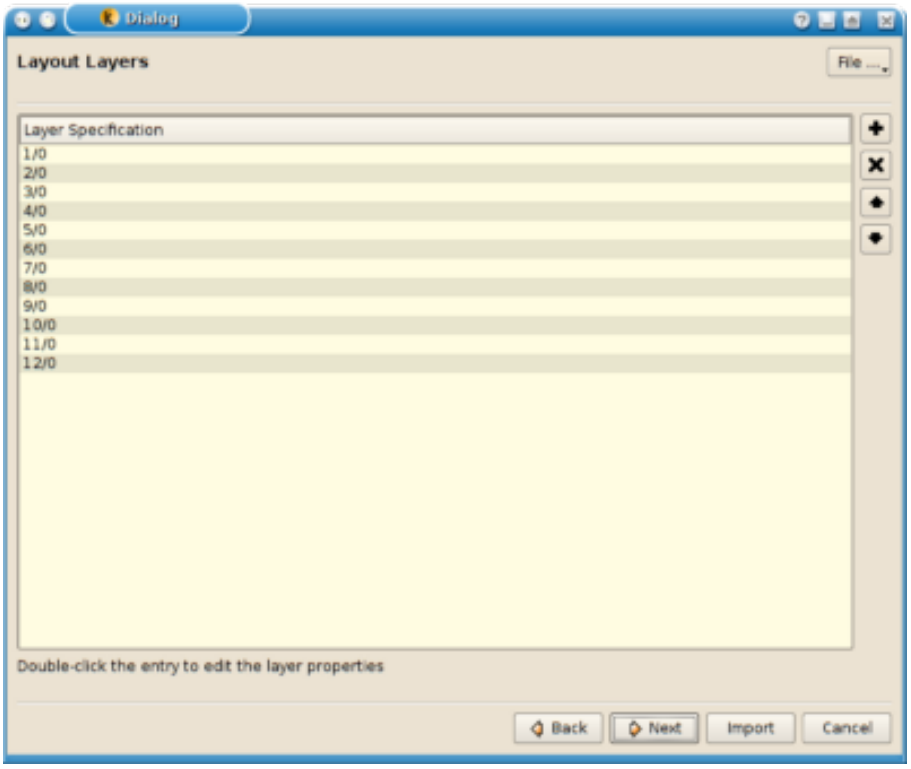


图 3.7: 自由映射流程第三步：目标图层

在 `Layers` 页指定全部目标图层，即把用作导入目标的图层列成一张表。顺序同样无所谓，但按工艺次序排列会让下一步的分配轻松一些。同样用箭头按钮调整顺序。

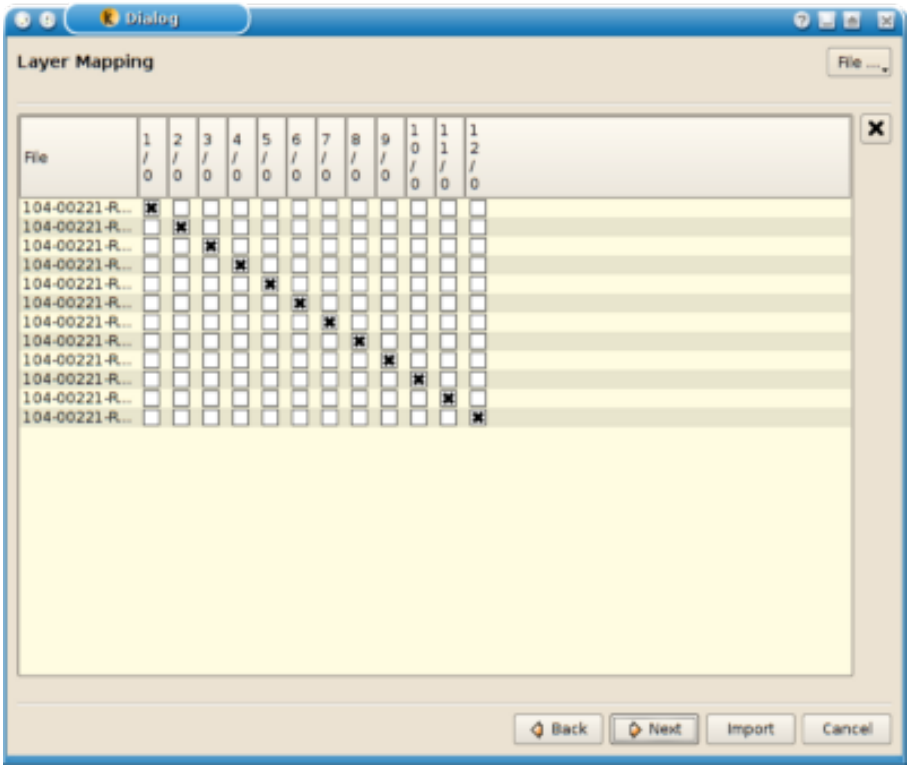


图 3.8: 自由映射流程第四步：图层映射矩阵

在 `Layer Mapping` 页把每个文件分配给一个或多个 GDS 图层。分配以矩阵形式表示：行是文件或图层，列是目标图层，矩阵中的“x”表示把该行的文件导入该列所指的图层。一个文件可以导入多个图层，效果上等于把内容复制到多个图层。

3.4.4 通用选项

`Coordinate Mapping` 页用于指定把 PCB 数据变换到 GDS 空间的方式。PCB 与 GDS 极少共用同一个原点，因此可以指定一个变换，在导入时施加到 Shape 上。

有两种指定方式：

- 给出对应点：系统会计算一个变换，把给定的 PCB 坐标映射到给定的 GDS 坐标。最多可以给三对坐标：给一对即可推导出平移；给两对可以在平移之外解出旋转；给三对则还可解出缩放。
- 直接指定变换：可以在下方输入框中显式写出变换。格式为：`x,y` 表示单纯平移（x、y 以微米为单位）；`rx` 或 `mx` 表示旋转角度 x、或沿角度 x 的直线镜像；`*x` 表示缩放系数 x；各种组合还可以叠加使用。

提示：两种指定方式可以组合使用，例如给出一对坐标确定平移，再显式指定旋转。

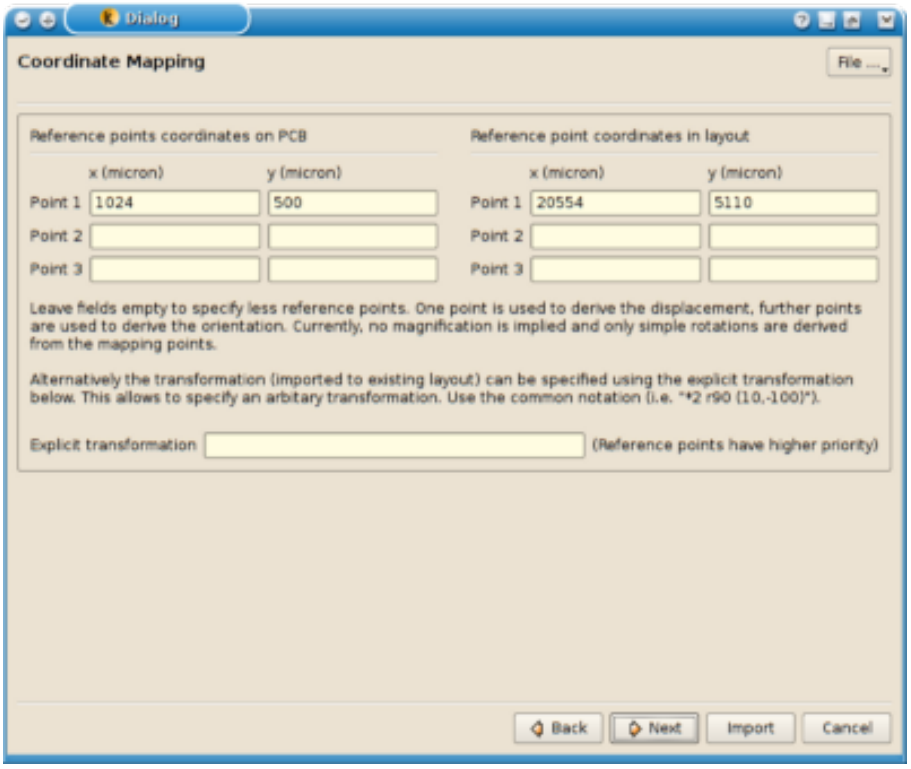


图 3.9: 叠层流程第六步：坐标变换

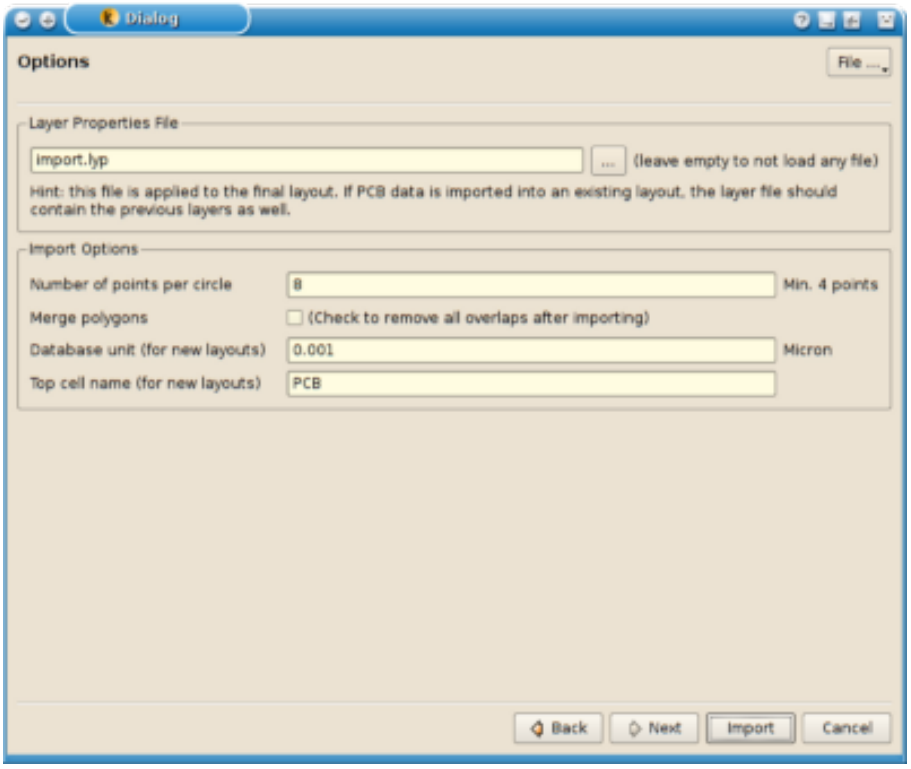


图 3.10: 叠层流程最后一步：选项

最后，Options页可以设置各种选项：

- Layer properties file：若指定，导入完成后会载入该图层属性文件；该文件路径相对于 base 目录。
- Number of points per circle：KLayout 会把 PCB 版图常用的圆形孔径解析成 Polygon 以做几何运算。该选项指定用多少点来近似整圆：点越少精度越低，但数据量也越小。
- Merge polygons：勾选后，所有重叠或相触的 Polygon 会被合并。注意，使用 clear 层时会隐式发生合并，因为用于挖去 clear 区域的布尔运算本身会合并既有版图，这种隐式合并无法关闭。
- Database unit 与 top cell name：用于为新版图指定数据库单位与顶层 Cell 名；仅在导入模式会新建版图时才生效。

3.5 导入其它版图文件

该功能可以把其它版图合并进当前已加载的版图。所谓合并，就是把指定版图的层次插入到当前版图之中；有多种模式控制层次如何合并。该功能位于File菜单的Import > Other File Into Current。

导入另一个版图的流程如下：

- 指定要导入的文件，至少需要文件名。此外还可以指定一个 Cell，此时只导入该 Cell（直接或间接）引用的那些 Cell。导入可以单独指定读取选项，其作用方式与常规加载时的读取选项相同。
- 指定导入模式，各模式见下文。
- 指定图层映射：Shape 可以按原图层导入，也可以使用一个偏移量，加到原图层上得到导入后的目标图层。例如偏移1000/0表示层号加 1000、datatype 保持不变。
- 指定可选的变换，导入的版图会据此变换。变换可以显式给出，也可以用最多三个点两两对应来指定。

有四种导入模式，控制被导入版图的层次如何插入现有版图：

- Merge：把被导入 Cell 的内容放入当前 Cell，其子层次加到当前 Cell 之下。
- Extra：创建新的顶层 Cell，其中包含被导入 Cell 的层次树。该模式下若被导入的版图有多个顶层 Cell，可以一次导入多个，此时把 Cell 指定留空即可。
- Instantiate：把被导入的 Cell 作为一套独立层次实例化进当前 Cell。
- Merge hierarchy：第四种模式稍复杂。它基本类似Merge，但会识别出互相对应的 Cell，并把被导入 Cell 的内容合并进对应的原有 Cell 中。算法判断“对应”的条件是：被导入子 Cell 的平坦实例与对应原有 Cell 的平坦实例完全相等（这里的

平坦实例指该 Cell 在当前 Cell 语境下的实例）。实现上先逐步筛减候选列表，最后用名字相似度来消除歧义。

导入功能会用 `$x` 后缀生成新的 Cell 名，以避免重名。

3.6 net 追踪功能

net tracing（网络追踪）功能通过检测相互接触、共同构成导电区域的 Shape 来追踪一条 net，其特性是可以指定一叠金属（或一般的“导电”）图层，各层之间可选地通过 via 图形相连。追踪算法会沿着 via 图形跨层，形成到其它金属层的连接。导电材料的覆盖范围可以由单个版图图层派生，也可以由若干图层的布尔组合派生，例如把有源区减去 poly 区域，就能选出晶体管的源、漏区。

3.6.1 单网络追踪（Single net tracing）

该算法面向单条 net 的提取，采用增量式提取方式。因此提取一条小的 net 相对较快，而提取电源网之类的大 net 则明显比目前的层次化 LVS 工具慢得多。

net tracing 功能位于 `Tools` 菜单中。界面允许追踪多条 net，结果保存在一个已提取 net 的列表中。若 net 上有 label，会用其推导 net 名；此外还会列出提取过程中经过的 Cell，从而可以看出这条 net 连接了哪些 Cell。

提取 net 之前必须先指定一叠图层（layer stack）。在界面上按 `Layer Stack` 按钮打开 layer stack 对话框，或使用 `Tools` 菜单中的 `Edit Layer Stack`。图层必须以 `layer/datatype` 记法指定，via 的指定是可选的：若不指定 via 图层，则要求两个金属层的 Shape 直接接触才算连接；若指定了 via 图层，则必须有 via 图形才构成连接。

layer stack（也称 connectivity）同样是 technology 的组成部分：给版图选定某个 technology 后，就采用该 technology 专属的叠层。要编辑所有 technology 的叠层，请用 Technology Manager（`Tools` > `Manage Technologies`）中各 technology 的 `Connectivity` 部分。

一个 connectivity 定义可以包含多套叠层，使用时须选定其一。当你的工艺配置中存在多套布线金属方案时，这很有用，不必为每套方案分别建立 technology。若声明了多套叠层，可以在 `Trace Net` 对话框中 `Layer Stack` 右侧的下拉按钮，以及左下角 `Configuration` 按钮处选择其一。

在叠层定义中，KLayout 允许使用 symbolic layer 与布尔表达式，从而可以给图层赋予有意义的名字（如 `POLY`、`VIA1`），也可以使用派生图层（如用 `ACTIVE-POLY` 表示 CMOS 晶体管的源漏区）。相关说明见第原文对应章节节与第原文对应章节节。

定义并选定 layer stack 之后，按 `Trace Net` 按钮并在版图中单击一点，即可追踪该处的 net：算法从该点下的 Shape 出发提取整条 net，并把结果列入对话框左侧的 net 列表。若勾选了 `Lock`，则不必再按 `Trace Net` 按钮，直接单击另一点即可继续追踪另一条 net。

有时会遇到很大的 net（例如电源网），单击之后追踪器会开始运行，提取耗时很长。若并不关心这类 net 的细节，可以限制追踪深度：遇到指定数目的 Shape 后追踪器即停止，并把已收集的 Shape 作为一条不完整的 net 报告出来。

要配置该深度，在追踪对话框底部的 `Trace depth` 框中填入期望的 Shape 数。注意：由于内部还会用到一些标记 Shape（它们计入数量但不作为 net 的一部分输出），net 中实际的 Shape 数可能略少于所设深度。

`Trace Path` 功能类似，但要求指定两个点，让算法找出两点之间的最短连接（按 Shape 数量而非几何长度衡量）。若两点并不连通，会给出提示说明从一点到另一点没有通路。

net 的显示方式可以多方配置：在 trace net 对话框中按 `Configure` 打开配置对话框，除用于显示 net 的标记颜色与样式外，还可以指定窗口是否以及如何调整以适合该 net。

3.6.2 追踪全部网络（Tracing all nets）

该算法借自 LVS 功能，那里还在此基础上扩展出器件识别与网表生成。在 net tracer 的语境下，net 仅由相连的 Shape 组成，不挂接器件。由于 LVS 一次扫描就提取全部 net，这里使用该功能也能一次得到所有 net。虽然信息更丰富，但追踪全部 net 通常比追踪一条大 net（如电源网）更快。LVS 的 net 提取器还支持层次化处理，性能提升显著，net 的表示也更紧凑。

要提取全部 net，使用 `Tools` 菜单中的 `Trace All Nets`，它会立即开始提取，并沿用单网络追踪器中的 connectivity 定义。你可以从单网络追踪器界面、Technology Manager，或 `Tools` 菜单中的 `Edit Layer Stack` 修改这套 layer stack。

追踪完成后会打开网表浏览器（netlist browser）对话框。在其中可以：

- 在中央视图左半边浏览电路层次（取自 Cell 层次）。
- 在视图右半边浏览各电路的 net，单击某条 net 即高亮该 net。
- 用 `Configure` 按钮配置 net 的高亮方式。
- 用右上角的 `File` 菜单按钮，把全部或选中的 net 导出到版图、把网表（含 Shape）存成文件、从文件载回，以及管理网表数据库。
- 用搜索框查找 net 名（若有 label）与电路。
- 用左上角的 `back` 与 `forward` 按钮在浏览历史中前进、后退。

提取出的 net 以 KLayout 专用格式 `L2N`（layout to netlist）读写，其中既含 net 也含构成 net 的 Shape，因此追踪结果可以存盘并日后取回。

Chapter 4

Design Rule Check (DRC)

4.1 DRC 基础

KLayout 从 0.23 版起支持 design rule check（DRC，设计规则检查）。DRC 功能涵盖：

- 基本 DRC 检查，例如最小宽度与最小间距检查。
- 图层生成方法，例如布尔运算与 sizing。
- 扩展的几何检查，例如重叠、包围、内部与外部检查。
- 支持由 Polygon 派生或由其它函数输出的 Edge 对象。Edge 对象适合实现与边相关的运算，例如选择性 sizing。
- 可同时处理多个输入版图。
- 支持 Text 对象图层。用 Text 对象给 Polygon 打标记、给 net 加 label 都很方便。
- Cell 过滤与局部（限定区域的）运算。
- 面向大型版图的 tiling 方式，可配置为使用多核 CPU。
- 层次化选项。

DRC 功能由 DRC 脚本控制。DRC 脚本本质上是运行在 DRC 引擎环境中的一段代码，语言基于 KLayout 内置的 Ruby 解释器，并把底层对象模型包装成一套精炼而富于表达力的语言。脚本是即时执行的，因此可以在其间嵌入条件判断或循环。

DRC 脚本的输出可以送往某个版图图层，或送往 report database；后者在 marker 数据库浏览器中可视化。

4.1.1 基本脚本

runset 的写法在第 4.2 节详细介绍。下面是一个简单的 DRC 脚本示例：

```
report("A simple script") active = input(1, 0) poly = input(6, 0)
    gate = active & poly gate.width(0.3).output("gate_width", "Gate
    width violations") metal1 = input(17, 0) metal1.width(0.7).output
    ("m1_width", "M1 width violations") metal1.space(0.4).output("
    m1_space", "M1 spacing violations")
```

这段脚本先用布尔“AND”从 active 与 poly 两层求出栅极 poly 的区域（active 层为 GDS 第 1 层，poly 层为 GDS 第 6 层），再在这个求得的 gate 层上做 0.3 微米的

最小宽度检查；此外还对第一层金属（GDS 第 17 层）做宽度与间距检查。

下面逐行拆解：

- 报告目标。`report("A simple script")` 指示脚本把输出送往 report database，后者在 marker 数据库浏览器中显示。括号内是 report database 的描述文字。
- 输入图层。`active = input(1, 0)` 创建一个输入图层。所谓“图层”在 DRC 中就是 Shape、Edge 或 Edge pair 的集合（Edge pair 是检查方法产生的一类对象）。`input` 是返回图层对象的函数，各种检查与运算都以面向对象的方式作用于这些图层对象，背后是 Ruby 解释器。变量 `active` 即表示该图层。
- `poly = input(6, 0)` 再创建一个输入图层，表示多晶硅（GDS 第 6 层、datatype 0）。
- 布尔与运算。`gate = active & poly` 求 active 与 poly 的布尔“AND”，运算符 `&` 即交集运算。结果作为一个新图层对象存入变量 `gate`；这是临时图层，不会出现在输出中，但可供后续语句使用。
- 宽度检查并输出。`gate.width(0.3).output(...)` 把两步合成一句：先在 gate 层上以 0.3 微米为最小宽度做宽度检查，再把违规标记输出到指定的报告类别。方法是作用于具体对象的函数，DRC 脚本用 `.` 记法调用。
- `metal1 = input(17, 0)` 和前文一样，从 GDS 第 17 层、datatype 0 取输入并创建图层对象。
- `metal1.width(0.7).output("m1_width", ...)` 与 gate 类似，做宽度检查（这次是 0.7 微米），并把违规标记输出到 report database 的某个类别。
- `metal1.space(0.4).output("m1_space", ...)` 再做一次几何检查：这次是最小间距 0.4 微米的 spacing 检查。

脚本也可以写成若干等价形式。例如下面是极简版的栅极宽度检查：

```
(input(1, 0)*input(6, 0)).width(0.3).output("gate_width", "Gate width
    violations")
```

布尔运算还提供了一些别名，若偏好面向对象写法，可以用 `and` 方法：

```
input(1, 0).and(input(6, 0)).width(0.3).output("gate_width", "Gate
   width violations")
```

也可以用函数式写法：

```
and(input(1, 0), input(6, 0)).width(0.3).output("gate_width", "Gate
   width violations")
```

尺寸可以用不同方式给出：

```
# floating-point (will default to micrometer) gate.width(0.3).output("
```

```
gate_width", "Gate width violations") # floating-point with unit
gate.width(0.3.micron).output("gate_width", "Gate width violations
") gate.width(300.nm).output("gate_width", "Gate width violations
") # integer values will give dimensions in database units! gate.
width(300).output("gate_width", "Gate width violations") # as
variable min_width = 300.nm gate.width(min_width).output("
gate_width", "Gate width violations")
```

4.1.2 安装与运行脚本

要创建 DRC 脚本，选择 Tools > DRC > New DRC Script。KLayout 会打开 Macro 开发 IDE 并新建脚本；第一件事是给它起个合适的名字——光标已停在名字上，直接输入新名字并回车即可。

下一步给 DRC 脚本写点描述：按宏编辑器工具栏上的 Properties 按钮，输入描述文字，这段文字会出现在菜单项中。

此后该 DRC 脚本就会带着描述出现在 Tools > DRC 下，选中该条目即运行脚本。

要编辑 DRC 脚本，从 Tools > DRC 选 Edit DRC Scripts，或进入宏编辑器 IDE 后选择 DRC 类别（宏列表上方的标签）。

DRC 脚本可以像普通宏一样共享或全局安装。可以放进 macros 目录，但更推荐部署在 macros 同级、名为 drc 的目录中。KLayout 会在多处搜索 DRC 脚本或宏，包括安装路径与应用目录（通常在用户主目录）。这两处都可以用来存放 DRC 脚本。

对进阶用户而言，DRC 脚本其实就是绑定到 DRC 解释器（而非普通 Ruby 解释器）上的宏，可以像普通宏一样创建与运行：用 Macros > Macro Development 进入宏 IDE，用 “add macro” 功能（+ 按钮）新建 DRC 脚本，并在选择模板时（General 部分）选 DRC scripts。

宏开发设施的更多说明见第原文对应章节节。

默认情况下 DRC 脚本放在宏 IDE 的 DRC 类别中。宏类别只是组织宏的手段，并不决定运行行为；因此除 DRC 类别所选的 drc 目录外，DRC 脚本也可以放在别处。

DRC 脚本可以像普通宏那样在调试器中执行，用断点或单步来调试。底层的 DRC 命令会映射到 Ruby 函数，因此单步进入这些命令就能看到其背后的实现代码。

DRC 脚本以 KLayout 宏格式存为 .lydrc 文件，这是 XML 文件，脚本代码放在其中。KLayout 也识别后缀为 .drc 的纯文本文件，但这类文件通常缺少必要的元信息，无法绑定到菜单项，因此只能从宏 IDE 中运行。

4.1.3 DRC 豁免流程（DRC waiving）

DRC 豁免是对无法避免的违规做签字放行的过程。通常并不鼓励这种做法，因为存在 DRC 违规就无法保证器件的可制造性；更糟的是，把不干净的版图送去制造可能

带来污染风险，而制造商往往会尽力避免这一点，因此不满足 DRC 的版图通常会被拒收。

不过仍有若干正当理由可以豁免：有时某些 DRC 规则因为器件不使用某项工艺选项而不适用；又或者某条规则只是推荐性的，违反它并不违规。这种情况下，可以自担风险豁免该违规。豁免并非正式流程，通常由制造商自行酌情处理。

KLayout 的 DRC 豁免流程如下：

- 第一步，DRC 运行通过 `report` 命令并指定目标文件，生成一个 RDB 数据库。
- 在 marker 浏览器中审查这份报告，可以添加注释、设置标志、添加截图；也正是此时把你认为确需豁免的违规设为豁免状态：marker 浏览器有 `waive` 按钮，可对选中 marker 设置或取消 waived 标志。
- 完成后，用 marker 浏览器 `File` 菜单中的 `Save As Waiver DB` 把编辑过的数据库另存为 waiver DB。技术上它把 report database 写到第二个文件，文件名为原文件名加后缀 `.w`。
- 再次运行 DRC 时，KLayout 会找到该 waiver DB 文件，并把其中的属性（标志、图像、注释等）应用到当前的 report database 上，其中就包括给对应条目打上 waived 标志。

在豁免环节，KLayout 把属性应用到“值（即 Shape）、类别与 Cell”都相同的条目上。这意味着一旦 Cell 改名，豁免就不再生效，或最坏情况下被套到错误的 Cell 上。因此豁免应放在流程后期、Cell 名不太可能再改动时进行。

4.1.4 把 KLayout 当作独立的 DRC 引擎

配合一种特殊的 DRC 脚本，KLayout 可以作为非交互的 DRC 引擎使用。由于独立引擎模式下没有“当前版图”，这类脚本必须用 `source` 函数显式指定输入；输出同样要显式指定，或用 `target` 生成输出版图，或用 `report` 生成输出 report database（详见上文）。

下面是个例子：从 `input.gds` 读取第 1 层、datatype 0，做 200 纳米的 sizing，并把结果写入 `out.gds` 的第 10 层、datatype 0：

```
source("input.gds") target("out.gds") input(1, 0).sized(200.nm).
    output(10, 0)
```

再看一例，把结果存到 report database 文件 `out.lyrdb` 的 `sized` 类别中：

```
source("input.gds") report("Output Database", "out.lyrdb") input(1,
    0).sized(200.nm).output("sized")
```

要运行这类脚本，把脚本存成 `.drc` 后缀的文件，然后如下运行（把 `my.drc` 换成你的文件名）。建议关闭一些此处用不到的功能，并用 `-b` 让 KLayout 进入非交互批处

理模式：

```
klayout -b -r my.drc
```

`-b`会禁用全部界面功能，让 KLayout 进入引擎模式，在 Unix 上不需要显示连接；同时不再从各搜索位置隐式加载宏（仍可用`-rm`显式加载），也不再读写配置文件，从而减少 I/O 并避免多个 KLayout 实例之间的写冲突。

4.2 DRC runset 编写

本文详细介绍如何编写 DRC runset。DRC 函数的完整参考见第原文对应章节节。

4.2.1 runset 基础

runset 本质上是一段运行在 DRC runset 解释器环境中的 Ruby 脚本。在这一层面上，runset 只用很少的几个类，另有若干全局函数：

- Layers（`DRC::DRCLayer`类）：图层或取自原版图输入，或由生成信息的函数创建；可用于各项检查和派生运算。
- Sources（`DRC::DRCSource`类）：source 表示“从哪个版图对象取输入”。总存在一个默认 source，即默认版图。

还有一些全局函数，不需要任何对象即可调用。

runset 的基本要素是输入与输出规格：输入通过`input`方法给出，它会创建一个图层对象。

总体而言，runset 语言提供了丰富的同义写法——同一个结果常有多条路径可达。

脚本以即时模式执行：每个函数立即生效，结果可以立即用于条件判断或循环。

由于 runset 是运行在 KLayout 脚本引擎中的 Ruby 脚本，可以直接使用 KLayout 完整的数据库访问层。

4.2.2 包含其它文件

DRC 脚本语言基于 Ruby，因而具备许多原生特性。原则上脚本里可以用`load`引入另一份脚本，但 Ruby 的这个机制有其局限。

可惜`load`会为变量创建局部上下文，因此不能用`load`读入一个“定义若干变量”的文件来共享变量。

为解决这个问题，KLayout 提供了一个专用扩展：通过预处理把另一份文件嵌入当前源码，效果上等同把被包含文件的内容原样插入。

记法如下：

```
# %include to_include.drc
```

这会包含 `include.drc`。若未给绝对路径，则相对当前文件所在目录查找。

文件名也可以加引号，并支持表达式插值（语法见第原文对应章节节），因此可以读取环境变量来定位文件。

```
# %include $(env("HOME"))/to_include.drc
```

由于 Ruby 看不到原始文件，某些内部机制（例如内省）报告的会是错误的文件名与行号；多数情况下这无关紧要。

4.2.3 输入与输出

输入通过 `input` 方法或全局函数给出。`input` 本质上是 source 对象的方法，而总有一个默认 source，即当前版图。

`input` 会创建一个图层对象，代表指定输入图层上的 Shape。指定图层有多种方式。

```
# GDS layer 17, datatype 0 l = input(17) # GDS layer 17, datatype 10
   l = input(17, 10) # By expression (here: GDS layers 1-10, datatype
    0 plus layer 21, datatype 10) # All shapes are combined into one
   layer l = input("1-10/0", "21/10")
```

输入也可以取自默认版图之外的其它版图，做法是用全局函数 `layout` 创建 source 对象：

```
# layer 17 from second layout loaded l = layout("@2").input(17) #
   layer 100, datatype 1 and 2 from "other_layout.gds" other_layout =
    layout("other_layout.gds") l1 = other_layout.input(100, 1) l2 =
   other_layout.input(100, 2)
```

输出默认送往默认版图，即当前视图中最先加载的那个。输出规格包括图层与 datatype。

```
# send output to the default layout: layer 17, datatype 0 l.output
   (17, 0) # send output to the default layout: layer named "OUT" l.
   output("OUT") # send output to the default layout: layer 17,
   datatype 0, named "OUT" l.output(17, 0, "OUT")
```

输出可以用 `target` 函数送往其它版图：

```
# send output to the second layout loaded: target("@2") # send output
    to "out.gds", cell "OUT_TOP" target("out.gds", "OUT_TOP")
```

输出也可以用 `report` 函数送往 report database：

```
# send output to a report database with description "Output database"
    # - after the runset has finished this database will be shown
```

```
report("Output database") # send output to a report database saved
 to "drc.lyrdb" report("Output database", "drc.lyrdb")
```

把输出送往 report database 时，规格中必须给出正式名称，描述则是可选的。`output`方法会创建……

```
# specify report database for output report("The output database")
   ... # Send data from layer l to new category "check1" l.output("
   check1", "The first check")
```

`report`与`target`规格必须写在实际的 output 语句之前；可以给出多个 report 与 target 规格，把输出分流到多处。

用`output`写出的图层可以再作为输入使用，但要注意 input 语句应放在 output 语句之后。

也可以再打开若干“旁路”report 与 target，把图层送往这些输出，同时不打断默认输出。

要打开“旁路 report”，像使用`report`那样使用`new_report`：它不会切换输出，而是返回一个新的 report 对象。

```
# opens a new side report side_report = new_report("Another report")
   ... # Send data from layer l to new category "check1" to the side
   report l.output(side_report, "check1", "The first check")
```

同理，可用`new_target`打开“旁路 target”，把某些图层写到其它版图文件，这在需要输出多份数据时很方便。

```
# opens a new side target for debugging debug_out = new_target("debug
   .gds") ... # Send data from layer l to the debug output, layer
   100/0 l.output(debug_out, 100, 0)
```

4.2.4 尺寸规格

尺寸规格在很多地方用到：坐标、间距与宽度、长度等。所有地方都遵循以下规则：

- 浮点数默认按微米解释。
- 整数默认按数据库单位解释（注意：不是整数微米!）。
- 为明确起见，可以给数值加上单位。

单位通过单位方法追加：

`0.1`：0.1 微米

`200`：200 个数据库单位

`200.dbu`：200 个数据库单位

`200.nm`：200 纳米

`2.um`或 `2.micron`：2 微米

`0.2.mm`：0.2 毫米

`1e-5.m`：1e-5 米（即 10 微米）

面积的单位通常是平方微米。同样可以加单位，以明确采用某种面积单位。

`0.1.um2`或 `0.1.micron2`：0.1 平方微米

`0.1.mm2`：0.1 平方毫米

角度一律以度为单位；加上 degree 单位可以更明确地表示你要用度。

`45.degree`：45 度

4.2.5 对象与方法

runset 是用面向对象语言写成的脚本，完全可以不按对象方式书写。

在 KLayout 的 DRC 语言中，图层是一个带有若干方法的对象，布尔运算即通过这些方法实现。

```
input(1, 0).sized(0.1).output(100, 0)
```

与其它方法一样，`size`有两种形式：原地（in-place）版本与异地（out-of-place）版本。

下面是上面那段代码用原地版本写成的样子：

```
layer = input(1, 0) layer.size(0.1) layout.output(100, 0)
```

原地版本在内存上略微经济，因为异地版本每步运算都要新建一个图层对象、保留一份中间结果。

```
layer = input(1, 0) layer2 = layer layer.size(0.0) layer.output(100,
    0) layer2.output(101, 0)
```

这段代码会给 100 层与 101 层产生相同的 sizing 结果，因为 `layer2 = layer1`只是共享同一个对象。

该问题可以通过使用异地版本，或用 `dup`创建深拷贝来解决。

```
# out-of-place size: layer = input(1, 0) layer2 = layer layer = layer
    .sized(0.0) layer.output(100, 0) layer2.output(101, 0) # deep copy
     before size: layer = input(1, 0) layer2 = layer.dup layer.size
    (0.0) layer.output(100, 0) layer2.output(101, 0)
```

有些方法还提供包括函数式调用在内的不同写法，例如 width 检查既可以写成 `layer.width(0.3)`，也可以写成 `width(layer, 0.3)`。

```
# method style: layer.width(0.2).output("width violations") #
    function style: w = width(layer, 0.2) output(w, "width violations
    ")
```

函数式写法是为不熟悉面向对象风格、更习惯函数调用的用户准备的。

下面简要列出部分可用方法：

源、输入与输出：`source`、`layout`、`cell`、`select`、`clip`、`input`、`labels`、`polygons`、`output`、`report`、`target`

DRC 函数：`width`、`space`、`separation`（`sep`）、`notch`、`isolated`（`iso`）、`enclosing`（`enc`）、`enclosed`、`overlap`

通用 DRC（见下文）：`drc`

布尔运算：`and`（与）、`or`（或）、`not`（非）、`xor`（异或）、`join`（并）

sizing：`size`、`sized`

合并：`merge`、`merged`

图形选择：`covering`、`in`、`inside`、`interacting`、`outside`、`overlapping`、`pull_inside`、`pull_interacting`

过滤器：`rectangles`、`rectilinear`、`with_area`、`with_bbox_height`、`with_bbox_width`、`with_bbox_max`、`with_bbox_min` 等

文字过滤：`texts`、`texts_not`

变换：`moved`、`rotated`、`scaled`、`transformed`；这些方法同样有原地版本

Polygon 处理：`extents`、`hulls`、`holes`

Edge 处理：`centers`、`end_segments`、`start_segments`、`extended`、`extended_in`、`extended_out`

信息查询：`length`、`perimeter`、`area`、`polygons?`、`edges?`、`edge_pairs?`、`is_box?`、`is_clean?`、`is_empty?`、`is_merged`

图层模式：`raw`、`clean`

图层类型转换：`edges`、`first_edges`、`second_edges`、`polygons`

4.2.6 Polygon 层与 edge 层

KLayout 认识四种图层类型：polygon、edge、edge pair 与 text 层。其中 polygon 与 edge 层是最基础的两类，其余两类都由它们派生。

polygon 层由原始图层经 `input` 或 `polygons` 创建；`input` 还会把很小的文字当作极小的图形读入，而 `polygons` 只取图形。

polygon 层描述有面积的物体（绘图视图中的“填充图形”），可以是 Polygon，也可以是 Box 或 Path 这类有面积的图形，运算时统一按 Polygon 处理。

polygon 分 raw 与 merged 两种：merged polygon 由一条外壳轮廓加零到多条内部孔洞轮廓组成。

edge 层可由 polygon 层派生，用来描述一个 Polygon 的各个单独的边（“侧边”）。

edge 可以按长度与角度过滤；`extended` 可在边上立起 Polygon（通常是矩形）。

edge 不区分其来自 Polygon 的孔洞还是外壳；edge 的方向则是沿轮廓行进方向确定的，因此判断“里外”时需留意边界的绕行方向。

merged edge 是合并后的结果：共线的边合并成一条，退化的边（单点）被去掉。

用 `edges` 方法可以把 Polygon 分解成 edge；另一种生成 edge 的方式是从……取边。

4.2.7 文字集合

从 0.27 版起，KLayout 支持 text 层。Text 本质上就是带字符串的位置。

Text 可以用来选取 Polygon，或在 net 提取中充当 net 名。

文字集合保存在 text 层中，用 `labels` 方法（而非 `input`）创建。

text 层支持以下运算：

- 与 polygon 层做布尔 AND：选出位于某个 Polygon 之内或边界上的文字；`interact` 则选出与 Polygon 接触或重叠的文字。
- 与 polygon 层做布尔 NOT：选出不在任何 Polygon 之内的文字，`not_interact` 用于选出不与任何 Polygon 接触的文字。
- 作为 `interact` 的第二个图层：这样可以选出被特定文字标记的 Polygon。
- 按字符串过滤：可以按固定文字或 glob 通配符过滤文字内容。
- `flatten` 会打平 text 层的层次结构。
- 围绕文字所在位置生成 Polygon 或 edge：`polygons` 与 `edges`

4.2.8 edge pair 与 edge pair 集合

edge pair 是由两条边组成的对象。描述 DRC 违规时它很方便，因为违规常见的形态就是“两条边相距过近”或“两条边之间的间距不足”。

edge pair 集合的行为类似普通图层，但为其定义的方法很少。edge pair 可以被分解为组成它的两条边，或直接转成 Polygon 标记。

用 `polygons` 可把 edge pair 转成 Polygon。edge pair 的面积可能为零（例如两条边共线时）。

4.2.9 raw 与 clean 图层模式

KLayout 的 DRC 引擎有两种解释图层几何信息的基本方式：clean 模式下，参与运算的 Polygon 会先被合并，重叠部分只保留一份；

clean 是默认模式：无论从输入取得还是新生成的图层都按 clean 模式处理。要切换到 raw 模式，用 `raw` 方法。

在 clean 模式下，多数函数会隐式合并 Polygon 与 edge，文档中称为“implicit merge”。

一个应用例子：在 sizing 之后检测重叠区域：

```
overlaps = layer.size(0.2).raw.merged(2)
```

该语句的效果如下：

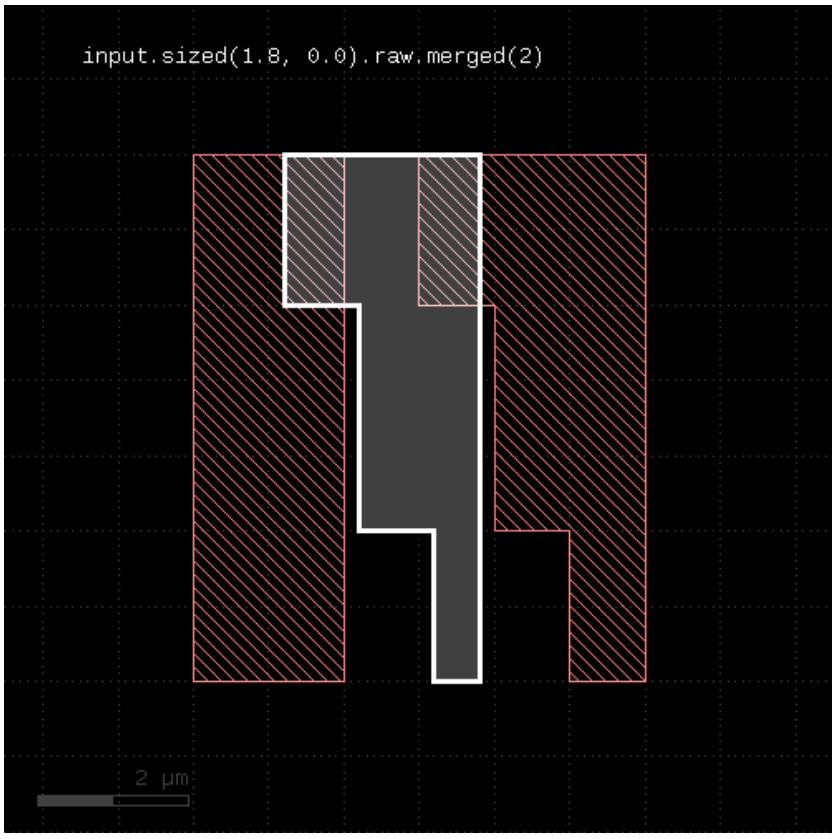


图 4.1

`merged` 参数为 2 时，输出的是被三个及以上 Polygon 重叠的区域。

请注意，`raw` 与 `clean` 会改动图层状态，所以要当心下面的陷阱：

```
layer = input(1, 0) layer.raw.sized(0.1).output(100, 0) # this check
    will now be done on a raw layer, since the # previous raw call was
     putting the layer into raw mode layer.width(0.2).ouput(101, 0)
```

下面两张图展示 raw 与 clean 模式的效果：

4.2.10 屏蔽

“屏蔽”（shielding）是指 DRC 测量不会“穿透”版图特征去看。启用屏蔽后，DRC 测量间距时会把中间的遮挡图形考虑在内，只统计未被遮挡的视线。

下面的例子演示屏蔽的效果：右侧例子关闭了屏蔽，因此两条边即便中间隔着别的图形，仍会按直线距离判定间距。

屏蔽虽然更自然，却可能带来不利影响：它在零距离处就已生效，因此紧贴在一起的图形会被判为不存在间距，从而产生与直觉不符的结果。

4.2.11 通用 DRC

从 0.27 版起，DRC 语言新增了“通用 DRC”（universal DRC）功能。

一方面，它让 DRC 检查的书写更方便，因为可以用自然的表达式来描述；

```
... drc_w = input(1, 0).width(0.2) ...
```

可以写成：

```
... drc_w = input(1, 0).drc(width < 0.2) ...
```

`drc`方法即“通用 DRC”方法，它接受一个运算符。简单情形下运算符就是普通的比较。

`drc( measurement != value )`：在尺寸不等于给定值处生成标记

`drc( measurement == value )`：在尺寸等于给定值处生成标记

`drc( measurement less_op value )`（`less_op`为 < 或 <=）：在尺寸小于或等于给定值处生成标记

`drc( measurement greater_op value )`（`greater_op`为 > 或 >=）：在尺寸大于或等于给定值处生成标记

`drc( lower_value less_op measurement greater_op upper_value )`：在尺寸落在上下界之间处生成标记

`measurement`可以是`width`、`notch`、`isolated`（`iso`）、`separation`（`sep`）、`overlap`、`enclosed`或`enclosing`……

```
... l1 = input(1, 0) l2 = input(2, 0) drc_sep = l1.drc(separation(l2)
    <= 0.5) ...
```

选项同样与 measurement 一起给出，记法与普通方法一致。

```
... drc_w = input(1, 0).drc(width(projection) < 0.2) ...
```

但通用 DRC 远不止是书写检查的便捷方式：它提供了一系列选项，允许在单个测量内部对图形做局部运算与过滤。

例如布尔 `and` 运算符会在这一循环内部做“局部”的 AND，从而可以高效地表达“同时满足 space 与 width 条件”的复杂检查。

```
... drc_ws = input(1, 0).drc((width < 0.2) & (space < 0.3)) ...
```

该 AND 在主图形的边上计算，返回同时满足 space 与 width 条件的那些边段，而不必把整层图形先合并再判断。

```
... drc_ws1 = input(1, 0).width(0.2).edges drc_ws2 = input(1, 0).
    space(0.3).edges drc_ws = drc_ws1 & drc_ws2 ...
```

原因是：局部循环中的布尔运算在某个输入为空时可以短路跳过。

总体而言，通用 DRC 功能丰富，支持基于 Polygon 或边属性的过滤，也支持多条件组合，适合表达较复杂的规则。

4.2.12 日志与输出详细程度

runset 执行过程中会写一份日志，列出所调用的方法及其执行时间；日志便于定位耗时最长的步骤。

日志显示在日志窗口中；若日志窗口未打开，在类 Linux 系统上则输出到终端。

`log` 函数适合在处理过程中打印结果计数：

```
... drc_w = input(1, 0).width(0.2) log("Number of width violations:
    #{drc_w.data.size}") ...
```

`error` 函数可以无条件输出错误消息，并按错误格式呈现；日志中可以据此快速找出脚本自身的问题。

```
log_file("drc_log.txt") verbose(true) info("This message will be sent
    to the log file") ...
```

`profile` 函数会在 DRC 运行期间收集性能剖析信息，脚本结束时把各函数的累计耗时打印出来。

```
# enables profiling profile ...
```

4.2.13 tiling 选项

tiling 是降低某次运算内存需求的方法。对大型版图，把整层拉进来会占用大量内存，拆成小块逐块处理则可以把峰值压下来。

tiling 并非没有代价：有些运算的影响范围可能是无限的，例如选取边之后沿着边一直延伸下去，这类运算就无法简单地分块。

所幸许多运算的范围有限，因而可以有效地应用 tiling。

对边长度示例这类非局部运算，某些情况下可以推算出有限的范围，例如边的最大长度已知时，扩展范围就有上界，从而仍可分块。

用 `tiles` 函数启用 tiling；`threads` 函数指定 tiling 时使用的 CPU 核数。

```
# Use a tile size of 1mm tiles(1.mm) # Use 4 CPU cores threads(4) ...
    tiled operations ... # Disable tiling flat ... non-tiled
    operations ...
```

某些运算会隐式指定 tile 边界。若边界已知（见上面边长度一例），显式指定边界可以避免为每个 tile 预留过大的重叠区域。

```
# Use a tile border of 10 micron: tile_borders(10.um) ... tile
    operations with a 10 micron border ... # Disable the border
    no_borders
```

关于 tile 尺寸：通常毫米量级就足够；太小的 tile 会让边界重叠与合并的开销超过收益。

4.2.14 层次模式

tiling 之外还有层次模式。在层次模式下，DRC 函数在局部 Cell 环境中运算，把重复的单元内容只算一次。

要启用层次化运算，使用 `deep` 语句：

```
report("deep 2") # enable deep (hierarchical) operations deep poly =
    input(3) spc = poly.space(0.5) spc.output("poly space >0.5")
```

`deep` 与 tiling 不兼容：`tiles` 会关闭 `deep`，反之亦然。要关闭 deep 模式，用 `flat`。

deep 处理是图层属性。指定 `deep` 之后，由 `input` 派生的图层即被声明为 deep 图层，后续运算会在层次上保持这一性质。

在布尔运算这类二元运算中，若参与的两个图层都是 deep 的，运算就按层次进行。

要判断某图层是否为 deep，用 `is_deep?`。

```
report("deep 2") # enable deep (hierarchical) operations deep poly =
    input(3) puts poly.is_deep? # -> true poly.flatten puts poly.
    is_deep? # -> false
```

多数运算都支持层次化，只有少数例外，特别是变换类运算与需要全局信息的运算，会在执行前自动转为平坦处理。

当版图因 Cell variant 而需要回写时，会出现新的 Cell 版本。

把层次化运算的结果送往 report database 时，标记会列在检测到它的那个 Cell 之

下，而不是全部堆到顶层。

4.2.14.1 限制

需要合并 Polygon 的函数会利用 net 聚类算法来形成合并后的 Polygon。

除上述 Cell variant 情形之外，DRC 的层次模式不会改动 Cell 层次，因此版图与 schematic 的层次对应关系得以保留。

4.2.15 DRC 与用户属性

DRC 特性对用户属性（user properties）有一定支持。用户属性是挂在图形上的键值对集合。

对 DRC 而言，挂在一个图形上的属性集合被视为整体；DRC 可以在多个方面作用于这些属性：

忽略属性：这是默认模式

使用属性：把检查与某些布尔运算配置成只考虑具有特定属性的图形

映射/过滤：改动属性的键或集合，或把它们去掉

传递属性：经由 `size` 或 `with_...` 之类的选择器传递

此外 DRC 函数也能生成属性，目前只有 `nets` 方法会给图形附上 net 身份属性，用以标明该图形属于哪条 net。

截至本文撰写时，用户属性支持仍有些实验性，但潜力很大。

目前，以下运算可以按“属性相同或不同”来限定作用对象：

Polygon 布尔运算：`and`、`not`、`andnot`

polygon 模式下的 DRC 检查，例如 `separation`、`space`、`isolated` 等

许多运算会传递属性，例如 edge pair 转 Polygon、edge pair 转 edge、Polygon 转 edge，以及多边形的布尔运算与 sizing。

属性的生成目前支持：

从输入版图读取属性

用 `nets` 功能生成 net 身份属性

属性的修改只提供非常基础的支持：可以整层或按条件移除属性，也可以按映射表改写属性的键，但目前还不能做更复杂的计算。

总体而言，一旦图层带上属性，属性不同的图形就被视为互不作用。

4.2.15.1 读取用户属性

默认情况下用户属性不会被读入图形容器，需要显式开启：

```
l1 = input(1, 0, enable_props)
```

此时可以从属性集合中挑选特定的键，例如只选键为……的值。

```
l1 = input(1, 0, select_props(17, 18))
```

也可以挑选并在键之间映射，像这样：

```
l1 = input(1, 0, map_props({ 17 => 1, 18 => 18 }))
```

这会把键为 17 的值映射到键 1，同时按原键读取键 18 的值；其它键的值则被丢弃，不进入结果图层。

4.2.15.2 修改属性

图层带上属性之后，也可以把它们去掉：

```
layer_without_properties = layer.remove_props
```

同样可以施加 `select_props` 或 `map_props`，按特定键过滤或映射键。

```
reduced_layer = layer.select_props(17, 18) reduced_layer = layer.
   map_props({ 17 => 1, 18 => 18 })
```

4.2.15.3 以 net 标识生成属性

最主要的应用是用 `nets` 方法生成 net 身份属性：

```
connect(metal1, via1) connect(via1, metal2) metal1_nets = metal1.nets
```

默认会在属性键 0 上生成唯一的 net 标识（circuit 名与 net 名组成的元组）。可以用参数指定改用其它键。

```
metal1_nets = metal1.nets(prop(17))
```

`nets` 函数有若干选项，例如可以按名字或 circuit+ 名字过滤特定 net，也可以指定是否把未命名的 net 一并纳入。

```
metal1_vdd_net = metal1.nets(prop(nil), "VDD")
```

4.2.15.4 在检查与布尔运算中使用属性

属性的主要用途是在运算中使用。要把布尔运算限定在属性不同的图形之间，可以给运算加上属性条件。

```
connect(metal1, via1) connect(via1, metal2) metal1_nets = metal1.nets
    metal2_nets = metal2.nets metal1_over_metal2_connected =
   metal1_nets.and(metal2_nets, props_eq)
   metal1_over_metal2_unconnected = metal1_nets.and(metal2_nets,
   props_ne)
```

也可以用 `props_copy` 让该运算在输出上保留原属性：

```
result_with_props = metal1_nets.and(metal2_nets, props_eq +
   props_copy)
```

同样，属性也能参与检查：

```
connect(metal1, via1) connect(via1, metal2) metal1_nets = metal1.nets
    metal2_nets = metal2.nets metal1_space_connected = metal1_nets.
   space(0.4.um, props_eq) metal1_space_unconnected = metal1_nets.
   space(1.um, props_ne)
```

`props_eq`、`props_ne` 与 `props_copy` 在通用 DRC 函数（`drc`）上同样可用，这开启了用通用写法做属性相关检查的可能。

```
connect(metal1, via1) connect(via1, metal2) metal1_nets = metal1.nets
    metal2_nets = metal2.nets critical_area = l1_nets.drc(primary.
   sized(0.2.um) & foreign.sized(0.2.um), props_ne)
```

Chapter 5

Layout vs. Schematic（LVS）

5.1 LVS 概览

5.1.1 LVS 脚本的基本用法

从 0.26 版起，KLayout 把 LVS 作为内置功能支持。LVS 是版图验证中的一个重要步骤：它确保画出来的电路与设计意图中的 schematic 一致。

基本做法很简单：先分析输入的版图，从中提取出 netlist；再把这份 netlist 与参考 netlist（schematic）比对。若两者等价，电路就很可能按预想的方式工作。

除版图之外，LVS 脚本还需要一份 schematic netlist。目前 KLayout 可以读取 SPICE 格式的 netlist，读取器也支持一定程度的配置，因此可望消化相当一部分实用的 SPICE netlist。

基本思路虽简单，细节却相当复杂。本文档旨在说明 KLayout 为实现 LVS 所提供的方案，以及这一过程带来的各种约束。

KLayout 的 LVS 与 DRC 脚本一样集成在 Macro Development IDE 中；事实上 LVS 是 DRC 脚本的扩展，DRC 的全部功能在 LVS 脚本中同样可用。netlist 提取在 DRC 框架内完成，只是该框架被赋予了识别器件与连接关系的能力。

LVS 脚本在宏编辑器 IDE 中创建、编辑与调试，统一在 `LVS` 标签页中管理。IDE 的更多细节见第原文对应章节节；DRC 脚本的入门见第 4.1 节。

LVS 脚本的 XML 形式用 `.lylvs` 后缀（对应 DRC 的 `.lydrc`），纯文本形式用 `.lvs`（同 `.drc`）。与 DRC 脚本一样，LVS 脚本也能在批处理模式下独立执行，参见第 4.1 节中“把 KLayout 当作独立的 DRC 引擎”一节。

5.1.2 KLayout 的 LVS 实现

KLayout 内部的 LVS 实现在连接关系、器件识别与输入输出通道方面都设计得高度灵活，要点如下：

- 尽量少做假设：KLayout 力图少设前提——它不要求 label（虽然 label 有帮助），也不要求特定的层次结构、Cell 名或几何形状。netlist 提取完全由版图中的 Polygon 得出，label 与 Cell 层次只是提供额外信息。
- 层次化分析：KLayout 具备层次化版图处理引擎来支持层次化 LVS。所谓层次化处理，是指布尔运算尽可能在局部 Cell 环境中完成；相应地，器件也在其所属的版

图 Cell 内被识别。

- 层次稳定：KLayout 不会改动版图的层次结构，也不会引入 variant——至少对布尔运算及部分其它运算是如此。这样即便做过层次化 DRC 运算，版图与 schematic 的层次之间仍能保持对应。只有少数分析会引入 variant。
- 灵活的引擎：netlist 生成引擎在器件识别与连接提取方面高度灵活。首先，几乎所有 DRC 功能都能用来推导用于器件构造与连接提取的中间图层；其次，器件识别可以用脚本定制。
- 灵活的 I/O：netlist 是 KLayout 的对象树，其组成（net、device、circuit、subcircuit 等）都完整映射为脚本对象（主类见 API 文档）。因此 netlist 可以在 LVS 脚本内或其它场合被分析与操纵。
- 界面集成：KLayout 提供 netlist 提取结果与 LVS 报告（cross reference、错误）的浏览器。

5.1.3 术语

KLayout 使用一套专门的术语，说明如下：

- Circuit（电路）：由器件、pin 与 subcircuit 连接而成的图。图的节点是连接至少两个元素的 net。若由版图导出，一个 circuit 对应某个特定的版图 Cell。
- Abstract circuit（抽象电路）：清空了内部结构的 circuit。这类 circuit 没有 net，只定义 pin，本质上是“黑盒”，LVS 需把其内部结构视为无关（don't care）。抽象电路适合用来表示那些不在版图中出现、但存在于 schematic 中的子单元。
- Pin（引脚）：circuit 与外界连接的点。circuit 可以以 subcircuit 的形式嵌入别的 circuit；连到这些 subcircuit 的 pin 上的 net 会传播进该 subcircuit，并在那里连接更多元素。pin 通常挂在一个 net 上，某些情况下可以挂多个。
- Subcircuit（子电路）：嵌入另一个 circuit 中的 circuit。同一个 circuit 可以被多次使用，因此多个 subcircuit 可以引用同一个 circuit。若由版图导出，一个 subcircuit 对应某个特定的 Cell Instance。
- Device（器件）：描述原子功能单元的 n 端实体，可以是有源器件（晶体管），也可以是无源器件（电阻、电容）。
- Device class（器件类）：某一类型的器件。器件类有“种”（kind）之分，每种之下可以有多个类。例如 MOS 晶体管的种是 `MOS4`（四端 MOS 晶体管），而在 CMOS 工艺中通常至少有 `NMOS` 与 `PMOS` 两个类。
- Device extraction（器件提取）：检测器件并在导电区域与器件本体之间建立连接的过程，这些连接最终构成器件的 terminal。
- Device combination（器件合并）：把同类多个器件组合成单个器件的过程。例如串联电阻可以并成一个；更重要的是，并行的 MOS 晶体管（“指状”晶体管）会

合并成单个器件。

- Terminal（端子）：器件的 pin。terminal 通常按功能命名，例如 MOS 晶体管的栅极叫 G。
- Connectivity（连接关系）：对工艺叠层中导电区域的描述。图层之间存在层内与层间连接：层内连接指同层上相互接触的 Polygon 构成一个连通（即导电）区域；层间连接则由 via 之类建立。
- Netlist（网表）：由 circuit 与 subcircuit 构成的层次结构。netlist 通常有一个顶层 circuit，其它 circuit 通过 subcircuit 从它被调用。
- Extracted netlist（提取网表）：由版图导出的 netlist。有时该词指带寄生元件（由连线几何导出的电阻、电容）的网表；在 KLayout 的 LVS 语境中，它指纯粹的连通性 netlist。
- Schematic（原理图）：作为 LVS 参考的 netlist。“schematic” 指的是设计者画出的 netlist，它由物理实现过程转成版图；LVS 则把版图转回 extracted netlist 并与 schematic 比对。
- Annotated layout 与 Net geometry（标注版图与 net 几何）：把属于各 net 的 Polygon 归集起来。circuit 中的每条 net 都由一组 Polygon 表示，对应原有连线几何与器件 terminal 的几何。由于 net 可以经 pin 传播到 subcircuit，net 及其几何标注也会随之传播。
- Layout to netlist database（L2N DB）：把 extracted netlist 的信息与标注版图合在一起的数据结构。L2N 数据库可用于可视化 net、从已知位置探测 net，以及做其它分析与处理；有相应的 API 供访问。
- Cross reference（交叉引用）：LVS netlist 比对（pairing）中两个 netlist 之间相互匹配的对象列表，同时也列出不匹配项与 “不精确配对”。不精确配对指并不完全一致、但仍很可能应视为同一对象的一对。
- LVS database（LVS 数据库）：L2N 数据库、schematic netlist 与 cross reference 的组合，是 LVS 结果的完整映像；有相应 API 供访问其中元素。
- Labels（标签）：画在版图中的 Text 对象，用来在特定图层的特定位置打上文字。label 最典型的用途是给 net 命名：若 label 参与连接关系，由它形成的 net 就会以该文字作为 net 名。

5.2 LVS 入门

5.2.1 LVS 入门

为介绍 LVS 功能，我们取最简单的 CMOS 结构：两管反相器。

5.2.1.1 版图

该反相器由两个 MOS 晶体管构成。单个晶体管由一块有源区（ACTIVE 层上的矩形）和横跨其上的栅极（POLY 层）组成，栅极在源、漏区（栅极左右两侧）之间形成沟道；CONTACT 提供与上层金属的连接。

实际版图做成标准单元的形式，多个标准单元可以沿水平方向排成一行。电源轨做在第二层金属上：上方为 VDD，下方为 VSS。n-well 延伸到单元顶部，并应与相邻单元的 well 区域相连：

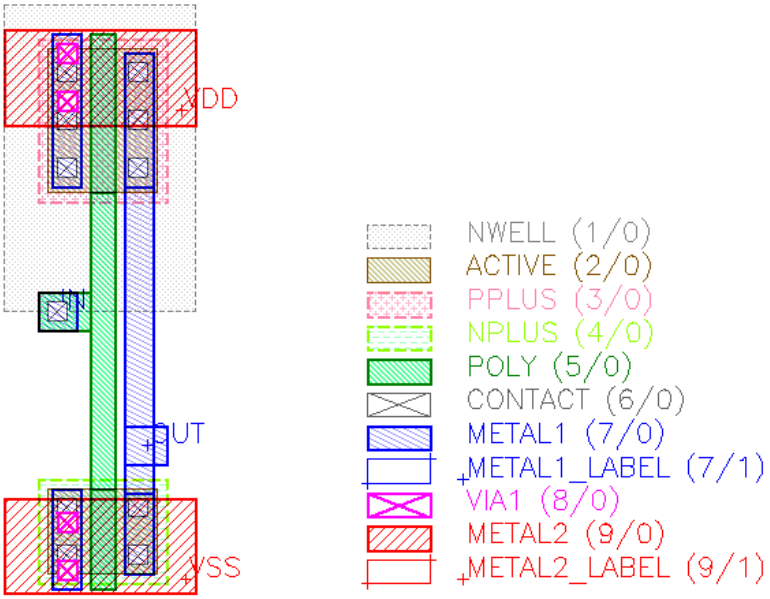


图 5.1: 两管反相器版图

5.2.1.2 schematic

可以为这个反相器画一个简化 schematic（左图），也可以画一个更接近实际、包含晶体管体电位（bulk）的 schematic（右图）。让晶体管体区保持在确定电位以避免闩锁效应很重要，因此这些 terminal 也需要各自的 pin。

做 LVS 首先需要一份参考 schematic，即与带 bulk 连接的 schematic 相对应的 SPICE netlist：

```
* Simple CMOS inverer circuit (inv.cir) .SUBCKT INVERTER VSS IN OUT
   NWELL SUBSTRATE VDD Mp VDD IN OUT NWELL PMOS W=1.5U L=0.25U Mn OUT
    IN VSS SUBSTRATE NMOS W=0.9U L=0.25U .ENDS
```

我们要分析的电路是一个被嵌入更大电路中的 Cell，因此把反相器描述成 subcircuit 比较合适。若 netlist 中只有 subcircuit，KLayout 会分析该 circuit；否则会把全局定义视为主电路。

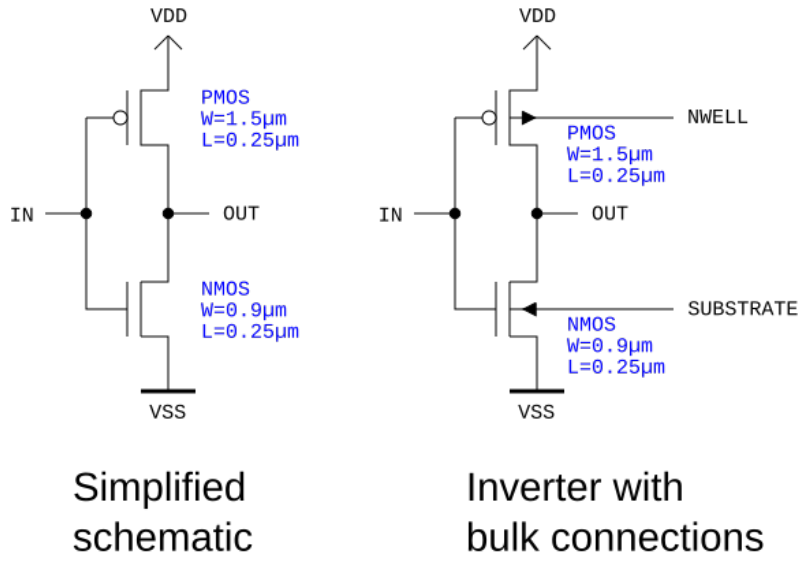


图 5.2: 反相器 schematic：简化版（左）与含 bulk 连接的版本（右）

5.2.2 LVS 脚本示例

用于比对上述版图与 schematic 的 LVS 脚本如下（细节见第 5.2 节）：

```
# LVS script (demo technology, KLayout manual) # Preamble: deep #
   Reports generated: report_lvs # LVS report window # Drawing layers
   : nwell = input(1, 0) active = input(2, 0) pplus = input(3, 0)
   nplus = input(4, 0) poly = input(5, 0) contact = input(6, 0)
   metal1 = input(7, 0) metal1_lbl = labels(7, 1) via1 = input(8, 0)
   metal2 = input(9, 0) metal2_lbl = labels(9, 1) # Bulk layer for
   terminal provisioning: bulk = polygon_layer # Computed layers:
   active_in_nwell = active & nwell pactive = active_in_nwell & pplus
    pgate = pactive & poly psd = pactive - pgate active_outside_nwell
    = active - nwell nactive = active_outside_nwell & nplus ngate =
   nactive & poly nsd = nactive - ngate # Device extraction # PMOS
   transistor device extraction extract_devices(mos4("PMOS"), { "SD"
   => psd, "G" => pgate, "W" => nwell, "tS" => psd, "tD" => psd, "tG"
    => poly, "tW" => nwell }) # NMOS transistor device extraction
   extract_devices(mos4("NMOS"), { "SD" => nsd, "G" => ngate, "W" =>
   bulk, "tS" => nsd, "tD" => nsd, "tG" => poly, "tW" => bulk }) #
   Define connectivity for netlist extraction # Inter-layer connect(
   psd, contact) connect(nsd, contact) connect(poly, contact) connect
   (contact, metal1) connect(metal1, metal1_lbl) # attaches labels
   connect(metal1, via1) connect(via1, metal2) connect(metal2,
   metal2_lbl) # attaches labels # Global connect_global(bulk, "
```

```
    SUBSTRATE") connect_global(nwell, "NWELL") # Compare section
    schematic("inv.cir") align # flattens unpaired circuits netlist.
    simplify # removes floating nets, combines devices compare
```

想试这段脚本，可以加载 KLayout 源码中的反相器版图 `testdata/lvs/inv.oas`，打开宏编辑器 IDE（`Tools` > `Macro Development`），在 LVS 标签页新建脚本并粘贴上面的内容，然后运行。LVS 报告浏览器会打开，并把一切显示为绿色。

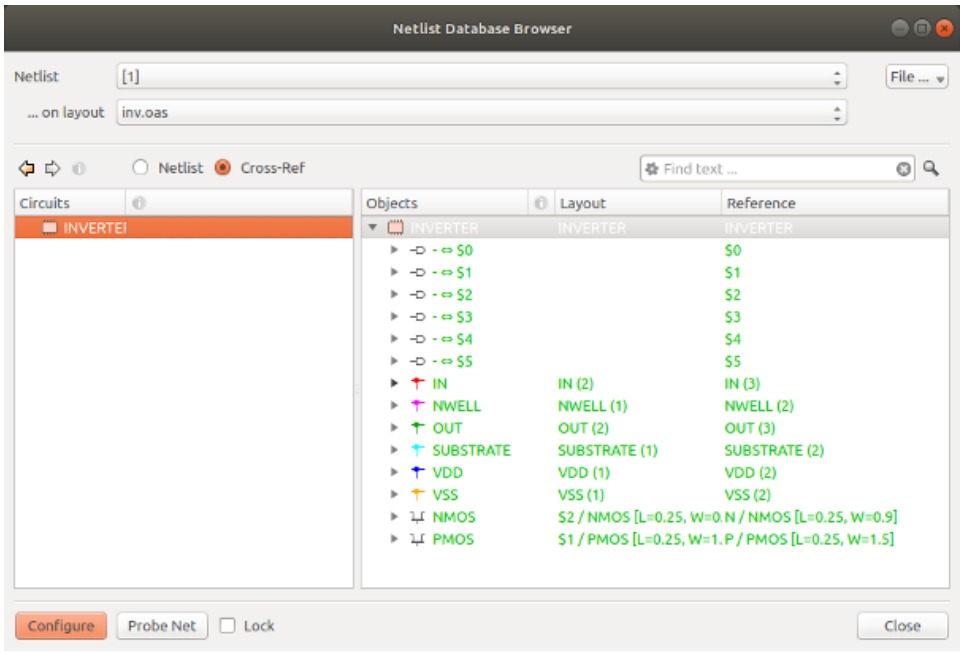


图 5.3: LVS 报告浏览器：全部匹配时显示为绿色

5.2.3 LVS 脚本的结构

LVS 脚本中第一条、也是最重要的一条语句应是 `deep`开关，它启用层次化模式。若不启用层次化模式，生成的 netlist 不含 subcircuit；这种平坦 netlist 比对效率低，也难调试。因此用 `deep`切到层次化模式：

```
deep
```

我们还指示 LVS 生成报告，并在结束后于报告浏览器中打开：

```
report_lvs
```

需要的话也可以把报告写入文件（见 `report_lvs`）：

```
report_lvs("inv.lvsdb")
```

接下来声明输入图层：

```
nwell = input(1, 0) active = input(2, 0) pplus = input(3, 0) nplus =
    input(4, 0) poly = input(5, 0) contact = input(6, 0) metal1 =
    input(7, 0) metal1_lbl = labels(7, 1) via1 = input(8, 0) metal2 =
    input(9, 0) metal2_lbl = labels(9, 1)
```

`input`与`labels`都是从版图源（如同 DRC，通常就是当前版图）取图层的函数。`input`取各类 Shape，而`labels`只取 Text；这里用`labels`取第一层金属的 label（GDS 第 7 层、datatype 1），第二层金属同理。

此外再创建一个空图层，用来表示 substrate。该层并不构成封闭区域，而是一堆 Shape，它们稍后会全部连到同一条（全局）net 上：

```
bulk = polygon_layer
```

给图层起的名字其实就是代表版图图层的变量。和 DRC 一样，可以用它们算出若干派生图层：

```
active_in_nwell = active & nwell pactive = active_in_nwell & pplus
    pgate = pactive & poly psd = pactive - pgate active_outside_nwell
    = active - nwell nactive = active_outside_nwell & nplus ngate =
    nactive & poly nsd = nactive - ngate
```

这些式子都是布尔运算：`&`是逻辑与，`-`是逻辑非。因此`active_in_nwell`是 ACTIVE 位于 NWELL 之内的部分，`active_outside_nwell`则是其外的部分。这些式子的主要目的是把 PMOS 与 NMOS 的源漏区区分开。

我们还分别取出 PMOS（pgate）与 NMOS（ngate）的栅区。准备好这些材料，就可以进入器件提取：

```
extract_devices(mos4("PMOS"), { "SD" => psd, "G" => pgate, "W" =>
    nwell, "tS" => psd, "tD" => psd, "tG" => poly, "tW" => nwell })
```

`extract_devices`的第一个参数是器件提取器，即负责实际提取某类器件的对象。这里的模板是`MOS4`，而我们想产生一个名为`PMOS`的新器件类，`mos4("PMOS")`即创建这样的提取器。

第二个参数是“图层符号 → 图层”的映射。每种器件提取器都定义了一套特定的图层符号；对任何器件都需要两类图层：提取器用来识别器件的输入图层，以及器件的 terminal 连接层。

输入图层用大写字母命名，terminal 输出图层则用小写`t`加 terminal 名表示。上面那份声明是完整的，但由于`tW`默认取`W`、`tS`与`tD`默认取`SD`，可以写得更简短：

```
extract_devices(mos4("PMOS"), { "SD" => psd, "G" => pgate, "W" =>
    nwell, "tG" => poly })
```

`NMOS`类也需要一个提取器，构造方式与 PMOS 完全相同：

```
extract_devices(mos4("NMOS"), { "SD" => nsd, "G" => ngate, "W" =>
    bulk, "tS" => nsd, "tD" => nsd, "tG" => poly, "tW" => bulk })
```

有了器件，工作就完成了一半。接下来要交代连接关系（见 `connect`）：

```
connect(psd, contact) connect(nsd, contact) connect(poly, contact)
    connect(contact, metal1) connect(metal1, metal1_lbl) # attaches
    labels connect(metal1, via1) connect(via1, metal2) connect(metal2,
     metal2_lbl) # attaches labels
```

这些语句把 PMOS 源漏区（psd）与 CONTACT 相连，把 NMOS 源漏区（nsd）也与 CONTACT 相连，POLY 同样连到 CONTACT。注意我们在器件提取时已把 psd、nsd、poly 指定为 terminal 输出 `tS`、`tD`、`tG`；把它们纳入连接关系之后，这些区域就成为对应 terminal 的一部分。

金属叠层很简单（CONTACT 接 METAL1，METAL1 经 VIA1 接 METAL2）。把 label 图层纳入连接关系，label 就自动挂到 net 上：net 提取器会从这些相连的 Text 对象中取出字符串，作为相应 net 的名字。

此外还需要做两处特殊连接（见 `connect_global`）：

```
connect_global(bulk, "SUBSTRATE") connect_global(nwell, "NWELL")
```

全局连接的意思是：某图层上的所有 Shape 都属于同一条 net——即便彼此不接触——且这条 net 在 circuit 与 subcircuit 之间共享。对 bulk 层而言这当然是正确的，但对 NWELL 层未必：孤立的 NWELL 区块并不应连在一起。

至此，生成 netlist 所需的全部输入都已给定，接下来指定参考 netlist：

```
schematic("inv.cir")
```

还有两步可选但值得推荐：层次对齐与提取网表的简化：

```
align # flattens unpaired circuits netlist.simplify # removes floating
    nets, combines devices
```

`align`会把另一个 netlist 中不存在的 circuit 去掉，把其内容并入父 Cell。这样就能甩掉版图中常见、但 schematic 中并没有对应 Cell 的辅助单元（例如器件 PCell）。`netlist.simplify`则通过移除悬空 net、合并器件（例如指状晶体管）来精简 netlist，并会为接口建立 pin。

顺序应是先 `align`，再 `netlist.simplify`；两者都必须在 `compare`之前执行才有效。`align`见第 5.1 节，`netlist.simplify`见第 5.9 节。

最后，一切就绪后即可触发比对：

```
compare
```

若在脚本开头插入一条写出 netlist 的语句（见 `target_netlist`），就能得到提取网表的 SPICE 版本：

```
# SPICE output statement (insert at beginning of script):
   target_netlist("inv_extracted.cir", write_spice, "Extracted by
   KLayout")
```

由于 LVS 匹配成功，提取网表与参考网表几乎一致，只是多了源漏区面积、周长之类几何参数：

```
* Extracted by KLayout * cell INVERTER .SUBCKT INVERTER * net 1 IN *
   net 2 VSS * net 3 VDD * net 4 OUT * net 5 NWELL * net 6 SUBSTRATE
   * device instance $1 r0 *1 1.025,4.95 PMOS M$1 3 1 4 5 PMOS L=0.25
   U W=1.5U AS=0.675P AD=0.675P PS=3.9U PD=3.9U * device instance $2
   r0 *1 1.025,0.65 NMOS M$2 2 1 4 6 NMOS L=0.25U W=0.9U AS=0.405P AD
   =0.405P PS=2.7U PD=2.7U .ENDS INVERTER
```

5.2.4 带下压二极管的反相器

上面那个反相器 Cell 单独使用并没有太大用处，因为它缺少把 n-well 与 substrate 固定在确定电位的手段。这要靠下压二极管（tie-down diode）来实现。

下压二极管就是有源区上的接触。这些有源区在 substrate 上注 p+、在 n-well 内注 n+（与晶体管相反的掺杂类型）。有了这样的掺杂，金属接触就不会在硅体上形成肖特基势垒，而是表现为欧姆接触。所以严格说，这种“二极管”并不是整流意义上的真正二极管。

修改后的版图如下：

对应的 schematic 如下：

有了这个电路，n-well 始终处于 VDD 电位，substrate 则被拉到 VSS：

```
* Simple CMOS inverer circuit .SUBCKT INVERTER_WITH_DIODES VSS IN OUT
    VDD Mp VDD IN OUT VDD PMOS W=1.5U L=0.25U Mn OUT IN VSS VSS NMOS
   W=0.9U L=0.25U .ENDS
```

把下压二极管的提取包括进来之后，LVS 脚本略长一些：

```
# LVS script (demo technology, KLayout manual) # Preamble: deep #
   Reports generated: report_lvs # LVS report window # Drawing layers
   : nwell = input(1, 0) active = input(2, 0) pplus = input(3, 0)
   nplus = input(4, 0) poly = input(5, 0) contact = input(6, 0)
   metal1 = input(7, 0) metal1_lbl = labels(7, 1) via1 = input(8, 0)
   metal2 = input(9, 0) metal2_lbl = labels(9, 1) # Bulk layer for
   terminal provisioning bulk = polygon_layer # Computed layers
   active_in_nwell = active & nwell pactive = active_in_nwell & pplus
```

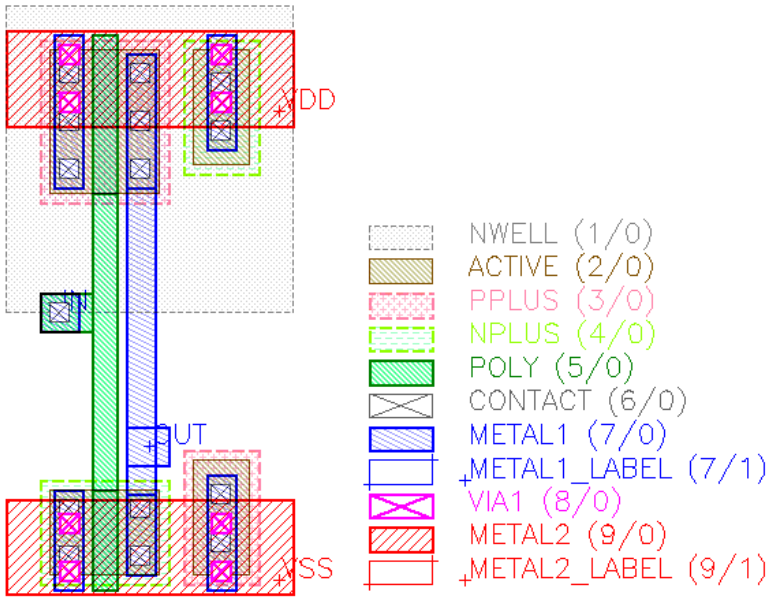


图 5.4: 带下压二极管的版图

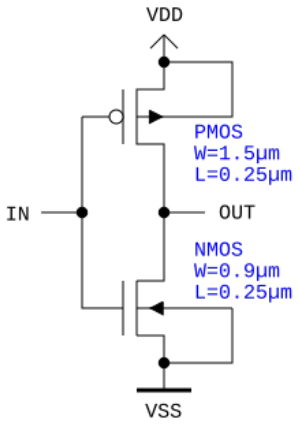


图 5.5: 带下压二极管的 schematic

```
 pgate = pactive & poly psd = pactive - pgate ntie =
active_in_nwell & nplus active_outside_nwell = active - nwell
nactive = active_outside_nwell & nplus ngate = nactive & poly nsd
= nactive - ngate ptie = active_outside_nwell & pplus # Device
extraction # PMOS transistor device extraction extract_devices(
mos4("PMOS"), { "SD" => psd, "G" => pgate, "W" => nwell, "tS" =>
```

```
    psd, "tD" => psd, "tG" => poly, "tW" => nwell }) # NMOS transistor
     device extraction extract_devices(mos4("NMOS"), { "SD" => nsd, "G
    " => ngate, "W" => bulk, "tS" => nsd, "tD" => nsd, "tG" => poly, "
    tW" => bulk }) # Define connectivity for netlist extraction # Inter
    -layer connect(psd, contact) connect(nsd, contact) connect(poly,
    contact) connect(ntie, contact) connect(nwell, ntie) connect(ptie,
     contact) connect(contact, metal1) connect(metal1, metal1_lbl) #
    attaches labels connect(metal1, via1) connect(via1, metal2)
    connect(metal2, metal2_lbl) # attaches labels # Global
    connect_global(bulk, "SUBSTRATE") connect_global(ptie, "SUBSTRATE
    ") # Compare section schematic("inv2.cir") align netlist.simplify
    compare
```

主要差别在于计算 n 型下压区（n-well 内）与 p 型下压区的区域，这很简单：

```
ntie = active_in_nwell & nplus ptie = active_outside_nwell & pplus
```

器件提取不变，但要把下压区纳入连接关系：

```
connect(ntie, contact) connect(nwell, ntie) connect(ptie, contact)
```

把 ntie 连到 contact 与 nwell 之后，就连通到了 n-well，n-well 因而表现得像一个导电层（尽管电阻会很大）。别忘了 PMOS 的器件提取器把 bulk terminal 也放在 nwell 上，因此晶体管会自动连到 n-well 所在的 net。

ptie 不能简单地这样连，因为“substrate”并没有对应的 Polygon；但可以把 ptie 纳入全局连接：

```
connect_global(bulk, "SUBSTRATE") connect_global(ptie, "SUBSTRATE")
```

此时 nwell 不再列入全局连接，于是不会再错误地把所有 nwell 区域都当作连在一起。

提取出的网表显示 NMOS 与 PMOS 的 bulk terminal 都接在源极上（漏源在此等价）：

```
* Extracted by KLayout * cell INVERTER_WITH_DIODES .SUBCKT
    INVERTER_WITH_DIODES * net 1 IN * net 2 VDD * net 3 OUT * net 4
    VSS * device instance $1 r0 *1 1.025,4.95 PMOS M$1 2 1 3 2 PMOS L
    =0.25U W=1.5U AS=0.675P AD=0.675P PS=3.9U PD=3.9U * device
    instance $2 r0 *1 1.025,0.65 NMOS M$2 4 1 3 4 NMOS L=0.25U W=0.9U
    AS=0.405P AD=0.405P PS=2.7U PD=2.7U .ENDS INVERTER_WITH_DIODES
```

5.3 LVS 器件

5.3.1 器件提取器与器件类

KLayout 用两个概念来处理器件的多样性：

device class（器件类）即器件的分类。既有电阻、MOS 晶体管这类较宽泛的类别，也可以建立更具体的类别来表示具体形态，例如 NMOS 与 PMOS。器件类还决定器件如何合并。

器件类的文档见第 5.4 节。

device extractor（器件提取器）才是真正分析版图并产出器件的对象。与器件类一样，提取器也有通用类型，每种提取器产出属于某个特定类的器件。

器件提取器的文档见第 5.5 节。

5.4 LVS 器件类

KLayout 实现了多种标准器件类，它们是构造各类具体器件的基础。例如 `MOS4` 类就是 NMOS 与 PMOS 晶体管具体器件类的基础。

5.4.1 Resistor（电阻）

DeviceClassResistor

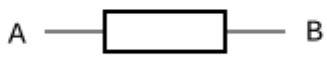


图 5.6: 普通电阻

普通电阻有两个 terminal：A 与 B，具有以下参数：

R：电阻值，单位欧姆

L：长度，单位微米

W：宽度，单位微米

A：电阻区面积，单位平方微米

P：电阻区周长，单位微米

电阻可以按并联或串联方式合并。

在 SPICE 中，普通电阻由 R 元件表示；对应的 API 类见第 5.4 节。

5.4.2 带 bulk terminal 的电阻

```
DeviceClassResistorWithBulk
```

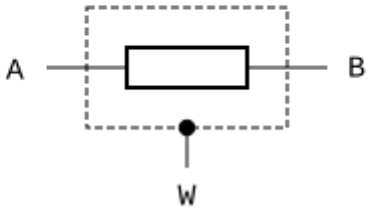


图 5.7: 带 bulk terminal 的电阻

带 bulk terminal 的电阻是普通电阻的扩展：参数相同，但多一个 terminal W，连到电阻所在的区域（例如 well 或 substrate）。

若这些电阻的 bulk terminal 连在同一条 net 上，它们可以按并联或串联方式合并。

带 bulk 电阻的 API 类见第 5.4 节。

5.4.3 Capacitor（电容）

```
DeviceClassCapacitor
```

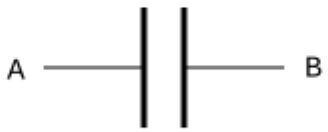


图 5.8: 普通电容

普通电容有两个 terminal：A 与 B，具有以下参数：

C：电容值，单位法拉

A：电容区面积，单位平方微米

P：电容区周长，单位微米

在 SPICE 中，普通电容由 C 元件表示；对应的 API 类见第 5.4 节。

5.4.4 带 bulk terminal 的电容

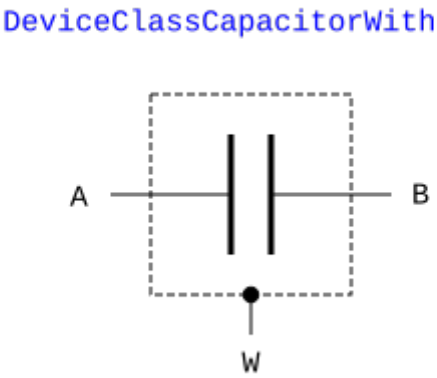


图 5.9: 带 bulk terminal 的电容

带 bulk terminal 的电容是普通电容的扩展：参数相同，但多一个 terminal W，连到电容所在的区域（例如 well 或 substrate）。

若这些电容的 bulk terminal 连在同一条 net 上，它们可以按并联或串联方式合并。

带 bulk 电容的 API 类见第 5.4 节。

5.4.5 Diode（二极管）

二极管有两个 terminal：A 与 C，分别为阳极与阴极，具有以下参数：

A：二极管面积，单位平方微米

P：二极管周长，单位微米

二极管按并联方式合并（A 对 A、C 对 C），合并后其面积与周长相加。

在 SPICE 中，二极管由 D 元件表示，并以器件类名作为 model 名；对应的 API 类见第 5.4 节。

5.4.6 MOS 晶体管

三端 MOS 晶体管的 terminal 为 S、G、D，即源、栅、漏，其中 S 与 D 可互换。其参数为：

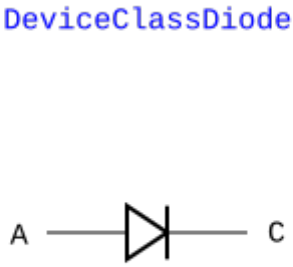


图 5.10: 二极管

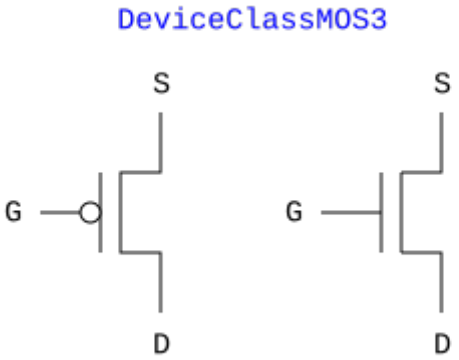


图 5.11: 三端 MOS 晶体管

W：栅宽，单位微米

L：栅（沟道）长，单位微米

AS：源区面积，单位平方微米

PS：源区周长，单位微米

AD：漏区面积，单位平方微米

PD：漏区周长，单位微米

当源漏相连、栅极相连且栅长完全相同时，MOS3 晶体管按并联方式合并，合并后宽度、面积与周长相加。

5.4.7 带 bulk 的 MOS 晶体管

三端 MOS 晶体管的 API 类见第 5.4 节。

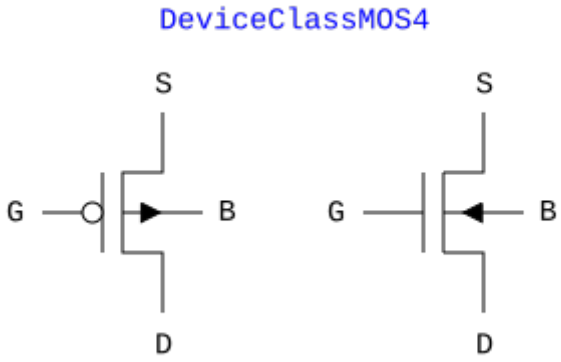


图 5.12: 四端 MOS 晶体管

四端晶体管是三端晶体管的扩展，多一个 bulk terminal [B]。这大概是最重要的晶体管器件，因为四端版本与 SPICE 的 [M]元件兼容。

若这些 MOS 晶体管的 bulk terminal 连在同一条 net 上，它们可以像三端版本那样按并联方式合并。

在 SPICE 中，MOS4 器件由 [M]元件表示，并以器件类名作为 model 名；对应的 API 类见第 5.4 节。

5.4.8 双极晶体管

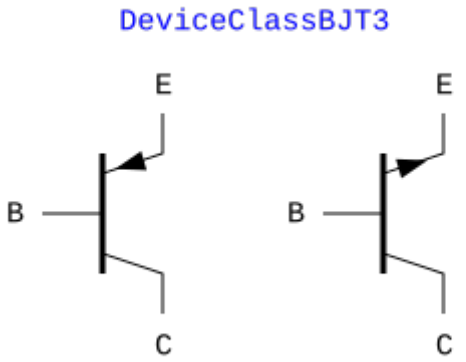


图 5.13: 三端双极晶体管

三端双极晶体管可以是 NPN 型或 PNP 型。在 KLayout 中，这种器件类型既能表示横向型也能表示纵向型。其参数为：

AE：发射区面积，单位平方微米

PE：发射区周长，单位微米

NE：发射极个数（初始为 1）

AB：基区面积，单位平方微米

PB：基区周长，单位微米

AC：集电区面积，单位平方微米

PC：集电区周长，单位微米

提取时，多发射极版本会被提取成多个器件——每个发射区一个——此时 NE = 1。双极晶体管按并联方式合并，合并后发射极参数 AE、PE 与 NE 相加。

在 SPICE 中，BJT3 器件由 Q 元件表示，并以器件类名作为 model 名；对应的 API 类见第 5.4 节。

5.4.9 带衬底的双极晶体管

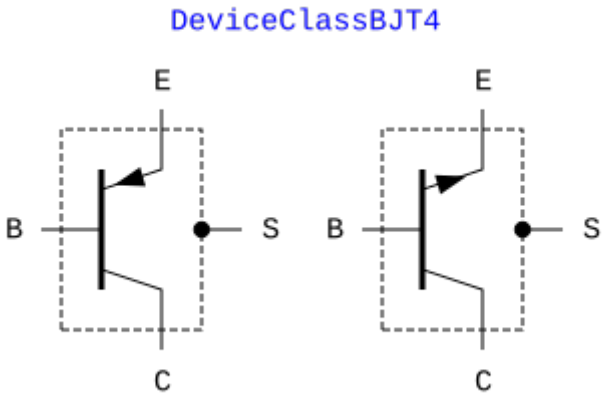


图 5.14: 带衬底的四端双极晶体管

四端晶体管是三端晶体管的扩展，多一个 bulk terminal S。

若这些双极晶体管的 bulk terminal 连在同一条 net 上，它们可以像三端版本那样按并联方式合并。

在 SPICE 中，BJT4 器件由带四个节点的 Q 元件表示，并以器件类名作为 model 名；对应的 API 类见第 5.4 节。

5.5 LVS 器件提取器

器件提取器是器件提取过程中真正的“干活者”。KLayout 自带多种现成的器件提取器，也可以在 LVS 脚本框架内（用 Ruby 编写）实现自定义提取器。

5.5.1　电阻提取器（resistor 与 resistor_with_bulk）

电阻提取器假设版图由一段电阻“连线”加两个端帽（contact）组成：连线用图层符号 R 指定，端帽用 C 指定。

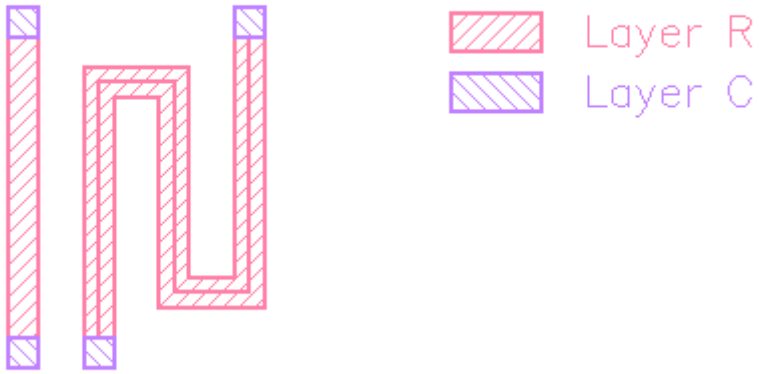


图 5.15: 电阻提取器的输入：R 连线与 C 端帽

提取器由方块数（number of squares）与方块电阻（sheet resistance）计算阻值，方块电阻在创建提取器时给出。

```
sheet_rho = 0.5 model_name = "RES" extract_devices(resistor(
    model_name, sheet_rho), { "R" => res_layer, "C" => contact_layer
    })
```

普通电阻有两个 terminal，分别输出到 tA 与 tB 两个 terminal 图层上。若未指定 tA 或 tB，则 A 或 B terminal 分别写到 C 图层上。

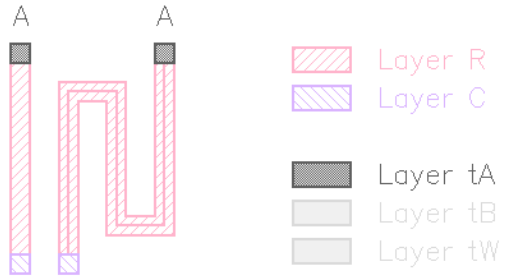


图 5.16: tA terminal 输出

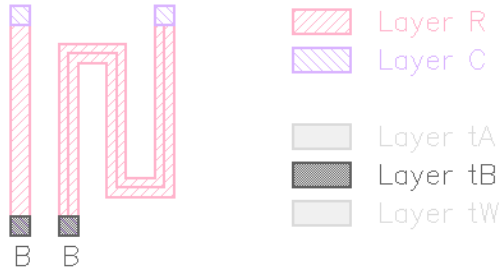


图 5.17: tB terminal 输出

对带 bulk 的电阻，连线区域作为 W terminal 输出到 tW terminal 图层。

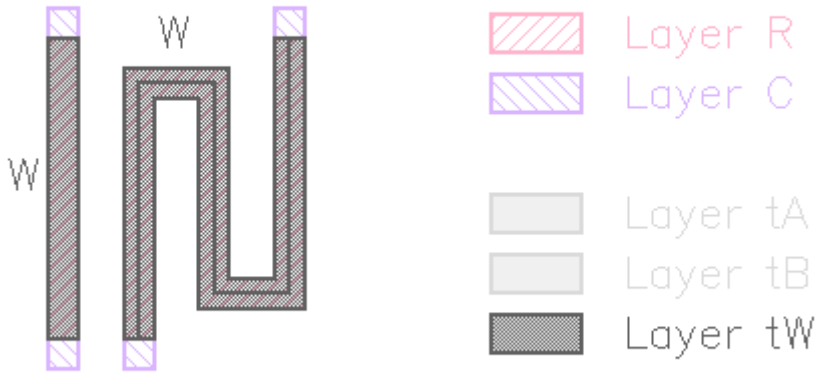


图 5.18: tW terminal 输出

注意：阻值计算用的是一种简单近似——沿 R Polygon 的周长走一圈，把周长分成与 C 层接触的部分和不接触的部分，由此得出方块数。因此这是一种粗略但通常可用的估算。

5.5.2 电容提取器（capacitor 与 capacitor_with_bulk）

电容被假定由两块“极板”构成（即纵向电容），极板分别在 P1 与 P2 层上；电容取自这两层的重叠区域。

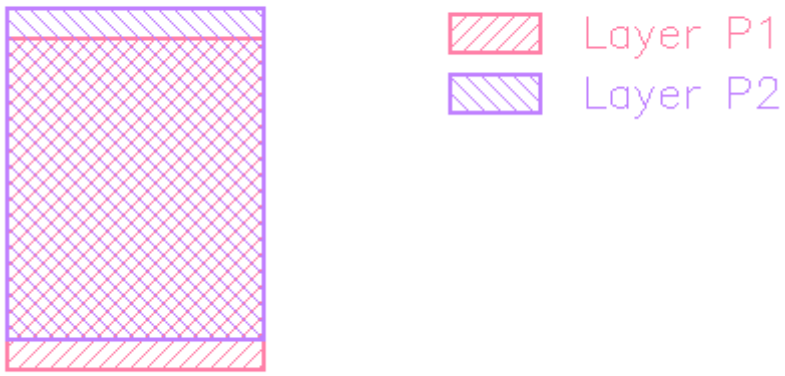


图 5.19: 纵向电容：两块极板 P1 与 P2 的重叠

提取器由重叠面积与单位面积电容（F/平方微米）计算电容值。

```
area_cap = 1.5e-15 model_name = "CAP" extract_devices(capacitor(
    model_name, area_cap), { "P1" => metal1, "P2" => metal2 })
```

普通电容有两个 terminal，分别输出到 tA 与 tB 图层。若未指定，A 与 B terminal 分别写到 P1 与 P2 图层上。

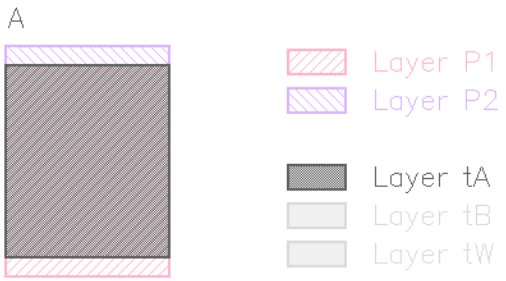


图 5.20: tA terminal

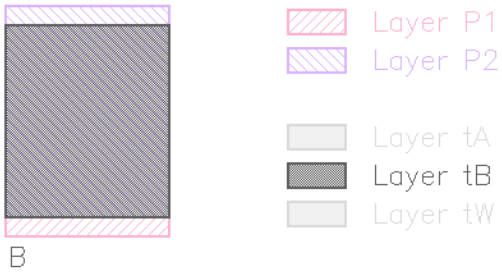


图 5.21: tB terminal

对带 bulk 的电容，电容区域作为 W terminal 输出到 tW terminal 图层。

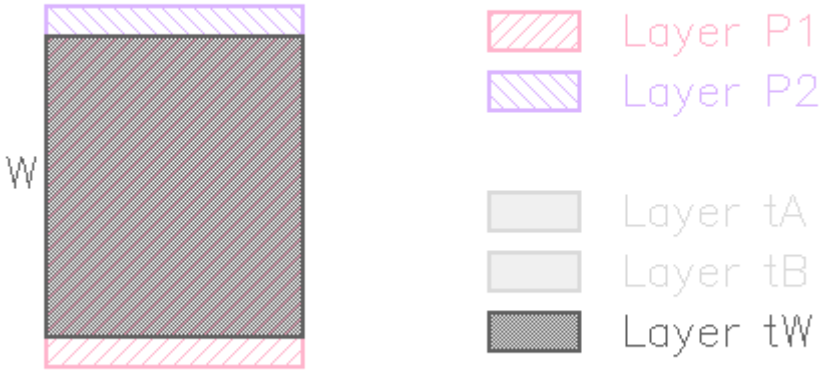


图 5.22: tW terminal

5.5.3 二极管提取器（diode）

二极管被假定由两个纵向注入区（well、扩散区）构成：一个是 p 型（P 层），另一个是 n 型（N 层）。这两层同时也构成二极管的阳极（p）与阴极（n）。

```
model_name = "DIODE" extract_devices(diode(model_name), { "P" =>
    pplus, "N" => nwell })
```

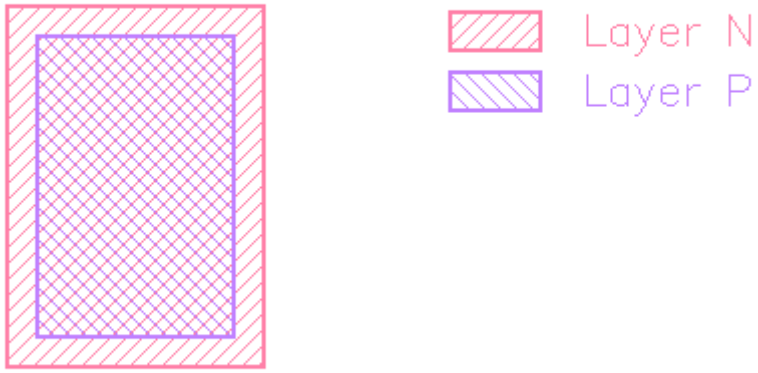


图 5.23: 二极管提取输入：P 与 N 两个注入区

二极管有两个 terminal，分别输出到 tA 与 tC 图层。若未指定 tA，A terminal 写到 P 层；若未指定 tC，C terminal 写到 N 层。

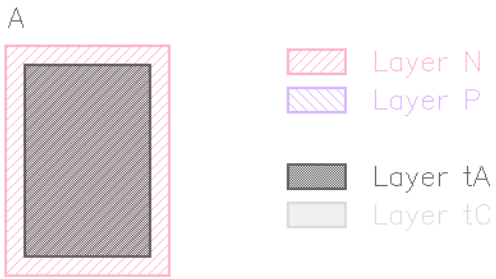


图 5.24: tA terminal

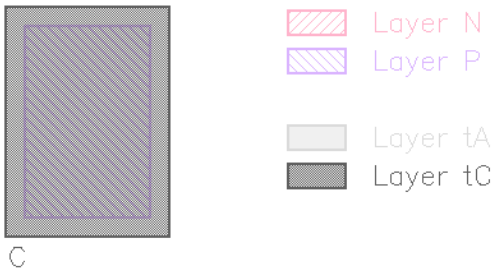


图 5.25: tC terminal

5.5.4 MOS 晶体管提取器（mos3 与 mos4）

MOS 晶体管靠栅极（G 输入）与源漏区（SD 输入）识别：源漏区必须与栅极图形分离，栅极与源漏区相接触的边决定器件宽度，而垂直于该边的方向决定沟道长度。

```
model_name = "PMOS" extract_devices(mos4(model_name), { "SD" => (
    active - poly) & pplus, "G" => (active & poly), "W" => nwell, "tS"
     => active, "tD" => active, "tG" => poly, "tB" => nwell })
```

MOS3 器件产生三个 terminal，分别输出到 tS、tG、tD 图层（依次为源、栅、漏）。

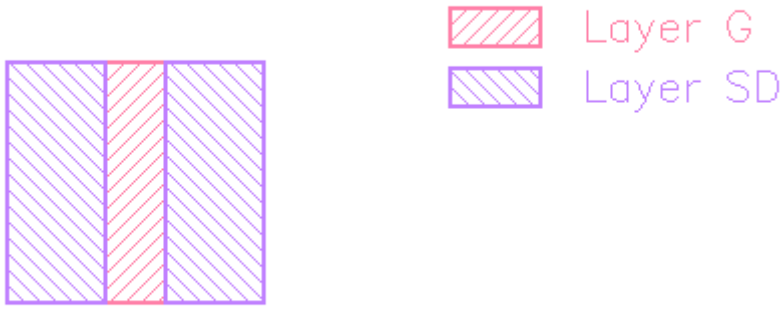


图 5.26: MOS 提取输入：G 与 SD

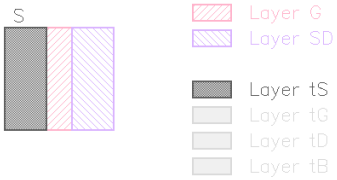


图 5.27: tS terminal

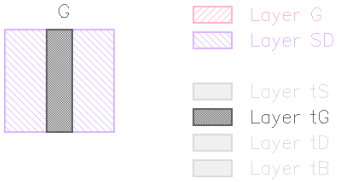


图 5.28: tG terminal

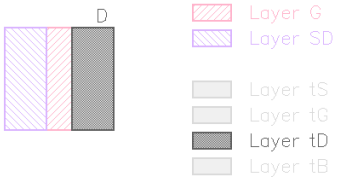


图 5.29: tD terminal

MOS4 器件多一个 terminal（bulk），写到 `tB`。

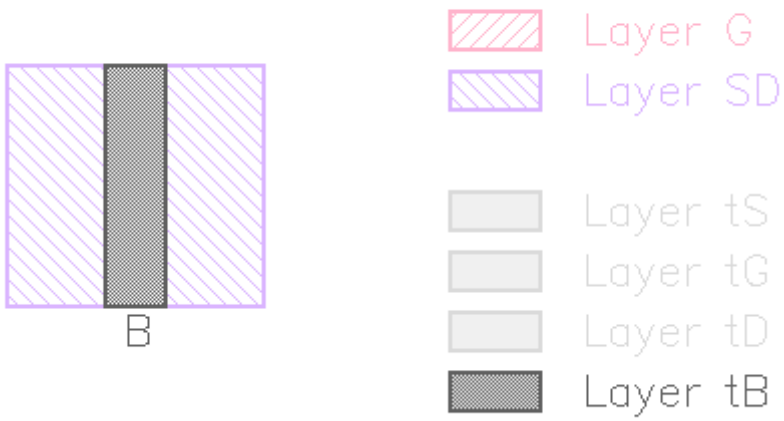


图 5.30: MOS4 的 bulk terminal

5.5.5 扩散 MOS 晶体管提取器（dmos3 与 dmos4）

DMOS 器件与 MOS 器件基本相同，区别在于其源、漏是区分开的。扩散型 MOS 晶体管常常如此，故得此名。

DMOS 与 MOS 器件共用同一个器件类。DMOS 器件被配置为源漏不可互换，因此若它们被互换，netlist 比对会报告为错误。

DMOS 晶体管靠栅极（G输入）、源区（S输入）与漏区（D输入）识别；源漏区须与栅极分离，其与栅极相接触的边决定器件宽度，垂直方向的尺寸决定沟道长度。

5.5.6 双极晶体管提取器（bjt3 与 bjt4）

双极晶体管基本上分两类：纵向型与横向型。

横向晶体管由注入或扩散的 well 在晶圆上形成交替的 n/p 结构，其基本识别区是基区：集电区与发射区位于基区之内或与其重叠，采用相反的掺杂类型。

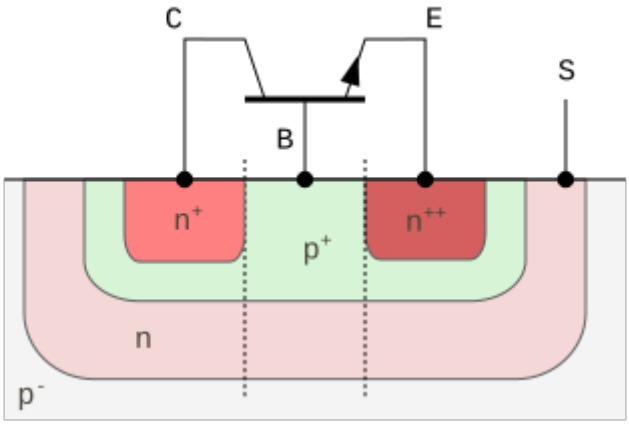


图 5.31: 横向双极晶体管的结构

纵向晶体管由叠放的 n/p well 构成，有时在标准 CMOS 工艺中会作为寄生成分出现。例如以 substrate 作集电区、nwell 作基区、pplus 注入作发射区，就能构成一个 PNP 晶体管。

两种情况下，一个基区之内都可能有多个发射区；此时每个发射区各提取一个晶体管。

5.5.6.1 纵向双极晶体管

纵向双极晶体管的输入取自 B（基区）、C（集电区）与 E（发射区），其中 C是可选的。

尤其对双极器件而言，给出有用的 terminal 输出层很重要。通常 well 与扩散区会通过 contact 相连（此处暂不考虑肖特基二极管），因此把 terminal 送到与 contact 相接触的形状上是明智的做法。

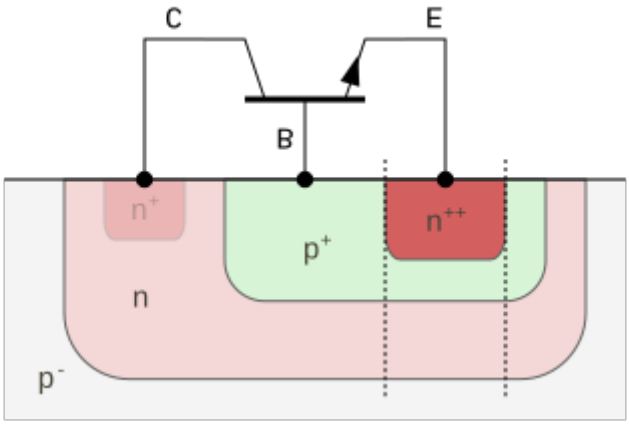


图 5.32: 纵向双极晶体管的结构

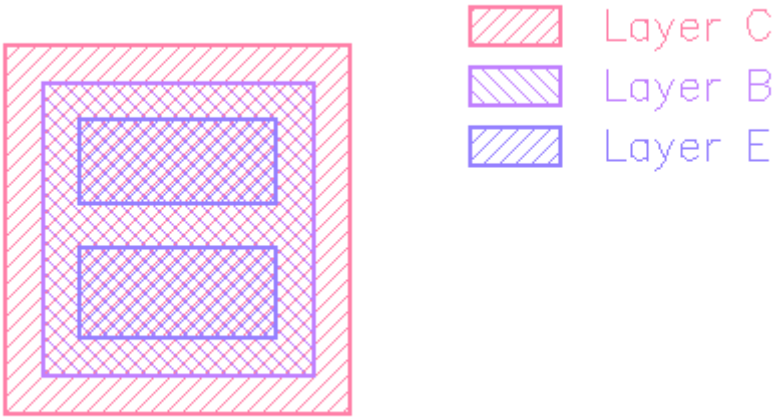


图 5.33: 纵向双极：输入 B、C、E

```
model_name = "PNP" extract_devices(bjt3(model_name), { "C" =>
    collector, "B" => base, "E" => emitter, "tC" => contact, "tB" =>
    contact, "tE" => contact })
```

BJT3 器件产生三个 terminal，分别输出到 `tC`、`tB`、`tE`图层（依次为集电、基区、发射区）。

若集电区为空（例如 p 型 substrate），则把基区形状复制到 `tC`输出层作为集电区 terminal。

BJT4 器件多一个 terminal（substrate），写到 `tS`。`tS`是发射区形状的副本，但连到 substrate terminal 上。

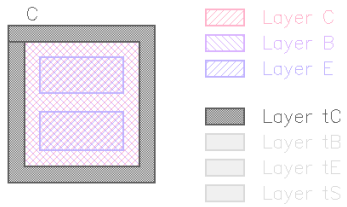


图 5.34: tC terminal

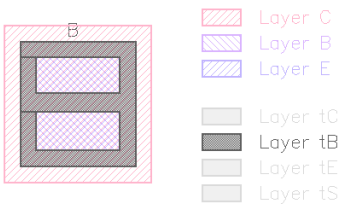


图 5.35: tB terminal

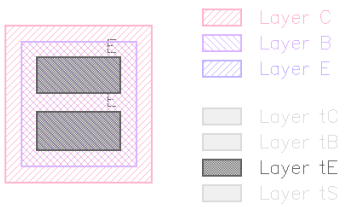


图 5.36: tE terminal

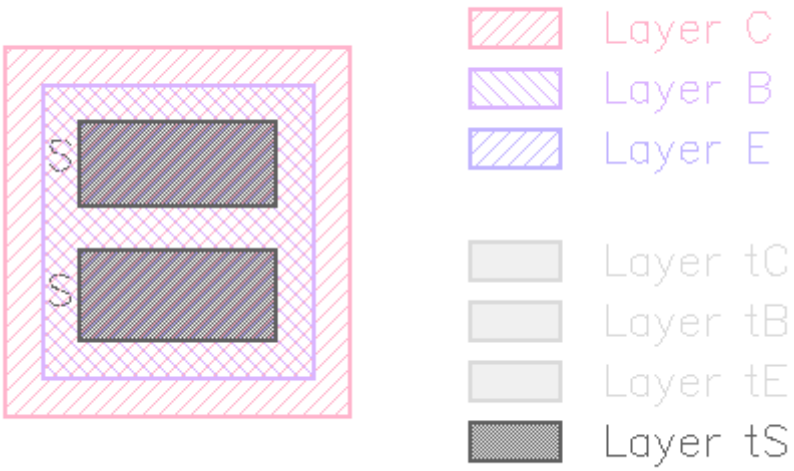


图 5.37: BJT4 的 substrate terminal

5.5.6.2 横向双极晶体管

横向双极晶体管的输入同样取自 `B`、`C`与 `E`。对横向晶体管，`C`不可省略，且不得完全覆盖基区。除此之外，BJT3 与 BJT4 提取器的用法完全相同。

典型的横向晶体管由基区内的一圈集电区（collector ring）与一个发射岛构成：

横向情形下双极提取器产生的 terminal 与纵向情形相同，只是几何不同：

同样地，对 BJT4，`tS`是发射区形状的副本但连到 substrate terminal。

5.5.7 器件提取器与器件类

`extract_devices`会返回所生成器件的 DeviceClass 对象，可用它做一些基本调整，其中最重要的是启用或禁用某些参数。

多数器件提取器实际提取的参数比默认给出的更多。例如电阻提取器不只是求阻值，还会求出电阻条的长度 L、宽度 W 以及面积 A 与周长 P；默认只有其中一部分参与比对。

参数可以用 `enable_parameter`完全启用，或用 `disable_parameter`禁用；`tolerance`可以启用某参数参与比对并指定比对容差；`ignore_parameter`则让某参

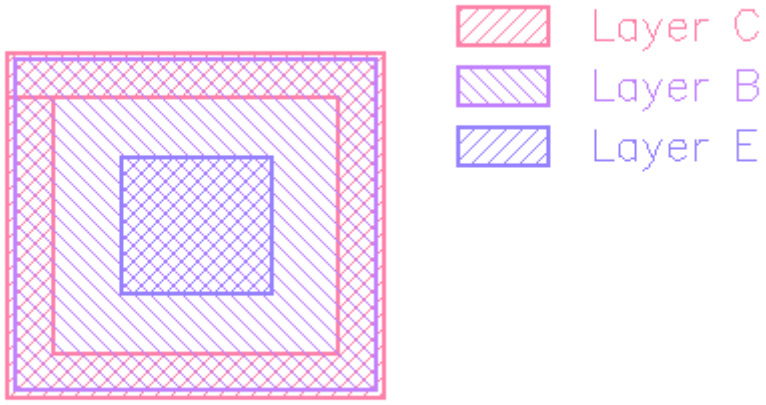


图 5.38: 横向双极：集电环与发射岛

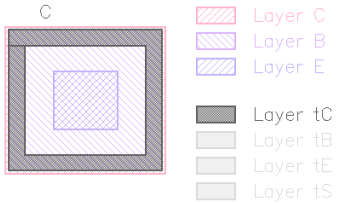


图 5.39: 横向情形的 terminal 几何

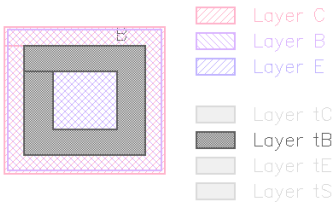


图 5.40: 横向 BJT4 的 substrate terminal

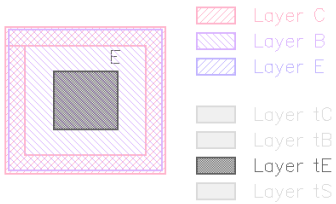


图 5.41: 横向情形 terminal 输出

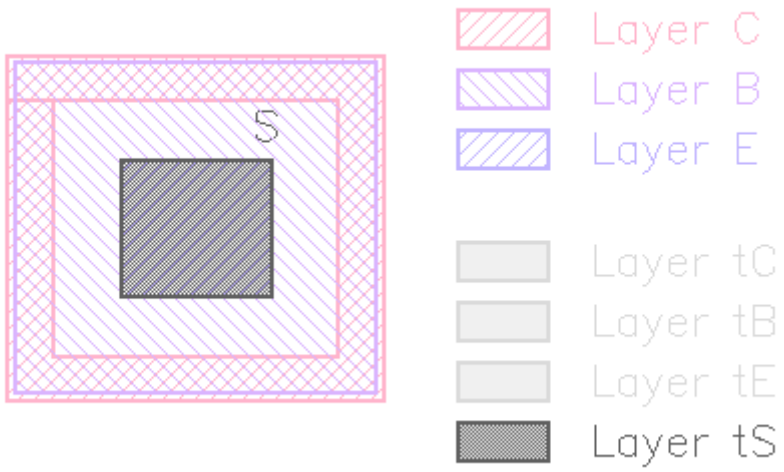


图 5.42: 横向 BJT4 的 tS

数在比对步骤中被忽略。

定制内置器件提取器的另一种方式是提供自定义的器件类。下面的代码新建一个电阻类，修改预配置的器件参数定义以启用 `W` 与 `L`。

```
class MyResistor < RBA::DeviceClassResistor def initialize super
    enable_parameter("W", true) enable_parameter("L", true) end end
    ... extract_devices(resistor("RES", 1, MyResistor), ...)
```

使用自定义器件类还带来一个可能：补充额外参数，或者借助电阻提取器的提取机制实现一种全新的器件。唯一的要求是参数与 terminal 定义必须兼容。

5.6 LVS 输入输出

LVS（就 netlist 提取而言 DRC 也一样）提供读写 netlist、schematic、标注版图与 LVS 结果的接口。I/O 分三大类：

- Netlist：纯粹的电路信息，配合 subcircuit 就构成层次化 netlist。目前可导入导出的格式是某种 SPICE netlist 方言，读写过程可以定制以获得一定灵活性。
- Layout-to-netlist database（L2N DB）：也称 extracted netlist 或 annotated layout，即取自原版图的 netlist 连同相应的 Shape。凡涉及非器件的 Shape，该数据库都能在几何上重建 net；器件则被抽象成类似“黑盒”的表示。
- LVS result database（LVS DB）：即 L2N 数据库加上参考 netlist 与 cross reference——后者是配对成功的 circuit、net、device、pin、subcircuit 及其状态信息的列表，既是查找表也是调试工具。

5.6.1 写出 netlist

可以把 netlist 写成文件，例如提供给（功能）仿真器。在 LVS 脚本中，全局语句 `target_netlist` 触发 netlist 写出（细节见 `target_netlist`）。

```
target_netlist("output.cir", write_spice, "Created by KLayout")
```

这条语句原则上可以出现在脚本任何位置，netlist 会在脚本成功执行后写出。第一个参数是文件路径（默认相对于原版图文件），第二个参数是 writer：`write_spice` 创建一个 SPICE netlist writer。

`write_spice` 配置函数有两个选项：

```
write_spice(use_net_names, with_comments)
```

两个选项都是布尔值。若存在且为真，第一个选项让 writer 使用真实的 net 名而不是数字编号；`with_comments` 若存在且为真，会在 netlist 中嵌入调试注释，显示实例位置、pin 名等。

还可以通过给 SPICE writer 提供 delegate 来做更多定制。做法是继承 `NetlistSpiceWriterDe` 并重实现其中一个或多个预留方法。

下面这个例子不输出器件元件，而是输出 subcircuit 调用：

```
# Write extracted netlist to extracted.cir using a special # writer
    delegate # This delegate makes the writer emit subcicuit calls
    instead of # standard elements for the devices class
    SubcircuitModels < RBA::NetlistSpiceWriterDelegate def
    write_header emit_line(".INCLUDE 'models.cir'") end def
    write_device(device) str = "X" + device.expanded_name device_class
     = device.device_class device_class.terminal_definitions.each do |
    td| str += " " + net_to_string(device.net_for_terminal(td.id)) end
     str += " " + device_class.name str += " PARAMS:" device_class.
    parameter_definitions.each do |pd| str += " " + pd.name + ("=%.12g"
     % device.parameter(pd.id)) end emit_line(str) end end # Prepare a
     writer using the new delegate custom_spice_writer = RBA::
    NetlistSpiceWriter::new(SubcircuitModels::new) custom_spice_writer
    .use_net_names= true custom_spice_writer.with_comments = false #
    The declaration of netlist production using the new custom writer
    target_netlist("extracted.cir", custom_spice_writer, "Extracted by
     KLayout")
```

对 LVS 入门一节中的简单反相器，这段脚本会产生如下 netlist。它不再打印默认的 `M`元件，而是产生 subcircuit 调用，从而可以在 subcircuit 里放更精细的模型；器件类名就用来指向这些模型 subcircuit。

```
* Extracted by KLayout .INCLUDE 'models.cir' .SUBCKT INVERTER X$1 VDD
     IN OUT NWELL PMOS PARAMS: L=0.25 W=1.5 AS=0.675 AD=0.675 PS=3.9
    PD=3.9 X$2 VSS IN OUT SUBSTRATE NMOS PARAMS: L=0.25 W=0.9 AS=0.405
     AD=0.405 PS=2.7 + PD=2.7 .ENDS INVERTER
```

netlist 也可以直接从 netlist 对象写出：脚本中先用 `netlist`函数取得对象（该函数若尚未提取会先触发一次提取），再调用它的写出方法。与 `target_netlist`不同，这种方式在调用处立即写出，而不是等脚本结束。

5.6.2 读取 netlist

读取 netlist 的主要用途是在 LVS 中作比对：参考 netlist 用 `schematic`函数读入（见 `schematic`）。

```
schematic("inverter.cir")
```

目前的 SPICE 支持有一些限制：默认只实现了部分元件，即 `M`（得到 `MOS4`器件类）、`Q`（得到 BJT3 或 BJT4）、`R`（电阻）、`C`（电容）与 `D`（二极管）。

与 SPICE writer 一样，reader 也可以提供 delegate 来定制：继承相应类并重实现其预留方法即可。

下面这个例子定制 reader，从 subcircuit 模型而不是 M 元件取 MOS 器件；它基本上正是前面 writer 定制的反向操作（仅针对 MOS 器件）。

```
# Provides a SPICE netlist reader delegate which turns # some
   subcircuit models (for subcircuits NMOS and PMOS) # into devices
   class SubcircuitModelsReader < RBA::NetlistSpiceReaderDelegate #
   implements the delegate interface: # says we want to catch these
   subcircuits as devices def wants_subcircuit(name) name == "NMOS"
   || name == "PMOS" end # implements the delegate interface: # take
   and translate the element def element(circuit, el, name, model,
   value, nets, params) if el != "X" # all other elements are left to
    the standard implementation return super end if nets.size != 4
   error("Subcircuit #{model} needs four nodes") end # provide a
   device class cls = circuit.netlist.device_class_by_name(model) if
   ! cls cls = RBA::DeviceClassMOS4Transistor::new cls.name = model
   circuit.netlist.add(cls) end # create a device device = circuit.
   create_device(cls, name) # and configure the device [ "S", "G", "D
   ", "B" ].each_with_index do |t,index| device.connect_terminal(t,
   nets[index]) end # parameters in the model are given in micrometer
    units, so # we need to translate the parameter values from SI to
   um values: device.set_parameter("W", (params["W"] || 0.0) * 1e6)
   device.set_parameter("L", (params["L"] || 0.0) * 1e6) return true
   end end # Instantiate a reader using the new delegate reader = RBA
   ::NetlistSpiceReader::new(SubcircuitModelsReader::new) # Import
   the schematic with this reader schematic("inv_xmodels.cir", reader
   )
```

5.6.3 Layout-to-Netlist 数据库 / 报告

layout-to-netlist 数据库（L2N DB）用全局函数 `report_netlist` 写出。该函数可放在脚本任意位置，写出发生在脚本成功执行之后：

```
report_netlist("extracted.l2n")
```

若不给文件名，则只打开 netlist 浏览器而不写文件。layout-to-netlist 数据库是 KLayout 专用格式，含 netlist 信息以及版图中的 Shape 与 Instance 信息；L2N 数据库可以读入 netlist 浏览器中查看。

5.6.4 Layout-vs-Schematic 数据库 / 报告

Layout-vs-schematic 数据库（LVS DB）用全局函数 `report_lvs` 写出。该函数同样可放在脚本任意位置，写出发生在脚本成功执行之后：

```
report_lvs("extracted.lvsdb")
```

若不给文件名，则只打开 netlist 浏览器而不写文件。LVS 数据库是 KLayout 专用格式，含提取网表信息、参考网表以及 cross reference 表；LVS 数据库可以读入 netlist 浏览器中查看。

5.7 LVS 连接关系

5.7.1 层内连接与层间连接

LVS 脚本中的连接关系设置决定了连接如何建立。连接通常通过铝、铜这类导电材料形成：表示同一材料的 Polygon 相互连通；连接也可以跨越多个 Polygon，只要它们彼此接触即构成同一导电区域。

连接常常跨层进行，例如 via 就是绝缘层上的一个孔，把两层金属连起来。这类连接用 `connect` 语句建模（见 `connect`）：

```
connect(layer1, layer2)
```

connect 语句表示：当 layer1 与 layer2 的 Polygon 相互重叠时，二者电气相连。layer1 与 layer2 可以是原始图层或派生图层。`connect` 语句应写在需要 netlist 之前——也就是 `compare` 或任何读取 netlist 的语句之前。

下面是一个三层金属叠层的纵向剖面示例，以及相应的 `connect` 语句：

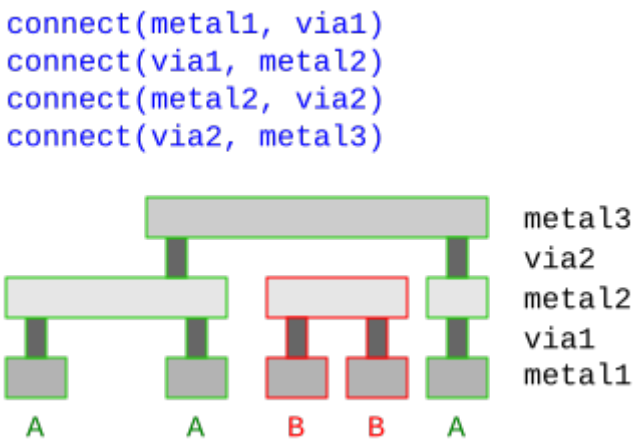


图 5.43

label 也可以纳入连接关系。label 通常放在金属层上：若 label 与金属 Shape 画在

同一层，用 `input` 读该层时它们会自动被包含；若只想从某层读 label，用 `labels` 函数。

要把 label 挂到金属层上，只需把 label 层与金属层连接起来：

```
metal1_labels = labels(10, 0) metal1 = input(11, 0) via1 = input(12,
    0) metal2_labels = labels(13, 0) metal2 = input(14, 0) connect(
    metal1, metal1_labels) connect(metal1, via1) connect(via1, metal2)
     connect(metal2, metal2_labels)
```

label 连到金属层之后，其文字将用于给形成的 net 命名。理想情况下，一条 net 上只有一段文字，或有多段但内容相同的文字，此时 net 名没有歧义；若一条 net 上有多个内容不同的 label，KLayout 会做出选择并给出警告。

5.7.2 全局连接

KLayout 支持“隐式全局连接”：某图层上的所有 Polygon 一律视为相连，无论它们在几何上是否接触。典型例子是 substrate（即 bulk），它代表略有导电性的衬底材料，而版图中并没有表示整块衬底的 Polygon。

```
connect_global(bulk, "VSS")
```

`connect_global` 的参数是全局连接的图层以及要创建的全局 net 名。下面的语句把 `bulk` 上的所有 Shape 连成一条 net，名为 `VSS`：每个 circuit 至少都有 `VSS` 这条 net。

5.7.3 隐式连接

隐式连接适合用来提供那些打算在更高层才建立的连接。设想某个 circuit 有一条很大的电源网：做版图时电源网可能还没有完全连通，因为计划在上层或由布线工具把它接起来。

为了能验证这样的 Cell，可以用“隐式连接”：给各部分 net 打上相同名字的 label，并假定它们相连。例如要给 `VDD` net 的各部分建立隐式连接，就在每部分上放一个 `VDD` label，然后加上：

```
connect_implicit("VDD")
```

`connect_implicit`（见 `connect_implicit`）可以出现多次，用来建立多条这样的连接。隐式连接在顶层被接受，但会给出警告，提示该连接应在更高层完成；在 sub-circuit 中，隐式连接同样会被报出，除非后续得到物理连接的确认。

也可以把某个版图声明为顶层版图，这会把“缺少物理连接”的警告升级为错误：

```
top_level(true)
```

在其他系统里，`connect_implicit` 这类功能也被称为“must connect”网。

可以用 glob 风格的通配符，把某一类 label 纳入 `connect_implicit` 语句：

```
connect_implicit("VDD*")
```

这样一来，所有带 `VDD1` label 的 net（或带 `VDD_5V` label 的 net）都会各自连在一起。但该语句只把 `VDD1` 与 `VDD1` 相连，不会把 `VDD1` 与 `VDD2` 相连；要让不同名字的 label 之间建立连接，需另外写语句。

`connect_implicit` 可以出现多次，每个语句都在扩大“哪些 label 会被连接”的范围。

上面的写法适用于所有 Cell。也可以把它限定到具体 Cell：第一个参数给 Cell 名的 glob 通配符，第二个参数给 net 名通配符。

下面这条语句把 `MEMCELL` 子 Cell 中所有带 `VDD` label 的 net 连起来：

```
connect_implicit("MEMCELL", "VDD")
```

该规则作用于所有匹配第一个参数通配符的 Cell。同样，`connect_implicit` 规则可以给多次，此时所有匹配到的规则共同起作用。

`connect_implicit` 语句必须写在 netlist 提取之前，通常在 `connect` 语句之前或紧随其后。

5.7.4 显式连接

显式连接适合用来强制建立那种 schematic 中存在、但在该 Cell 层面版图上并未物理建立的连接。例如下面这个反相器版图：

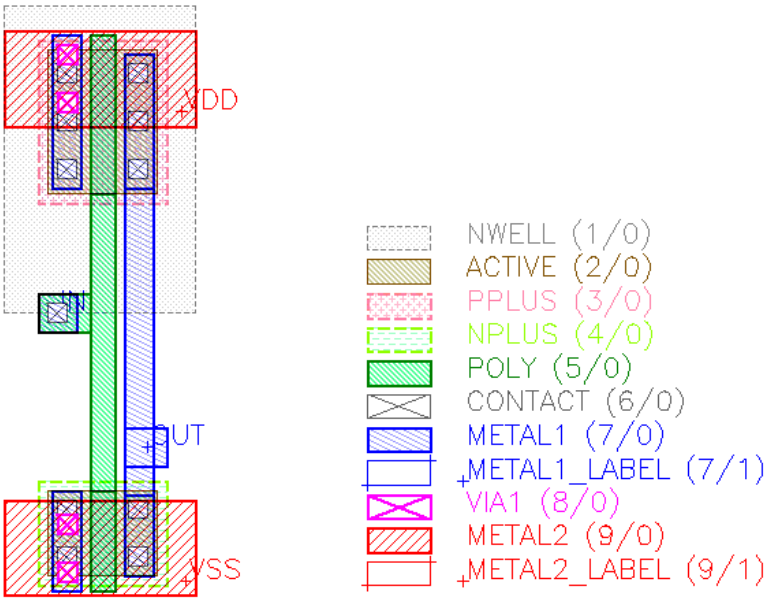


图 5.44

版图中没有下压二极管，因此 n-well 区没有物理连接，bulk 衬底也没有物理连接。

这样能省面积，但这些二极管迟早要通过别的方式补上，通常是在把标准单元组合起来的时候。

然而在反相器这一层并没有这种连接，于是反相器有独立的 bulk 与 n-well pin；而 schematic 有时是简化版，不提供这些 pin。这样就出现了版图与 schematic 之间的固有失配。

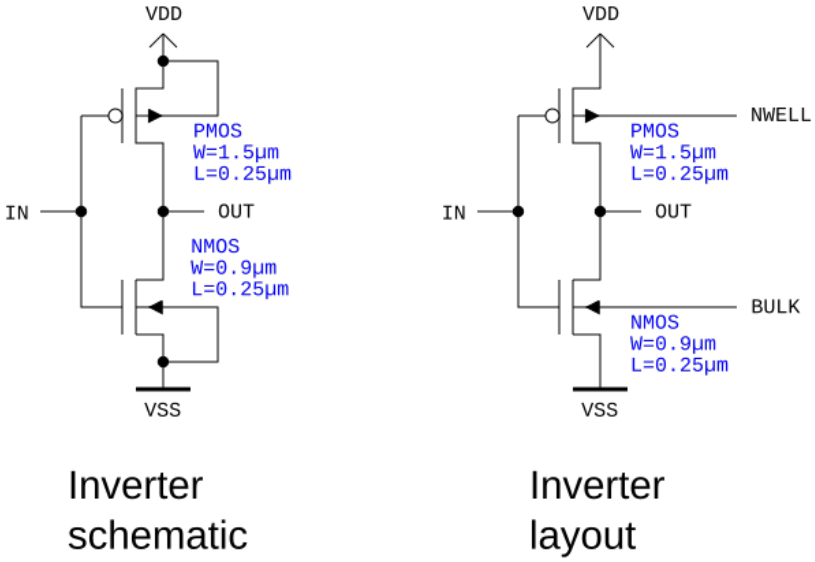


图 5.45

为使版图与 schematic 对齐，可以显式地把 bulk 与 VSS pin 相连，n-well 与 VDD 同理。这套机制与前面讲的 `connect_implicit` 类似，区别在于它能连接名字不同的 net。

要在上面的例子里建立显式连接，先确保 n-well 与 bulk 有合适的名字：n-well 通过在 n-well 岛上放 label 命名（例如 `NWELL`）；bulk 并不是真正带 Polygon 的版图图层，因此用 `connect_global` 给它一个全局名。

下面的代码把 `INV` Cell 内的 bulk net 与 `VSS` 连起来：

```
connect_global(bulk, "BULK") ... connect_explicit("INV", [ "BULK", "
    VSS" ])
```

注意该规则会形成一条名为 `BULK,VSS` 的新 net，把两条子 net 合并。

Cell 名可以是通配符，例如 `INV*` 会把该规则应用到所有以 `INV` 开头的 Cell。Cell 通配符不是必须的，省略时规则作用于所有 Cell。

与隐式连接一样，显式连接也会检查其在上一层是否落实——无论是物理连接，还是另一条显式/隐式连接。

显式连接还会对所涉及 net 名隐含地建立隐式连接，因此上例中 `VSS` 的各个片段

即便物理上未相连也被视为相连，同样会检查这些连接是否在更高层落实。

同样，`connect_explicit`语句必须写在 netlist 提取之前，通常在`connect`语句之前或紧随其后。

5.7.5 软连接

软连接（soft connection）用来找出“信号甚至电源经由高阻通路走线”这类布线问题。高阻通路可以是经多晶硅、注入区、well 或衬底形成的连接，这类区域的电阻常比金属高几个数量级。

举一例：

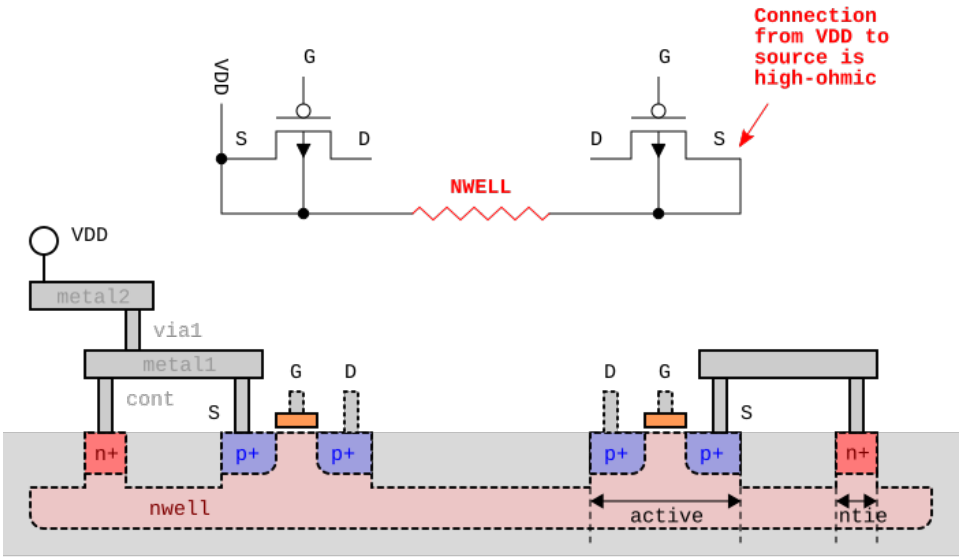


图 5.46

这是教科书式的平面 CMOS 工艺，两个 PMOS 器件位于同一个 n-well 中，例如可分属两组反相器。两个 PMOS 的源极都需要接到 VDD，n-well 区域同样需要接到 VDD。

这样的工艺叠层可以这样描述连接关系：

```
# Input layers nwell = ... active = ... pplus = ... nplus = ... poly
   = ... contact = ... metal1 = ... via1 = ... metal2 = ... #
   computed layers (nactive, pactive) = active.and_not(nwell) # PMOS
   and NMOS source/drain regions psd = (nactive & pplus) - poly nsd =
    (pactive & nplus) - poly # n tie and p tie (nwell and substrate
   contact) ntie = nactive & nplus ptie = pactive & pplus #
   connections # nwell connections connect(ntie, nwell) connect(
   contact, ntie) # substrate connections connect_global(ptie, "BULK
```

```
") connect(contact, ptie) # device connections connect(contact,
psd) connect(contact, nsd) connect(contact, poly) # metal
connections connect(metal1, contact) connect(via1, metal1) connect
(metal2, via1)
```

但问题来了：如图所示，左侧 PMOS 的源极确实接到了 VDD，右侧那个却缺少通往 VDD 的金属连接。仅从纯连通性看，这个晶体管的源极仍然连到了 VDD——只是经由了高阻的 n-well。

解决办法是引入软连接：把 `connect` 换成 `soft_connect`，把 `connect_global` 换成 `soft_connect_global`（见 `soft_connect` 与 `soft_connect_global`）。第一个图层是“上层”，第二个是“下层”。

软连接可以理解为有方向的连接：电流只能从上层流向下层，反之不行。因此只有当软连接的两个上层 terminal 都接到同一条物理 net 上时，才构成真正的连接。

要解决 n-well 的问题，把 n-tie 到 n-well 的那条连接语句改成 `soft_connect`：

```
soft_connect(ntie, nwell)
```

此时上面的图就变成这样：

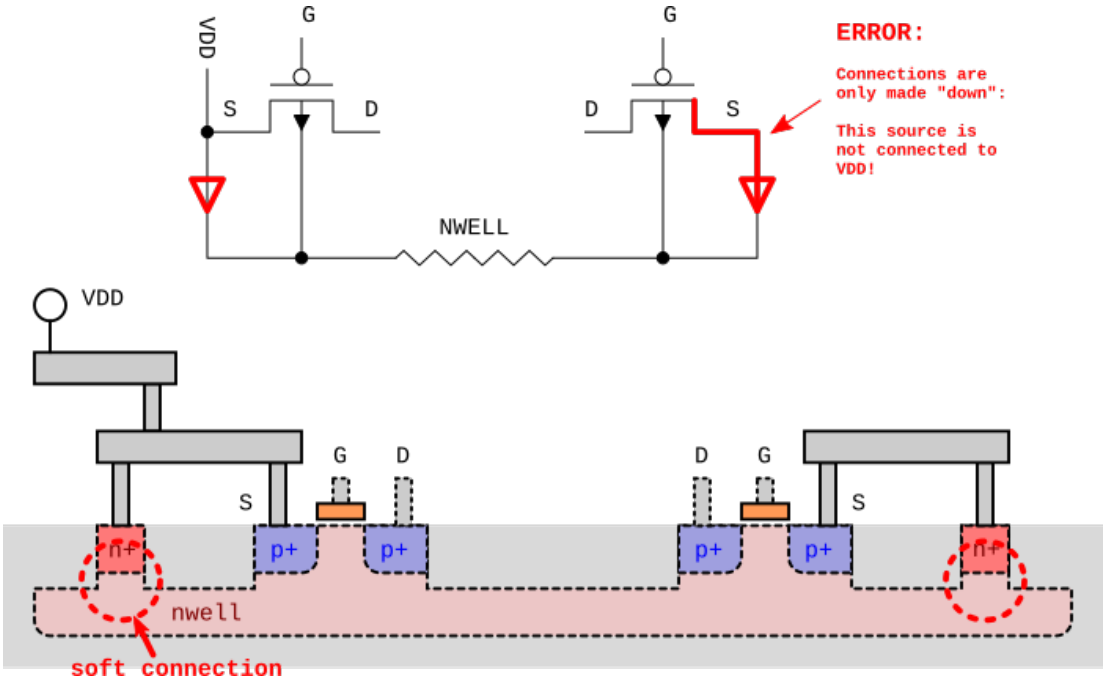


图 5.47

有了这个定义，netlist 提取器就能发现该故障并给出警告或错误（顶层模式下为错误）。警告显示在日志标签页，指出布线不完整，并给出被软连接分开的各子 net 的细节。

为完整起见，还需要纳入其它软连接，例如经衬底（到 BULK net 的全局软连接）、经源漏注入区、经 tie 注入区以及经多晶硅的连接：

```
# Input layers nwell = ... active = ... pplus = ... nplus = ... poly
    = ... contact = ... metal1 = ... via1 = ... metal2 = ... #
    computed layers (nactive, pactive) = active.and_not(nwell) # PMOS
    and NMOS source/drain regions psd = (nactive & pplus) - poly nsd =
     (pactive & nplus) - poly # n tie and p tie (nwell and substrate
    contact) ntie = nactive & nplus ptie = pactive & pplus #
    connections # nwell connections soft_connect(ntie, nwell)
    soft_connect(contact, ntie) # substrate connections
    soft_connect_global(ptie, "BULK") soft_connect(contact, ptie) #
    device connections soft_connect(contact, psd) soft_connect(contact
    , nsd) soft_connect(contact, poly) # metal connections connect(
    metal1, contact) connect(via1, metal1) connect(metal2, via1)
```

如代码所示，可以指定多条软连接；从检查的角度看，所有软连接都是同一种性质。

注意，两条方向相反的软连接会相互抵消，从而变成一条硬连接：

```
soft_connect(a, b) soft_connect(b, a)
```

注意：因此务必留意软连接的方向——上层先写高导电（低阻）图层，下层后写低导电（高阻）图层。

5.7.5.1 软连接与“must connect”网

软连接与 must connect 网（即 `connect_explicit` 与 `connect_implicit`）目的相同：发现不完整的布线。二者通常配合使用，例如对 VDD net 做 `connect_implicit`，并在 schematic 未把 VDD 与 NWELL 相连时对 VDD 与 NWELL 做 `connect_explicit`。

软连接在执行 `connect_explicit` 与 `connect_implicit` 之前就被检查，因此软连接错误无法通过声明 must connect 来掩盖。反过来这也不成问题，因为两种检查都会给出警告或错误（顶层模式下）。

5.8 LVS 比对

实际的比对步骤相当简单：只要已经设置好提取（`extract_devices`）、连接关系（`connect`、`connect_global`、`connect_implicit`），并提供了参考 netlist，就可以直接调用 `compare`。

```
compare
```

该方法（`compare`）会提取 netlist（若尚未提取）并与 schematic 比对，成功返回 true，否则返回 false——若你想据此做进一步处理会很有用。

比对步骤可以通过设置提示（hint）来配置。

5.8.1 net 等同提示

把两条 net 声明为等同有时很有用，至少在调试阶段如此：比对算法随后可以据此推断失配的真正原因。例如接线名与 schematic 中的名字不一致时，这样可以排除干扰。

```
same_nets("LOGIC", "VDD", "VDD:P")
```

这个例子假定电源网在版图中标为 `VDD`、在 schematic 中叫 `VDD:P`。正式验证时不要把这条语句留在脚本里。

`same_nets`也可以用来要求指定的 net 必须匹配。这在顶层很有用，例如检查每个 pad 上分配的 net 是否对应，从而保证 pad 接对。

```
same_nets!("CHIP", "VDD", "VDD")
```

带感叹号的写法含义不同：若版图或 schematic 中缺少 `VDD`网，或两者不匹配，就报告 net 失配。上面的写法还可以进一步组合使用。

```
same_nets!("CHIP", "VDD")
```

circuit 名与 net 名都支持通配符。下面的例子要求 `INVERTER`circuit 中所有以 `PAD` 开头的 net 在版图与 schematic 中都要有对应：

```
same_nets!("TOP", "PAD*")
```

因此，若版图中有 PAD1 而 schematic 中没有，就是错误；若两边都有 PAD2 但彼此不匹配，同样是错误。

`same_nets`与 `same_nets!`可以出现在 LVS 脚本的任何位置。

关于 `same_nets`的更多信息见 `same_nets`与 `same_nets!`。

5.8.2 circuit 等同提示

默认情况下，同名的 circuit 被视为等价。若不成立，可以用 `same_circuit`函数指定等价关系：

```
same_circuits("CIRCUIT_IN_LAYOUT", "CIRCUIT_IN_SCHEMATIC")
```

把 circuit 声明为“相同”并不意味着跳过比较，它仍然会被比较；该函数只是提示到哪里去找比对目标。

5.8.3 器件类等同提示

默认情况下同名的器件类被视为等价。若不成立，可以用 `same_device_classes` 指定：

```
same_device_classes("PMOS_IN_LAYOUT", "PMOS_IN_SCHEMATIC")
    same_device_classes("NMOS_IN_LAYOUT", "NMOS_IN_SCHEMATIC")
```

该方法也可以多次调用，建立多对多的等价关系：

```
same_device_classes("POLYRES", "RES") same_device_classes("WELLRES",
    "RES")
```

若某一侧给 `nil`，则相应器件基本被忽略：

```
# ignores "POLYRES" devices: same_device_classes("POLYRES", nil)
```

5.8.4 容差

比对器件参数时，默认要求严格相等。但对电阻这类器件，画出来的值通常很难与 schematic 中的值精确一致（尺寸会有取整、近似等误差）。

解决办法是指定参数容差。容差可以用绝对值或相对值给出：绝对容差表示版图参数可以在目标值上下偏差这么多；

相对容差则由目标值乘以该系数得出，例如 0.05 表示允许 5% 的偏差。

也可以同时给出绝对与相对容差，此时两者相加，允许的偏差更大。

指定绝对容差用 `tolerance` 函数：

```
tolerance("NMOS", "L", 0.05)
```

两个参数分别是器件类名与要应用容差的参数名。上例中为 `L` 指定了 50 纳米的容差。

相对容差作为第四个参数给出；可以把绝对容差设为 0，只保留相对容差。下面给出 1% 的容差：

```
tolerance("NMOS", "L", 0.0, 0.01)
```

容差还有更明确的写法：

```
tolerance("NMOS", "L", :absolute => 0.05)
```

或者：

```
tolerance("NMOS", "L", :relative => 0.01)
```

同时给出两者即表示绝对加相对容差。下面的调用为 `NMOS` 的 `L` 参数给出 50 纳米绝对容差与 1% 相对容差。

```
tolerance("NMOS", "L", 0.05, 0.01) tolerance("NMOS", "L", :absolute
    => 0.05, :relative => 0.01)
```

5.8.5 忽略参数

也可以让某些器件参数完全不参与 netlist 比对。例如不想比较 `NMOS` 的 `L` 参数，可以这样写：

```
ignore_parameter("NMOS", "L")
```

这条语句可以放在脚本中 `compare` 之前的任何位置。

默认只比较“主要”（primary）参数。以电阻为例，`R` 是主要参数，`L`、`W`、`A`、`P` 都不是。用 `tolerance` 启用某个参数会把它一并纳入比较。

5.8.6 启用与禁用参数

如前所述，某些器件参数是主要的，另一些不是。例如对电阻器件，`R`（阻值）是主要参数，而 `L`（长度）默认不是。

```
enable_parameter("RES", "L")
```

这带来两点影响：其一，`L` 参数会被写进 SPICE 输出 netlist；其二，它也会与 schematic 中的 `L` 参数比较。

相应地，主要参数也可以用如下方式禁用：

```
disable_parameter("RES", "R")
```

若该参数另有 `tolerance` 或 `ignore_parameter` 设置，或安装了自定义的器件比较器，则上述行为会被覆盖。netlist 输出仅在该参数参与比较时才受影响。

5.8.7 pin 互换

当逻辑元件的输入在逻辑上等价、物理上却不同时，pin 互换（pin swapping）会很有用，例如 CMOS NAND 门：两个输入在逻辑上可交换，但版图上左右两个输入的实现并不相同。

```
equivalent_pins("NAND_GATE", "A", "B")
```

第一个参数是版图 netlist 中的 circuit 名——只能在版图侧声明等价，不能对参考 schematic 声明。其后可以列出多个要互换的 pin。

5.8.8 消去电容与电阻

该功能可以消去“开路”的电阻与电容。目前串联电阻还不能被消去（短接）。

要消去所有阻值高于阈值的电阻，用 `max_res`。下面会消去所有阻值大于等于 1 kOhm 的电阻：

```
max_res(1000)
```

要消去所有容值低于阈值的电容，用 `min_caps`。下面会消去所有容值小于等于 0.1 fF 的电容：

```
min_caps(1e-16)
```

5.8.9 检查 pin 标注

LVS 本质上不依赖名字，除了消除歧义之外并不考虑 net 名：拓扑匹配优先。因此若 net 没有恰当地打 label，LVS 默认会把同拓扑但接错位置的 net 判为匹配。

这在涉及对外连接（例如 pad）时可能带来不良后果——芯片 pad 接错是致命错误。为缓解此问题，可用 `flag_mismatched_nets`功能检查名字不一致的 net。

该函数需要在比对步骤之后调用，即：

```
compare flag_missing_ports
```

若只想在比较成功时抑制伪错误，也可以这样写：

```
success = compare success && flag_missing_ports
```

该函数取 schematic 的顶层 circuit，检查所有连到 pin 的 net：核对对应版图 net 的名字（label），若名字不一致就报告。

在设计过程中自下而上推进时使用该功能，可以确保所有 pin 都被恰当标注。若你在顶层 circuit 中用 pin 来描述对外接口，这一点尤其重要。

注意，SPICE netlist 的顶层 circuit 也可能没有 pin（例如顶层不是 SUBCKT 时），这种情况下该函数不会做检查。

你可以使用

```
schematic.make_top_level_pins
```

在没有 pin 的情况下创建 pin。但这样会把每条 net 都变成 pin，从而强迫你标注顶层 circuit 中的每一条 net，因此更好的做法还是显式提供 pin。

5.8.10 比对与网表层次

好的版图是按层次构建的，netlist 比对也能利用层次结构。所谓“按层次”，是指 circuit 由若干 Cell 构成，而这些 Cell 又对应 subcircuit，形成与版图一致的多级结构。

给定层次化的版图与 schematic netlist，比对算法将自下而上进行：先比较叶 circuit（不含 subcircuit 调用的 circuit），若它们匹配，再继续向上。

要启用层次化提取，必须使用 `deep`模式（`deep`）；若缺少该语句，版图 netlist 就是平坦的（不含 subcircuit）。

（`align`）：它会把在另一侧 netlist 中不存在的 circuit 从版图或 schematic netlist 中移除。版图中常有只起辅助作用的 Cell，它们在 schematic 中并没有对应电路。

一般建议把 `align` 放在 `netlist.simplify` 之类 netlist 处理以及 `compare` 之前。

（`align`）还有一个很有用的副作用：它会去掉任一侧顶层 circuit 之上的 circuit，从而最终从 circuit 树中切出一棵子树，只保留两侧共有的部分。

5.8.11 比对算法的工作方式

netlist 比对算法的大致流程如下：

```
foreach circuit bottom up: if matching circuit found in reference
   netlist: if all subcircuits have been matched and pin matching has
    been established for them: compare net graph locally from this
   circuit else: skip circuit with warning else: issue a circuit
   mismatch error
```

这一流程的后果是：一旦某个 circuit 的 pin 无法与参考 circuit 的 pin 对应，或无法与该层次上的其它 circuit 对应，比对就不再处理其父级 circuit。

局部 net 图比对算法是一种回溯算法，并借助拓扑 net 分类进行提示。拓扑分类基于最近 net 邻域，也就是从某条 net 出发，经过器件与连线能到达哪些 net、各需几跳。

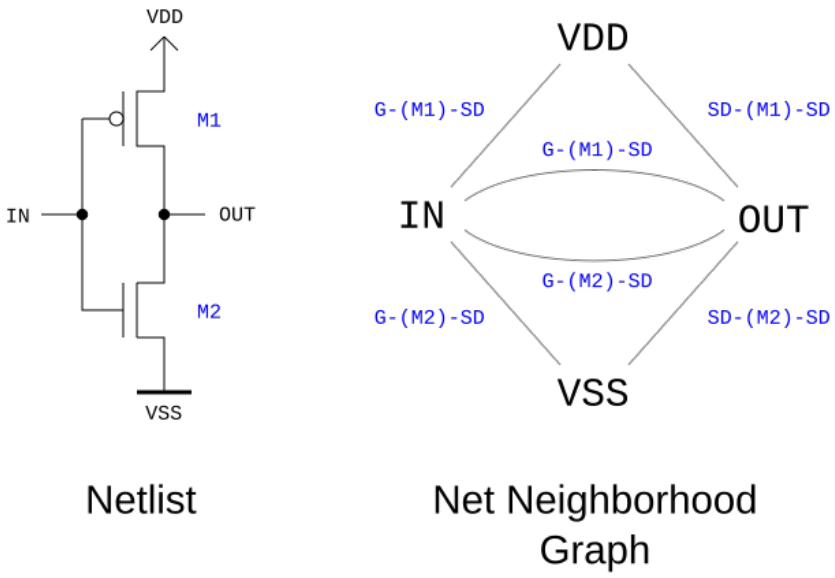


图 5.48

这里 IN net 的邻域是：经“栅极到源漏、越过 M1”的一跳到达 VDD；经“栅极到源漏、越过 M1 与 M2”的两跳到达 OUT；再经另一条路径到达 VSS。

在大型 netlist 中，算法会先按邻域度量尝试无歧义地匹配 net 并登记为成对 net，这些配对往往足以推断出其余 net 的对应关系，从而大幅缩小搜索空间。

5.9 LVS 网表调整

netlist 调整（tweaking）对于把 netlist 标准化很重要。若不做调整，提取出的 netlist 里可能含有冗余元素，或含有 schematic 中根本找不到对应的东西，比对就很难通过。

netlist 调整作用于已提取的 netlist 对象，可用 `netlist` 函数取得；该函数若尚未提取会先触发提取。

调整也可以作用在 schematic netlist 上，例如要把名为 `NMOS` 的模型 subcircuit 打平，可以这样写：

```
schematic.flatten_circuit("NMOS")
```

5.9.1 生成顶层 pin

提取出的 circuit 在顶层没有 pin，因为提取器无从得知该 circuit 对外接到哪里。比对函数不会去猜，因此顶层若无 pin，就只能做拓扑层面的比对。

KLayout 提供一个用简单启发式建立顶层 pin 的函数：顶层 circuit 中每一条有名字（有 label）的 net 都会生成一个 pin：

```
netlist.make_top_level_pins
```

5.9.2 器件合并

器件合并对于那些在版图中并非以单一实体表示的器件很重要，例如：

- 指状 MOS 晶体管：宽度很大的 MOS 晶体管常被拆成多段，以降低栅极与扩散区的寄生电阻、电容。
- 串联电阻：大电阻常被分成多条、再以折线（meander）方式串起来。从器件角度看，这样的电阻由若干段串联电阻构成。
- 阵列电容：大电容常被拆成多个小电容排成阵列并联，以控制寄生串联电阻。

这些情况下，schematic 通常把它们归并成单个器件，用集中参数值（总电阻、总电容、晶体管宽度）表示，因此提取网表也要做同样的合并。

```
netlist.combine_devices
```

合并串联器件可能留下悬空 net（原本连接这些器件的 net）。这类 net 可以用 `purge` 移除。见 `purge`，它是若干方法的封装。

建议在这一步之前先运行 `make_toplevel_pins` 与 `purge`（见下文）。

5.9.3 打平 circuit（消去层次）

经常需要打平那些并不代表某个组织层次的 circuit——它们只是别的什么东西的外壳。版图中器件常常由 PCell 或单元实现，这些在 schematic 中并没有对应层次。

要从提取网表中打平某个 circuit，用：

```
netlist.flatten_circuit("CIRCUIT_NAME")
```

`flatten_circuit`的参数是 glob 通配符（类 shell）。例如 `NMOS*`会打平所有以 `NMOS`开头的 circuit。

5.9.4 自动打平 circuit（网表对齐）

除了显式打平，也可以通过 `align`方法自动打平。

这一步是可选的，但很有用：它会找出没有对应 schematic circuit 的版图 Cell 并打平它们。“打平”意味着其内容被并入父级，本身不再作为一层出现。

`align`同样会打平那些没有对应版图 Cell 的 schematic circuit。

```
align
```

5.9.5 黑盒化（circuit 抽象）

circuit 抽象是一种降低验证开销的手段。在早期阶段，把某个 Cell 替换成简化版本或只留下 pin 的框架往往很有用。

与此相关的一个有用方法是 `blank_circuit`：清空某 circuit 的内部结构。此后比对算法会把两边都视为黑盒，只比较接口。

要清空某个 circuit 的内部，用该方法：

```
netlist.blank_circuit("CIRCUIT_NAME") schematic.blank_circuit("
    CIRCUIT_NAME")
```

注意：在本版本中，blank_circuit 要在 purge 或 simplify 之前使用（均为 LVS 脚本函数）。blank_circuit 会设置一个标志，阻止抽象 circuit 被清除。

它也有简写形式（`blank_circuit`）。与基于 netlist 的 `blank_circuit`不同，这个简写可以用在 LVS 脚本的任何位置：

```
blank_circuit("CIRCUIT_NAME")
```

两种情况下 `blank_circuit`的参数都是 glob 通配符（类 shell）。例如 `MEMORY*`会清空所有以 `MEMORY`开头的 circuit。

5.9.6 对称节点合并

有时版图里可以省掉一些连接，因为它们不承载电流。这能简化版图、允许更紧凑的排布，但最终会与 schematic 的连通性不一致。

来解决 `split_gates` 问题（见下文 `split_gates`）：在所示配置中，内部源漏节点是对称的，因此可以让 LVS 自动把它们视为同一节点。

```
join_symmetric_nets("NAND2")
```

不过还有一个更专门的特性（`split_gates`），它覆盖更多情形，但专门针对 MOS 器件。

5.9.7 分裂栅

下图描述了这种称为“分裂栅”（split gate）的情形。此例中 N1 与 N2 完全相同，互换它们不会改变电路的功能。

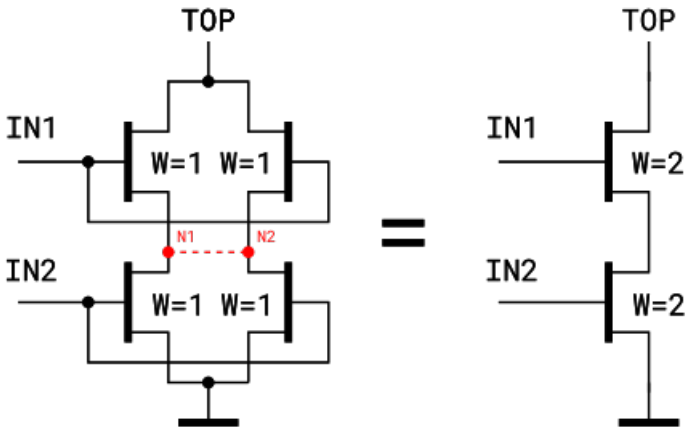


图 5.49

KLayout 提供一个特性（`split_gates`），会在 netlist 提取之后补上这类连接：

```
split_gates("NMOS")
```

该函数会分析提取网表中的所有 circuit，针对 `NMOS` 器件，把内部相关的分裂栅源漏节点连起来。若该函数找不到可连的分裂栅，则不影响结果。

器件名必须是 MOS3、MOS4、DMOS3 或 DMOS4 器件。所有相关器件的栅长必须完全一致；对 MOS4 与 DMOS4，同一栅极 net 上的所有器件还必须共享同一个 bulk net。

除器件名之外，还可以给一个 glob 风格的 circuit 通配符，此时分析只限于匹配该模式的 circuit。

`split_gates` 可以用在 LVS 脚本的任何位置。

5.9.8 清除冗余

提取网表中常含没有功能意义的元素：例如 via 单元会生成只有一个 pin、没有任何器件的 subcircuit；孤立的金属岛也会形成不含器件的 subcircuit。

(`purge`) 会移除所有悬空 net 与所有不含器件或 subcircuit 的 circuit；`purge_nets` 只清除悬空 net。悬空 net 指不连接任何器件或 subcircuit 的 net。

```
netlist.purge netlist.purge_nets
```

5.9.9 归一化封装（simplify）

`simplify` 是 `make_top_level_pins`、`purge`、`combine_devices` 与 `purge_nets` 的封装，按如下推荐顺序执行：

```
netlist.simplify
```

技术细节上，之所以把这序列里包含 `make_top_level_pins`，是因为有 pin 之后 net 就不算悬空，这样 `purge_nets` 会保留有 label 的 net 的 pin。

附录 A

许可证（GNU General Public License v3）

KLayout 及其文档的完整许可文本见随本工程附带的 `LICENSE-klayout.txt`；版权声明见 `COPYRIGHT-klayout.txt`。两者均为上游原文，未作改动。